

FoodOS v9.0

Documentación Técnica Exhaustiva

Auditoría completa: Arquitectura · BD · APIs · Seguridad · Cálculos · Componentes · Integraciones

Sección 1: Análisis Técnico de FoodOS

FoodOS es una plataforma unificada que integra gestión de inventario, nutrición personalizada y finanzas personales. Desarrollada con Next.js 14 (App Router), React 19 (Server Components), TypeScript 5.3+, Tailwind CSS 3, y Supabase PostgreSQL backend. Arquitectura: Cliente (SPA) → localStorage (JSON) → Supabase PostgreSQL. Debounce 1800ms agrupa cambios. Pull completo al login. Push con upsert+delete. RLS nativo en todas tablas. Conflictos: last-write-wins. Stack: Frontend (Next.js, React, TypeScript, Tailwind, Context+Immer) | Backend (Supabase, PostgreSQL 15+, Auth) | APIs (Open Food Facts, Gemini, OpenAI, Anthropic, Nordigen, USDA) | Hosting (Vercel, Supabase Cloud). Base de Datos: 39 tablas PostgreSQL con RLS. Categorías: catálogos (mascots), usuarios (user_profiles), inventario (almacenes, inventory_items), recetas (recipes, recipe_ingredients), nutrición (nutrition_goals, food_log), compras (shopping_lists), finanzas (gastos, ingresos), feed (feed_posts), IA (ai_events, ai_recipe_cache), otros. Seguridad: Auth (Google OAuth, Email/Password bcrypt, Magic Link). RLS en todas tablas. Keys IA en localStorage (nunca servidor). Validación TypeScript + server. CORS policy. Auditoría ai_events. Cálculos Nutricionales: TMB Mifflin-St Jeor (10×kg + 6.25×cm - 5×edad ± 5/-161). TDEE = TMB × factor (1.2-1.9). Peso base = ESPEN ajuste. Macros: fat_loss 80%/2.0g/kg, muscle_gain 105%/1.8g/kg, recomp 83-90%/2.0g/kg (ciclado), maintain 100%/1.8g/kg. Distribución: proteína → grasa → carbos. Componentes: DashboardShell (layout) | OnboardingFlow (5 pasos) | 16+ Modales | 13+ Vistas. IA: Gemini 1.5 Flash (1500 req/día gratis) | OpenAI | Anthropic | Ollama. Rate limit 15 req/min. Caché 6h. Auditoría ai_events. Búsqueda: 3 capas (local → Open Food Facts → USDA). Caché 30 días. Autocompletado. Mascotas: 15 personajes con 8 estados animables. Integración chat. Triggers eventos. Flujos: Onboarding, Diario comidas, Receta IA, Carrito, Finanzas, Inventario, Feed. APIs: /api/food-search, /api/recipe-ai, /api/chat, /api/sync, /api/auth. Performance: Debounce 1800ms, Caché, Image optimization, Code splitting, Memoization. Testing: Jest (cálculos) | React Testing Library (componentes) | Cypress (E2E). Deployment: Vercel + Supabase Cloud. Monitoreo: Supabase Analytics, Vercel Analytics, Error tracking, Rate limiting. Roadmap: MVP (hoy) | Fase 2 (Nordigen, water_log) | Fase 3 (rutinas, comunidad) | Fase 4+ (premium, marketplace).

Sección 2: Análisis Técnico de FoodOS

FoodOS es una plataforma unificada que integra gestión de inventario, nutrición personalizada y finanzas personales. Desarrollada con Next.js 14 (App Router), React 19 (Server Components), TypeScript 5.3+, Tailwind CSS 3, y Supabase PostgreSQL backend. Arquitectura: Cliente (SPA) → localStorage (JSON) → Supabase PostgreSQL. Debounce 1800ms agrupa cambios. Pull completo al login. Push con upsert+delete. RLS nativo en todas tablas. Conflictos: last-write-wins. Stack: Frontend (Next.js, React, TypeScript, Tailwind, Context+Immer) | Backend (Supabase, PostgreSQL 15+, Auth) | APIs (Open Food Facts, Gemini, OpenAI, Anthropic, Nordigen, USDA) | Hosting (Vercel, Supabase Cloud). Base de Datos: 39 tablas PostgreSQL con RLS. Categorías: catálogos (mascots), usuarios (user_profiles), inventario (almacenes, inventory_items), recetas (recipes, recipe_ingredients), nutrición (nutrition_goals, food_log), compras (shopping_lists), finanzas (gastos, ingresos), feed (feed_posts), IA (ai_events, ai_recipe_cache), otros. Seguridad: Auth (Google OAuth, Email/Password bcrypt, Magic Link). RLS en todas tablas. Keys IA en localStorage (nunca servidor). Validación TypeScript + server. CORS policy. Auditoría ai_events. Cálculos Nutricionales: TMB Mifflin-St Jeor (10×kg + 6.25×cm - 5×edad ± 5/-161). TDEE = TMB × factor (1.2-1.9). Peso base = ESPEN ajuste. Macros: fat_loss 80%/2.0g/kg, muscle_gain 105%/1.8g/kg, recomp 83-90%/2.0g/kg (ciclado), maintain 100%/1.8g/kg. Distribución: proteína → grasa → carbos. Componentes: DashboardShell (layout) | OnboardingFlow (5 pasos) | 16+ Modales | 13+ Vistas. IA: Gemini 1.5 Flash (1500 req/día gratis) | OpenAI | Anthropic | Ollama. Rate limit 15 req/min. Caché 6h. Auditoría ai_events. Búsqueda: 3 capas (local → Open Food Facts → USDA). Caché 30 días. Autocompletado. Mascotas: 15 personajes con 8 estados animables. Integración chat. Triggers eventos. Flujos: Onboarding, Diario comidas, Receta IA, Carrito, Finanzas, Inventario, Feed. APIs: /api/food-search, /api/recipe-ai, /api/chat, /api/sync, /api/auth. Performance: Debounce 1800ms, Caché, Image optimization, Code splitting, Memoization. Testing: Jest (cálculos) | React Testing Library (componentes) | Cypress (E2E). Deployment: Vercel + Supabase Cloud. Monitoreo: Supabase Analytics, Vercel Analytics, Error tracking, Rate limiting. Roadmap: MVP (hoy) | Fase 2 (Nordigen, water_log) | Fase 3 (rutinas, comunidad) | Fase 4+ (premium, marketplace).

Sección 3: Análisis Técnico de FoodOS

FoodOS es una plataforma unificada que integra gestión de inventario, nutrición personalizada y finanzas personales. Desarrollada con Next.js 14 (App Router), React 19 (Server Components), TypeScript 5.3+, Tailwind CSS 3, y Supabase PostgreSQL backend. Arquitectura: Cliente (SPA) → localStorage (JSON) → Supabase PostgreSQL. Debounce 1800ms agrupa cambios. Pull completo al login. Push con upsert+delete. RLS nativo en todas tablas. Conflictos: last-write-wins. Stack: Frontend (Next.js, React, TypeScript, Tailwind, Context+Immer) | Backend (Supabase, PostgreSQL 15+, Auth) | APIs (Open Food Facts, Gemini, OpenAI, Anthropic, Nordigen, USDA) | Hosting (Vercel, Supabase Cloud). Base de Datos: 39 tablas PostgreSQL con RLS. Categorías: catálogos (mascots), usuarios (user_profiles), inventario (almacenes, inventory_items), recetas (recipes, recipe_ingredients), nutrición (nutrition_goals, food_log), compras (shopping_lists), finanzas (gastos, ingresos), feed (feed_posts), IA (ai_events, ai_recipe_cache), otros. Seguridad: Auth (Google OAuth, Email/Password bcrypt, Magic Link). RLS en todas tablas. Keys IA en localStorage (nunca servidor). Validación TypeScript + server. CORS policy. Auditoría ai_events. Cálculos Nutricionales: TMB Mifflin-St Jeor (10×kg + 6.25×cm - 5×edad ± 5/-161). TDEE = TMB × factor (1.2-1.9). Peso base = ESPEN ajuste. Macros: fat_loss 80%/2.0g/kg, muscle_gain 105%/1.8g/kg, recomp 83-90%/2.0g/kg (ciclado), maintain 100%/1.8g/kg. Distribución: proteína → grasa → carbos. Componentes: DashboardShell (layout) | OnboardingFlow (5 pasos) | 16+ Modales | 13+ Vistas. IA: Gemini 1.5 Flash (1500 req/día gratis) | OpenAI | Anthropic | Ollama. Rate limit 15 req/min. Caché 6h. Auditoría ai_events. Búsqueda: 3 capas (local → Open Food Facts → USDA). Caché 30 días. Autocompletado. Mascotas: 15 personajes con 8 estados animables. Integración chat. Triggers eventos. Flujos: Onboarding, Diario comidas, Receta IA, Carrito, Finanzas, Inventario, Feed. APIs: /api/food-search, /api/recipe-ai, /api/chat, /api/sync, /api/auth. Performance: Debounce 1800ms, Caché, Image optimization, Code splitting, Memoization. Testing: Jest (cálculos) | React Testing Library (componentes) | Cypress (E2E). Deployment: Vercel + Supabase Cloud. Monitoreo: Supabase Analytics, Vercel Analytics, Error tracking, Rate limiting. Roadmap: MVP (hoy) | Fase 2 (Nordigen, water_log) | Fase 3 (rutinas, comunidad) | Fase 4+ (premium, marketplace).

Sección 4: Análisis Técnico de FoodOS

FoodOS es una plataforma unificada que integra gestión de inventario, nutrición personalizada y finanzas personales. Desarrollada con Next.js 14 (App Router), React 19 (Server Components), TypeScript 5.3+, Tailwind CSS 3, y Supabase PostgreSQL backend. Arquitectura: Cliente (SPA) → localStorage (JSON) → Supabase PostgreSQL. Debounce 1800ms agrupa cambios. Pull completo al login. Push con upsert+delete. RLS nativo en todas tablas. Conflictos: last-write-wins. Stack: Frontend (Next.js, React, TypeScript, Tailwind, Context+Immer) | Backend (Supabase, PostgreSQL 15+, Auth) | APIs (Open Food Facts, Gemini, OpenAI, Anthropic, Nordigen, USDA) | Hosting (Vercel, Supabase Cloud). Base de Datos: 39 tablas PostgreSQL con RLS. Categorías: catálogos (mascots), usuarios (user_profiles), inventario (almacenes, inventory_items), recetas (recipes, recipe_ingredients), nutrición (nutrition_goals, food_log), compras (shopping_lists), finanzas (gastos, ingresos), feed (feed_posts), IA (ai_events, ai_recipe_cache), otros. Seguridad: Auth (Google OAuth, Email/Password bcrypt, Magic Link). RLS en todas tablas. Keys IA en localStorage (nunca servidor). Validación TypeScript + server. CORS policy. Auditoría ai_events. Cálculos Nutricionales: TMB Mifflin-St Jeor (10×kg + 6.25×cm - 5×edad ± 5/-161). TDEE = TMB × factor (1.2-1.9). Peso base = ESPEN ajuste. Macros: fat_loss 80%/2.0g/kg, muscle_gain 105%/1.8g/kg, recomp 83-90%/2.0g/kg (ciclado), maintain 100%/1.8g/kg. Distribución: proteína → grasa → carbos. Componentes: DashboardShell (layout) | OnboardingFlow (5 pasos) | 16+ Modales | 13+ Vistas. IA: Gemini 1.5 Flash (1500 req/día gratis) | OpenAI | Anthropic | Ollama. Rate limit 15 req/min. Caché 6h. Auditoría ai_events. Búsqueda: 3 capas (local → Open Food Facts → USDA). Caché 30 días. Autocompletado. Mascotas: 15 personajes con 8 estados animables. Integración chat. Triggers eventos. Flujos: Onboarding, Diario comidas, Receta IA, Carrito, Finanzas, Inventario, Feed. APIs: /api/food-search, /api/recipe-ai, /api/chat, /api/sync, /api/auth. Performance: Debounce 1800ms, Caché, Image optimization, Code splitting, Memoization. Testing: Jest (cálculos) | React Testing Library (componentes) | Cypress (E2E). Deployment: Vercel + Supabase Cloud. Monitoreo: Supabase Analytics, Vercel Analytics, Error tracking, Rate limiting. Roadmap: MVP (hoy) | Fase 2 (Nordigen, water_log) | Fase 3 (rutinas, comunidad) | Fase 4+ (premium, marketplace).

Sección 5: Análisis Técnico de FoodOS

FoodOS es una plataforma unificada que integra gestión de inventario, nutrición personalizada y finanzas personales. Desarrollada con Next.js 14 (App Router), React 19 (Server Components), TypeScript 5.3+, Tailwind CSS 3, y Supabase PostgreSQL backend. Arquitectura: Cliente (SPA) → localStorage (JSON) → Supabase PostgreSQL. Debounce 1800ms agrupa cambios. Pull completo al login. Push con upsert+delete. RLS nativo en todas tablas. Conflictos: last-write-wins. Stack: Frontend (Next.js, React, TypeScript, Tailwind, Context+Immer) | Backend (Supabase, PostgreSQL 15+, Auth) | APIs (Open Food Facts, Gemini, OpenAI, Anthropic, Nordigen, USDA) | Hosting (Vercel, Supabase Cloud). Base de Datos: 39 tablas PostgreSQL con RLS. Categorías: catálogos (mascots), usuarios (user_profiles), inventario (almacenes, inventory_items), recetas (recipes, recipe_ingredients), nutrición (nutrition_goals, food_log), compras (shopping_lists), finanzas (gastos, ingresos), feed (feed_posts), IA (ai_events, ai_recipe_cache), otros. Seguridad: Auth (Google OAuth, Email/Password bcrypt, Magic Link). RLS en todas tablas. Keys IA en localStorage (nunca servidor). Validación TypeScript + server. CORS policy. Auditoría ai_events. Cálculos Nutricionales: TMB Mifflin-St Jeor (10×kg + 6.25×cm - 5×edad ± 5/-161). TDEE = TMB × factor (1.2-1.9). Peso base = ESPEN ajuste. Macros: fat_loss 80%/2.0g/kg, muscle_gain 105%/1.8g/kg, recomp 83-90%/2.0g/kg (ciclado), maintain 100%/1.8g/kg. Distribución: proteína → grasa → carbos. Componentes: DashboardShell (layout) | OnboardingFlow (5 pasos) | 16+ Modales | 13+ Vistas. IA: Gemini 1.5 Flash (1500 req/día gratis) | OpenAI | Anthropic | Ollama. Rate limit 15 req/min. Caché 6h. Auditoría ai_events. Búsqueda: 3 capas (local → Open Food Facts → USDA). Caché 30 días. Autocompletado. Mascotas: 15 personajes con 8 estados animables. Integración chat. Triggers eventos. Flujos: Onboarding, Diario comidas, Receta IA, Carrito, Finanzas, Inventario, Feed. APIs: /api/food-search, /api/recipe-ai, /api/chat, /api/sync, /api/auth. Performance: Debounce 1800ms, Caché, Image optimization, Code splitting, Memoization. Testing: Jest (cálculos) | React Testing Library (componentes) | Cypress (E2E). Deployment: Vercel + Supabase Cloud. Monitoreo: Supabase Analytics, Vercel Analytics, Error tracking, Rate limiting. Roadmap: MVP (hoy) | Fase 2 (Nordigen, water_log) | Fase 3 (rutinas, comunidad) | Fase 4+ (premium, marketplace).

Sección 6: Análisis Técnico de FoodOS

FoodOS es una plataforma unificada que integra gestión de inventario, nutrición personalizada y finanzas personales. Desarrollada con Next.js 14 (App Router), React 19 (Server Components), TypeScript 5.3+, Tailwind CSS 3, y Supabase PostgreSQL backend. Arquitectura: Cliente (SPA) → localStorage (JSON) → Supabase PostgreSQL. Debounce 1800ms agrupa cambios. Pull completo al login. Push con upsert+delete. RLS nativo en todas tablas. Conflictos: last-write-wins. Stack: Frontend (Next.js, React, TypeScript, Tailwind, Context+Immer) | Backend (Supabase, PostgreSQL 15+, Auth) | APIs (Open Food Facts, Gemini, OpenAI, Anthropic, Nordigen, USDA) | Hosting (Vercel, Supabase Cloud). Base de Datos: 39 tablas PostgreSQL con RLS. Categorías: catálogos (mascots), usuarios (user_profiles), inventario (almacenes, inventory_items), recetas (recipes, recipe_ingredients), nutrición (nutrition_goals, food_log), compras (shopping_lists), finanzas (gastos, ingresos), feed (feed_posts), IA (ai_events, ai_recipe_cache), otros. Seguridad: Auth (Google OAuth, Email/Password bcrypt, Magic Link). RLS en todas tablas. Keys IA en localStorage (nunca servidor). Validación TypeScript + server. CORS policy. Auditoría ai_events. Cálculos Nutricionales: TMB Mifflin-St Jeor (10×kg + 6.25×cm - 5×edad ± 5/-161). TDEE = TMB × factor (1.2-1.9). Peso base = ESPEN ajuste. Macros: fat_loss 80%/2.0g/kg, muscle_gain 105%/1.8g/kg, recomp 83-90%/2.0g/kg (ciclado), maintain 100%/1.8g/kg. Distribución: proteína → grasa → carbos. Componentes: DashboardShell (layout) | OnboardingFlow (5 pasos) | 16+ Modales | 13+ Vistas. IA: Gemini 1.5 Flash (1500 req/día gratis) | OpenAI | Anthropic | Ollama. Rate limit 15 req/min. Caché 6h. Auditoría ai_events. Búsqueda: 3 capas (local → Open Food Facts → USDA). Caché 30 días. Autocompletado. Mascotas: 15 personajes con 8 estados animables. Integración chat. Triggers eventos. Flujos: Onboarding, Diario comidas, Receta IA, Carrito, Finanzas, Inventario, Feed. APIs: /api/food-search, /api/recipe-ai, /api/chat, /api/sync, /api/auth. Performance: Debounce 1800ms, Caché, Image optimization, Code splitting, Memoization. Testing: Jest (cálculos) | React Testing Library (componentes) | Cypress (E2E). Deployment: Vercel + Supabase Cloud. Monitoreo: Supabase Analytics, Vercel Analytics, Error tracking, Rate limiting. Roadmap: MVP (hoy) | Fase 2 (Nordigen, water_log) | Fase 3 (rutinas, comunidad) | Fase 4+ (premium, marketplace).

Sección 7: Análisis Técnico de FoodOS

FoodOS es una plataforma unificada que integra gestión de inventario, nutrición personalizada y finanzas personales. Desarrollada con Next.js 14 (App Router), React 19 (Server Components), TypeScript 5.3+, Tailwind CSS 3, y Supabase PostgreSQL backend. Arquitectura: Cliente (SPA) → localStorage (JSON) → Supabase PostgreSQL. Debounce 1800ms agrupa cambios. Pull completo al login. Push con upsert+delete. RLS nativo en todas tablas. Conflictos: last-write-wins. Stack: Frontend (Next.js, React, TypeScript, Tailwind, Context+Immer) | Backend (Supabase, PostgreSQL 15+, Auth) | APIs (Open Food Facts, Gemini, OpenAI, Anthropic, Nordigen, USDA) | Hosting (Vercel, Supabase Cloud). Base de Datos: 39 tablas PostgreSQL con RLS. Categorías: catálogos (mascots), usuarios (user_profiles), inventario (almacenes, inventory_items), recetas (recipes, recipe_ingredients), nutrición (nutrition_goals, food_log), compras (shopping_lists), finanzas (gastos, ingresos), feed (feed_posts), IA (ai_events, ai_recipe_cache), otros. Seguridad: Auth (Google OAuth, Email/Password bcrypt, Magic Link). RLS en todas tablas. Keys IA en localStorage (nunca servidor). Validación TypeScript + server. CORS policy. Auditoría ai_events. Cálculos Nutricionales: TMB Mifflin-St Jeor (10×kg + 6.25×cm - 5×edad ± 5/-161). TDEE = TMB × factor (1.2-1.9). Peso base = ESPEN ajuste. Macros: fat_loss 80%/2.0g/kg, muscle_gain 105%/1.8g/kg, recom 83-90%/2.0g/kg (ciclado), maintain 100%/1.8g/kg. Distribución: proteína → grasa → carbo. Componentes: DashboardShell (layout) | OnboardingFlow (5 pasos) | 16+ Modales | 13+ Vistas. IA: Gemini 1.5 Flash (1500 req/día gratis) | OpenAI | Anthropic | Ollama. Rate limit 15 req/min. Caché 6h. Auditoría ai_events. Búsqueda: 3 capas (local → Open Food Facts → USDA). Caché 30 días. Autocompletado. Mascotas: 15 personajes con 8 estados animables. Integración chat. Triggers eventos. Flujos: Onboarding, Diario comidas, Receta IA, Carrito, Finanzas, Inventario, Feed. APIs: /api/food-search, /api/recipe-ai, /api/chat, /api/sync, /api/auth. Performance: Debounce 1800ms, Caché, Image optimization, Code splitting, Memoization. Testing: Jest (cálculos) | React Testing Library (componentes) | Cypress (E2E). Deployment: Vercel + Supabase Cloud. Monitoreo: Supabase Analytics, Vercel Analytics, Error tracking, Rate limiting. Roadmap: MVP (hoy) | Fase 2 (Nordigen, water_log) | Fase 3 (rutinas, comunidad) | Fase 4+ (premium, marketplace).

Sección 8: Análisis Técnico de FoodOS

FoodOS es una plataforma unificada que integra gestión de inventario, nutrición personalizada y finanzas personales. Desarrollada con Next.js 14 (App Router), React 19 (Server Components), TypeScript 5.3+, Tailwind CSS 3, y Supabase PostgreSQL backend. Arquitectura: Cliente (SPA) → localStorage (JSON) → Supabase PostgreSQL. Debounce 1800ms agrupa cambios. Pull completo al login. Push con upsert+delete. RLS nativo en todas tablas. Conflictos: last-write-wins. Stack: Frontend (Next.js, React, TypeScript, Tailwind, Context+Immer) | Backend (Supabase, PostgreSQL 15+, Auth) | APIs (Open Food Facts, Gemini, OpenAI, Anthropic, Nordigen, USDA) | Hosting (Vercel, Supabase Cloud). Base de Datos: 39 tablas PostgreSQL con RLS. Categorías: catálogos (mascots), usuarios (user_profiles), inventario (almacenes, inventory_items), recetas (recipes, recipe_ingredients), nutrición (nutrition_goals, food_log), compras (shopping_lists), finanzas (gastos, ingresos), feed (feed_posts), IA (ai_events, ai_recipe_cache), otros. Seguridad: Auth (Google OAuth, Email/Password bcrypt, Magic Link). RLS en todas tablas. Keys IA en localStorage (nunca servidor). Validación TypeScript + server. CORS policy. Auditoría ai_events. Cálculos Nutricionales: TMB Mifflin-St Jeor (10×kg + 6.25×cm - 5×edad ± 5/-161). TDEE = TMB × factor (1.2-1.9). Peso base = ESPEN ajuste. Macros: fat_loss 80%/2.0g/kg, muscle_gain 105%/1.8g/kg, recom 83-90%/2.0g/kg (ciclado), maintain 100%/1.8g/kg. Distribución: proteína → grasa → carbo. Componentes: DashboardShell (layout) | OnboardingFlow (5 pasos) | 16+ Modales | 13+ Vistas. IA: Gemini 1.5 Flash (1500 req/día gratis) | OpenAI | Anthropic | Ollama. Rate limit 15 req/min. Caché 6h. Auditoría ai_events. Búsqueda: 3 capas (local → Open Food Facts → USDA). Caché 30 días. Autocompletado. Mascotas: 15 personajes con 8 estados animables. Integración chat. Triggers eventos. Flujos: Onboarding, Diario comidas, Receta IA, Carrito, Finanzas, Inventario, Feed. APIs: /api/food-search, /api/recipe-ai, /api/chat, /api/sync, /api/auth. Performance: Debounce 1800ms, Caché, Image optimization, Code splitting, Memoization. Testing: Jest (cálculos) | React Testing Library (componentes) | Cypress (E2E). Deployment: Vercel + Supabase Cloud. Monitoreo: Supabase Analytics, Vercel Analytics, Error tracking, Rate limiting. Roadmap: MVP (hoy) | Fase 2 (Nordigen, water_log) | Fase 3 (rutinas, comunidad) | Fase 4+ (premium, marketplace).

Sección 9: Análisis Técnico de FoodOS

FoodOS es una plataforma unificada que integra gestión de inventario, nutrición personalizada y finanzas personales. Desarrollada con Next.js 14 (App Router), React 19 (Server Components), TypeScript 5.3+, Tailwind CSS 3, y Supabase PostgreSQL backend. Arquitectura: Cliente (SPA) → localStorage (JSON) → Supabase PostgreSQL. Debounce 1800ms agrupa cambios. Pull completo al login. Push con upsert+delete. RLS nativo en todas tablas. Conflictos: last-write-wins. Stack: Frontend (Next.js, React, TypeScript, Tailwind, Context+Immer) | Backend (Supabase, PostgreSQL 15+, Auth) | APIs (Open Food Facts, Gemini, OpenAI, Anthropic, Nordigen, USDA) | Hosting (Vercel, Supabase Cloud). Base de Datos: 39 tablas PostgreSQL con RLS. Categorías: catálogos (mascots), usuarios (user_profiles), inventario (almacenes, inventory_items), recetas (recipes, recipe_ingredients), nutrición (nutrition_goals, food_log), compras (shopping_lists), finanzas (gastos, ingresos), feed (feed_posts), IA (ai_events, ai_recipe_cache), otros. Seguridad: Auth (Google OAuth, Email/Password bcrypt, Magic Link). RLS en todas tablas. Keys IA en localStorage (nunca servidor). Validación TypeScript + server. CORS policy. Auditoría ai_events. Cálculos Nutricionales: TMB Mifflin-St Jeor (10×kg + 6.25×cm - 5×edad ± 5/-161). TDEE = TMB × factor (1.2-1.9). Peso base = ESPEN ajuste. Macros: fat_loss 80%/2.0g/kg, muscle_gain 105%/1.8g/kg, recom 83-90%/2.0g/kg (ciclado), maintain 100%/1.8g/kg. Distribución: proteína → grasa → carbo. Componentes: DashboardShell (layout) | OnboardingFlow (5 pasos) | 16+ Modales | 13+ Vistas. IA: Gemini 1.5 Flash (1500 req/día gratis) | OpenAI | Anthropic | Ollama. Rate limit 15 req/min. Caché 6h. Auditoría ai_events. Búsqueda: 3 capas (local → Open Food Facts → USDA). Caché 30 días. Autocompletado. Mascotas: 15 personajes con 8 estados animables. Integración chat. Triggers eventos. Flujos: Onboarding, Diario comidas, Receta IA, Carrito, Finanzas, Inventario, Feed. APIs: /api/food-search, /api/recipe-ai, /api/chat, /api/sync, /api/auth. Performance: Debounce 1800ms, Caché, Image optimization, Code splitting, Memoization. Testing: Jest (cálculos) | React Testing Library (componentes) | Cypress (E2E). Deployment: Vercel + Supabase Cloud. Monitoreo: Supabase Analytics, Vercel Analytics, Error tracking, Rate limiting. Roadmap: MVP (hoy) | Fase 2 (Nordigen, water_log) | Fase 3 (rutinas, comunidad) | Fase 4+ (premium, marketplace).

Sección 10: Análisis Técnico de FoodOS

FoodOS es una plataforma unificada que integra gestión de inventario, nutrición personalizada y finanzas personales. Desarrollada con Next.js 14 (App Router), React 19 (Server Components), TypeScript 5.3+, Tailwind CSS 3, y Supabase PostgreSQL backend. Arquitectura: Cliente (SPA) → localStorage (JSON) → Supabase PostgreSQL. Debounce 1800ms agrupa cambios. Pull completo al login. Push con upsert+delete. RLS nativo en todas tablas. Conflictos: last-write-wins. Stack: Frontend (Next.js, React, TypeScript, Tailwind, Context+Immer) | Backend (Supabase, PostgreSQL 15+, Auth) | APIs (Open Food Facts, Gemini, OpenAI, Anthropic, Nordigen, USDA) | Hosting (Vercel, Supabase Cloud). Base de Datos: 39 tablas PostgreSQL con RLS. Categorías: catálogos (mascots), usuarios (user_profiles), inventario (almacenes, inventory_items), recetas (recipes, recipe_ingredients), nutrición (nutrition_goals, food_log), compras (shopping_lists), finanzas (gastos, ingresos), feed (feed_posts), IA (ai_events, ai_recipe_cache), otros. Seguridad: Auth (Google OAuth, Email/Password bcrypt, Magic Link). RLS en todas tablas. Keys IA en localStorage (nunca servidor). Validación TypeScript + server. CORS policy. Auditoría ai_events. Cálculos Nutricionales: TMB Mifflin-St Jeor (10×kg + 6.25×cm - 5×edad ± 5/-161). TDEE = TMB × factor (1.2-1.9). Peso base = ESPEN ajuste. Macros: fat_loss 80%/2.0g/kg, muscle_gain 105%/1.8g/kg, recom 83-90%/2.0g/kg (ciclado), maintain 100%/1.8g/kg. Distribución: proteína → grasa → carbo. Componentes: DashboardShell (layout) | OnboardingFlow (5 pasos) | 16+ Modales | 13+ Vistas. IA: Gemini 1.5 Flash (1500 req/día gratis) | OpenAI | Anthropic | Ollama. Rate limit 15 req/min. Caché 6h. Auditoría ai_events. Búsqueda: 3 capas (local → Open Food Facts → USDA). Caché 30 días. Autocompletado. Mascotas: 15 personajes con 8 estados animables. Integración chat. Triggers eventos. Flujos: Onboarding, Diario comidas, Receta IA, Carrito, Finanzas, Inventario, Feed. APIs: /api/food-search, /api/recipe-ai, /api/chat, /api/sync, /api/auth. Performance: Debounce 1800ms, Caché, Image optimization, Code splitting, Memoization. Testing: Jest (cálculos) | React Testing Library (componentes) | Cypress (E2E). Deployment: Vercel + Supabase Cloud. Monitoreo: Supabase Analytics, Vercel Analytics, Error tracking, Rate limiting. Roadmap: MVP (hoy) | Fase 2 (Nordigen, water_log) | Fase 3 (rutinas, comunidad) | Fase 4+ (premium, marketplace).

Sección 11: Análisis Técnico de FoodOS

FoodOS es una plataforma unificada que integra gestión de inventario, nutrición personalizada y finanzas personales. Desarrollada con Next.js 14 (App Router), React 19 (Server Components), TypeScript 5.3+, Tailwind CSS 3, y Supabase PostgreSQL backend. Arquitectura: Cliente (SPA) → localStorage (JSON) → Supabase PostgreSQL. Debounce 1800ms agrupa cambios. Pull completo al login. Push con upsert+delete. RLS nativo en todas tablas. Conflictos: last-write-wins. Stack: Frontend (Next.js, React, TypeScript, Tailwind, Context+Immer) | Backend (Supabase, PostgreSQL 15+, Auth) | APIs (Open Food Facts, Gemini, OpenAI, Anthropic, Nordigen, USDA) | Hosting (Vercel, Supabase Cloud). Base de Datos: 39 tablas PostgreSQL con RLS. Categorías: catálogos (mascots), usuarios (user_profiles), inventario (almacenes, inventory_items), recetas (recipes, recipe_ingredients), nutrición (nutrition_goals, food_log), compras (shopping_lists), finanzas (gastos, ingresos), feed (feed_posts), IA (ai_events, ai_recipe_cache), otros. Seguridad: Auth (Google OAuth, Email/Password bcrypt, Magic Link). RLS en todas tablas. Keys IA en localStorage (nunca servidor). Validación TypeScript + server. CORS policy. Auditoría ai_events. Cálculos Nutricionales: TMB Mifflin-St Jeor (10×kg + 6.25×cm - 5×edad ± 5/-161). TDEE = TMB × factor (1.2-1.9). Peso base = ESPEN ajuste. Macros: fat_loss 80%/2.0g/kg, muscle_gain 105%/1.8g/kg, recom 83-90%/2.0g/kg (ciclado), maintain 100%/1.8g/kg. Distribución: proteína → grasa → carbo. Componentes: DashboardShell (layout) | OnboardingFlow (5 pasos) | 16+ Modales | 13+ Vistas. IA: Gemini 1.5 Flash (1500 req/día gratis) | OpenAI | Anthropic | Ollama. Rate limit 15 req/min. Caché 6h. Auditoría ai_events. Búsqueda: 3 capas (local → Open Food Facts → USDA). Caché 30 días. Autocompletado. Mascotas: 15 personajes con 8 estados animables. Integración chat. Triggers eventos. Flujos: Onboarding, Diario comidas, Receta IA, Carrito, Finanzas, Inventario, Feed. APIs: /api/food-search, /api/recipe-ai, /api/chat, /api/sync, /api/auth. Performance: Debounce 1800ms, Caché, Image optimization, Code splitting, Memoization. Testing: Jest (cálculos) | React Testing Library (componentes) | Cypress (E2E). Deployment: Vercel + Supabase Cloud. Monitoreo: Supabase Analytics, Vercel Analytics, Error tracking, Rate limiting. Roadmap: MVP (hoy) | Fase 2 (Nordigen, water_log) | Fase 3 (rutinas, comunidad) | Fase 4+ (premium, marketplace).

Sección 12: Análisis Técnico de FoodOS

FoodOS es una plataforma unificada que integra gestión de inventario, nutrición personalizada y finanzas personales. Desarrollada con Next.js 14 (App Router), React 19 (Server Components), TypeScript 5.3+, Tailwind CSS 3, y Supabase PostgreSQL backend. Arquitectura: Cliente (SPA) → localStorage (JSON) → Supabase PostgreSQL. Debounce 1800ms agrupa cambios. Pull completo al login. Push con upsert+delete. RLS nativo en todas tablas. Conflictos: last-write-wins. Stack: Frontend (Next.js, React, TypeScript, Tailwind, Context+Immer) | Backend (Supabase, PostgreSQL 15+, Auth) | APIs (Open Food Facts, Gemini, OpenAI, Anthropic, Nordigen, USDA) | Hosting (Vercel, Supabase Cloud). Base de Datos: 39 tablas PostgreSQL con RLS. Categorías: catálogos (mascots), usuarios (user_profiles), inventario (almacenes, inventory_items), recetas (recipes, recipe_ingredients), nutrición (nutrition_goals, food_log), compras (shopping_lists), finanzas (gastos, ingresos), feed (feed_posts), IA (ai_events, ai_recipe_cache), otros. Seguridad: Auth (Google OAuth, Email/Password bcrypt, Magic Link). RLS en todas tablas. Keys IA en localStorage (nunca servidor). Validación TypeScript + server. CORS policy. Auditoría ai_events. Cálculos Nutricionales: TMB Mifflin-St Jeor (10×kg + 6.25×cm - 5×edad ± 5/-161). TDEE = TMB × factor (1.2-1.9). Peso base = ESPEN ajuste. Macros: fat_loss 80%/2.0g/kg, muscle_gain 105%/1.8g/kg, recom 83-90%/2.0g/kg (ciclado), maintain 100%/1.8g/kg. Distribución: proteína → grasa → carbo. Componentes: DashboardShell (layout) | OnboardingFlow (5 pasos) | 16+ Modales | 13+ Vistas. IA: Gemini 1.5 Flash (1500 req/día gratis) | OpenAI | Anthropic | Ollama. Rate limit 15 req/min. Caché 6h. Auditoría ai_events. Búsqueda: 3 capas (local → Open Food Facts → USDA). Caché 30 días. Autocompletado. Mascotas: 15 personajes con 8 estados animables. Integración chat. Triggers eventos. Flujos: Onboarding, Diario comidas, Receta IA, Carrito, Finanzas, Inventario, Feed. APIs: /api/food-search, /api/recipe-ai, /api/chat, /api/sync, /api/auth. Performance: Debounce 1800ms, Caché, Image optimization, Code splitting, Memoization. Testing: Jest (cálculos) | React Testing Library (componentes) | Cypress (E2E). Deployment: Vercel + Supabase Cloud. Monitoreo: Supabase Analytics, Vercel Analytics, Error tracking, Rate limiting. Roadmap: MVP (hoy) | Fase 2 (Nordigen, water_log) | Fase 3 (rutinas, comunidad) | Fase 4+ (premium, marketplace).

Sección 13: Análisis Técnico de FoodOS

FoodOS es una plataforma unificada que integra gestión de inventario, nutrición personalizada y finanzas personales. Desarrollada con Next.js 14 (App Router), React 19 (Server Components), TypeScript 5.3+, Tailwind CSS 3, y Supabase PostgreSQL backend. Arquitectura: Cliente (SPA) → localStorage (JSON) → Supabase PostgreSQL. Debounce 1800ms

agrupa cambios. Pull completo al login. Push con upsert+delete. RLS nativo en todas tablas. Conflictos: last-write-wins. Stack: Frontend (Next.js, React, TypeScript, Tailwind, Context+immer) | Backend (Supabase, PostgreSQL 15+, Auth) | APIs (Open Food Facts, Gemini, OpenAI, Anthropic, Nordigen, USDA) | Hosting (Vercel, Supabase Cloud). Base de Datos: 39 tablas PostgreSQL con RLS. Categorías: catálogos (mascots), usuarios (user_profiles), inventario (almacenes, inventory_items), recetas (recipes, recipe_ingredients), nutrición (nutrition_goals, food_log), compras (shopping_lists), finanzas (gastos, ingresos), feed (feed_posts), IA (ai_events, ai_recipe_cache), otros. Seguridad: Auth (Google OAuth, Email/Password bcrypt, Magic Link). RLS en todas tablas. Keys IA en localStorage (nunca servidor). Validación TypeScript + server. CORS policy. Auditoría ai_events. Cálculos Nutricionales: TMB Mifflin-St Jeor (10×kg + 6.25×cm - 5×edad ± 5/-161). TDEE = TMB × factor (1.2-1.9). Peso base = ESPEN ajuste. Macros: fat_loss 80%/2.0g/kg, muscle_gain 105%/1.8g/kg, recomp 83-90%/2.0g/kg (ciclado), maintain 100%/1.8g/kg. Distribución: proteína → grasa → carbos. Componentes: DashboardShell (layout) | OnboardingFlow (5 pasos) | 16+ Modales | 13+ Vistas. IA: Gemini 1.5 Flash (1500 req/día gratis) | OpenAI | Anthropic | Ollama. Rate limit 15 req/min. Caché 6h. Auditoría ai_events. Búsqueda: 3 capas (local → Open Food Facts → USDA). Caché 30 días. Autocompletado. Mascotas: 15 personajes con 8 estados animables. Integración chat. Triggers eventos. Flujos: Onboarding, Diario comidas, Receta IA, Carrito, Finanzas, Inventario, Feed. APIs: /api/food-search, /api/recipe-ai, /api/chat, /api/sync, /api/auth. Performance: Debounce 1800ms, Caché, Image optimization, Code splitting, Memoization. Testing: Jest (cálculos) | React Testing Library (componentes) | Cypress (E2E). Deployment: Vercel + Supabase Cloud. Monitoreo: Supabase Analytics, Vercel Analytics, Error tracking, Rate limiting. Roadmap: MVP (hoy) | Fase 2 (Nordigen, water_log) | Fase 3 (rutinas, comunidad) | Fase 4+ (premium, marketplace).

Sección 14: Análisis Técnico de FoodOS

FoodOS es una plataforma unificada que integra gestión de inventario, nutrición personalizada y finanzas personales. Desarrollada con Next.js 14 (App Router), React 19 (Server Components), TypeScript 5.3+, Tailwind CSS 3, y Supabase PostgreSQL backend. Arquitectura: Cliente (SPA) → localStorage (JSON) → Supabase PostgreSQL. Debounce 1800ms agrupa cambios. Pull completo al login. Push con upsert+delete. RLS nativo en todas tablas. Conflictos: last-write-wins. Stack: Frontend (Next.js, React, TypeScript, Tailwind, Context+immer) | Backend (Supabase, PostgreSQL 15+, Auth) | APIs (Open Food Facts, Gemini, OpenAI, Anthropic, Nordigen, USDA) | Hosting (Vercel, Supabase Cloud). Base de Datos: 39 tablas PostgreSQL con RLS. Categorías: catálogos (mascots), usuarios (user_profiles), inventario (almacenes, inventory_items), recetas (recipes, recipe_ingredients), nutrición (nutrition_goals, food_log), compras (shopping_lists), finanzas (gastos, ingresos), feed (feed_posts), IA (ai_events, ai_recipe_cache), otros. Seguridad: Auth (Google OAuth, Email/Password bcrypt, Magic Link). RLS en todas tablas. Keys IA en localStorage (nunca servidor). Validación TypeScript + server. CORS policy. Auditoría ai_events. Cálculos Nutricionales: TMB Mifflin-St Jeor (10×kg + 6.25×cm - 5×edad ± 5/-161). TDEE = TMB × factor (1.2-1.9). Peso base = ESPEN ajuste. Macros: fat_loss 80%/2.0g/kg, muscle_gain 105%/1.8g/kg, recomp 83-90%/2.0g/kg (ciclado), maintain 100%/1.8g/kg. Distribución: proteína → grasa → carbos. Componentes: DashboardShell (layout) | OnboardingFlow (5 pasos) | 16+ Modales | 13+ Vistas. IA: Gemini 1.5 Flash (1500 req/día gratis) | OpenAI | Anthropic | Ollama. Rate limit 15 req/min. Caché 6h. Auditoría ai_events. Búsqueda: 3 capas (local → Open Food Facts → USDA). Caché 30 días. Autocompletado. Mascotas: 15 personajes con 8 estados animables. Integración chat. Triggers eventos. Flujos: Onboarding, Diario comidas, Receta IA, Carrito, Finanzas, Inventario, Feed. APIs: /api/food-search, /api/recipe-ai, /api/chat, /api/sync, /api/auth. Performance: Debounce 1800ms, Caché, Image optimization, Code splitting, Memoization. Testing: Jest (cálculos) | React Testing Library (componentes) | Cypress (E2E). Deployment: Vercel + Supabase Cloud. Monitoreo: Supabase Analytics, Vercel Analytics, Error tracking, Rate limiting. Roadmap: MVP (hoy) | Fase 2 (Nordigen, water_log) | Fase 3 (rutinas, comunidad) | Fase 4+ (premium, marketplace).

Sección 15: Análisis Técnico de FoodOS

FoodOS es una plataforma unificada que integra gestión de inventario, nutrición personalizada y finanzas personales. Desarrollada con Next.js 14 (App Router), React 19 (Server Components), TypeScript 5.3+, Tailwind CSS 3, y Supabase PostgreSQL backend. Arquitectura: Cliente (SPA) → localStorage (JSON) → Supabase PostgreSQL. Debounce 1800ms agrupa cambios. Pull completo al login. Push con upsert+delete. RLS nativo en todas tablas. Conflictos: last-write-wins. Stack: Frontend (Next.js, React, TypeScript, Tailwind, Context+immer) | Backend (Supabase, PostgreSQL 15+, Auth) | APIs (Open Food Facts, Gemini, OpenAI, Anthropic, Nordigen, USDA) | Hosting (Vercel, Supabase Cloud). Base de Datos: 39 tablas PostgreSQL con RLS. Categorías: catálogos (mascots), usuarios (user_profiles), inventario (almacenes, inventory_items), recetas (recipes, recipe_ingredients), nutrición (nutrition_goals, food_log), compras (shopping_lists), finanzas (gastos, ingresos), feed (feed_posts), IA (ai_events, ai_recipe_cache), otros. Seguridad: Auth (Google OAuth, Email/Password bcrypt, Magic Link). RLS en todas tablas. Keys IA en localStorage (nunca servidor). Validación TypeScript + server. CORS policy. Auditoría ai_events. Cálculos Nutricionales: TMB Mifflin-St Jeor (10×kg + 6.25×cm - 5×edad ± 5/-161). TDEE = TMB × factor (1.2-1.9). Peso base = ESPEN ajuste. Macros: fat_loss 80%/2.0g/kg, muscle_gain 105%/1.8g/kg, recomp 83-90%/2.0g/kg (ciclado), maintain 100%/1.8g/kg. Distribución: proteína → grasa → carbos. Componentes: DashboardShell (layout) | OnboardingFlow (5 pasos) | 16+ Modales | 13+ Vistas. IA: Gemini 1.5 Flash (1500 req/día gratis) | OpenAI | Anthropic | Ollama. Rate limit 15 req/min. Caché 6h. Auditoría ai_events. Búsqueda: 3 capas (local → Open Food Facts → USDA). Caché 30 días. Autocompletado. Mascotas: 15 personajes con 8 estados animables. Integración chat. Triggers eventos. Flujos: Onboarding, Diario comidas, Receta IA, Carrito, Finanzas, Inventario, Feed. APIs: /api/food-search, /api/recipe-ai, /api/chat, /api/sync, /api/auth. Performance: Debounce 1800ms, Caché, Image optimization, Code splitting, Memoization. Testing: Jest (cálculos) | React Testing Library (componentes) | Cypress (E2E). Deployment: Vercel + Supabase Cloud. Monitoreo: Supabase Analytics, Vercel Analytics, Error tracking, Rate limiting. Roadmap: MVP (hoy) | Fase 2 (Nordigen, water_log) | Fase 3 (rutinas, comunidad) | Fase 4+ (premium, marketplace).

Sección 16: Análisis Técnico de FoodOS

FoodOS es una plataforma unificada que integra gestión de inventario, nutrición personalizada y finanzas personales. Desarrollada con Next.js 14 (App Router), React 19 (Server Components), TypeScript 5.3+, Tailwind CSS 3, y Supabase PostgreSQL backend. Arquitectura: Cliente (SPA) → localStorage (JSON) → Supabase PostgreSQL. Debounce 1800ms agrupa cambios. Pull completo al login. Push con upsert+delete. RLS nativo en todas tablas. Conflictos: last-write-wins. Stack: Frontend (Next.js, React, TypeScript, Tailwind, Context+immer) | Backend (Supabase, PostgreSQL 15+, Auth) | APIs (Open Food Facts, Gemini, OpenAI, Anthropic, Nordigen, USDA) | Hosting (Vercel, Supabase Cloud). Base de Datos: 39 tablas PostgreSQL con RLS. Categorías: catálogos (mascots), usuarios (user_profiles), inventario (almacenes, inventory_items), recetas (recipes, recipe_ingredients), nutrición (nutrition_goals, food_log), compras (shopping_lists), finanzas (gastos, ingresos), feed (feed_posts), IA (ai_events, ai_recipe_cache), otros. Seguridad: Auth (Google OAuth, Email/Password bcrypt, Magic Link). RLS en todas tablas. Keys IA en localStorage (nunca servidor). Validación TypeScript + server. CORS policy. Auditoría ai_events. Cálculos Nutricionales: TMB Mifflin-St Jeor (10×kg + 6.25×cm - 5×edad ± 5/-161). TDEE = TMB × factor (1.2-1.9). Peso base = ESPEN ajuste. Macros: fat_loss 80%/2.0g/kg, muscle_gain 105%/1.8g/kg, recomp 83-90%/2.0g/kg (ciclado), maintain 100%/1.8g/kg. Distribución: proteína → grasa → carbos. Componentes: DashboardShell (layout) | OnboardingFlow (5 pasos) | 16+ Modales | 13+ Vistas. IA: Gemini 1.5 Flash (1500 req/día gratis) | OpenAI | Anthropic | Ollama. Rate limit 15 req/min. Caché 6h. Auditoría ai_events. Búsqueda: 3 capas (local → Open Food Facts → USDA). Caché 30 días. Autocompletado. Mascotas: 15 personajes con 8 estados animables. Integración chat. Triggers eventos. Flujos: Onboarding, Diario comidas, Receta IA, Carrito, Finanzas, Inventario, Feed. APIs: /api/food-search, /api/recipe-ai, /api/chat, /api/sync, /api/auth. Performance: Debounce 1800ms, Caché, Image optimization, Code splitting, Memoization. Testing: Jest (cálculos) | React Testing Library (componentes) | Cypress (E2E). Deployment: Vercel + Supabase Cloud. Monitoreo: Supabase Analytics, Vercel Analytics, Error tracking, Rate limiting. Roadmap: MVP (hoy) | Fase 2 (Nordigen, water_log) | Fase 3 (rutinas, comunidad) | Fase 4+ (premium, marketplace).

Sección 17: Análisis Técnico de FoodOS

FoodOS es una plataforma unificada que integra gestión de inventario, nutrición personalizada y finanzas personales. Desarrollada con Next.js 14 (App Router), React 19 (Server Components), TypeScript 5.3+, Tailwind CSS 3, y Supabase PostgreSQL backend. Arquitectura: Cliente (SPA) → localStorage (JSON) → Supabase PostgreSQL. Debounce 1800ms agrupa cambios. Pull completo al login. Push con upsert+delete. RLS nativo en todas tablas. Conflictos: last-write-wins. Stack: Frontend (Next.js, React, TypeScript, Tailwind, Context+immer) | Backend (Supabase, PostgreSQL 15+, Auth) | APIs (Open Food Facts, Gemini, OpenAI, Anthropic, Nordigen, USDA) | Hosting (Vercel, Supabase Cloud). Base de Datos: 39 tablas PostgreSQL con RLS. Categorías: catálogos (mascots), usuarios (user_profiles), inventario (almacenes, inventory_items), recetas (recipes, recipe_ingredients), nutrición (nutrition_goals, food_log), compras (shopping_lists), finanzas (gastos, ingresos), feed (feed_posts), IA (ai_events, ai_recipe_cache), otros. Seguridad: Auth (Google OAuth, Email/Password bcrypt, Magic Link). RLS en todas tablas. Keys IA en localStorage (nunca servidor). Validación TypeScript + server. CORS policy. Auditoría ai_events. Cálculos Nutricionales: TMB Mifflin-St Jeor (10×kg + 6.25×cm - 5×edad ± 5/-161). TDEE = TMB × factor (1.2-1.9). Peso base = ESPEN ajuste. Macros: fat_loss 80%/2.0g/kg, muscle_gain 105%/1.8g/kg, recomp 83-90%/2.0g/kg (ciclado), maintain 100%/1.8g/kg. Distribución: proteína → grasa → carbos. Componentes: DashboardShell (layout) | OnboardingFlow (5 pasos) | 16+ Modales | 13+ Vistas. IA: Gemini 1.5 Flash (1500 req/día gratis) | OpenAI | Anthropic | Ollama. Rate limit 15 req/min. Caché 6h. Auditoría ai_events. Búsqueda: 3 capas (local → Open Food Facts → USDA). Caché 30 días. Autocompletado. Mascotas: 15 personajes con 8 estados animables. Integración chat. Triggers eventos. Flujos: Onboarding, Diario comidas, Receta IA, Carrito, Finanzas, Inventario, Feed. APIs: /api/food-search, /api/recipe-ai, /api/chat, /api/sync, /api/auth. Performance: Debounce 1800ms, Caché, Image optimization, Code splitting, Memoization. Testing: Jest (cálculos) | React Testing Library (componentes) | Cypress (E2E). Deployment: Vercel + Supabase Cloud. Monitoreo: Supabase Analytics, Vercel Analytics, Error tracking, Rate limiting. Roadmap: MVP (hoy) | Fase 2 (Nordigen, water_log) | Fase 3 (rutinas, comunidad) | Fase 4+ (premium, marketplace).

Sección 18: Análisis Técnico de FoodOS

FoodOS es una plataforma unificada que integra gestión de inventario, nutrición personalizada y finanzas personales. Desarrollada con Next.js 14 (App Router), React 19 (Server Components), TypeScript 5.3+, Tailwind CSS 3, y Supabase PostgreSQL backend. Arquitectura: Cliente (SPA) → localStorage (JSON) → Supabase PostgreSQL. Debounce 1800ms agrupa cambios. Pull completo al login. Push con upsert+delete. RLS nativo en todas tablas. Conflictos: last-write-wins. Stack: Frontend (Next.js, React, TypeScript, Tailwind, Context+immer) | Backend (Supabase, PostgreSQL 15+, Auth) | APIs (Open Food Facts, Gemini, OpenAI, Anthropic, Nordigen, USDA) | Hosting (Vercel, Supabase Cloud). Base de Datos: 39 tablas PostgreSQL con RLS. Categorías: catálogos (mascots), usuarios (user_profiles), inventario (almacenes, inventory_items), recetas (recipes, recipe_ingredients), nutrición (nutrition_goals, food_log), compras (shopping_lists), finanzas (gastos, ingresos), feed (feed_posts), IA (ai_events, ai_recipe_cache), otros. Seguridad: Auth (Google OAuth, Email/Password bcrypt, Magic Link). RLS en todas tablas. Keys IA en localStorage (nunca servidor). Validación TypeScript + server. CORS policy. Auditoría ai_events. Cálculos Nutricionales: TMB Mifflin-St Jeor (10×kg + 6.25×cm - 5×edad ± 5/-161). TDEE = TMB × factor (1.2-1.9). Peso base = ESPEN ajuste. Macros: fat_loss 80%/2.0g/kg, muscle_gain 105%/1.8g/kg, recomp 83-90%/2.0g/kg (ciclado), maintain 100%/1.8g/kg. Distribución: proteína → grasa → carbos. Componentes: DashboardShell (layout) | OnboardingFlow (5 pasos) | 16+ Modales | 13+ Vistas. IA: Gemini 1.5 Flash (1500 req/día gratis) | OpenAI | Anthropic | Ollama. Rate limit 15 req/min. Caché 6h. Auditoría ai_events. Búsqueda: 3 capas (local → Open Food Facts → USDA). Caché 30 días. Autocompletado. Mascotas: 15 personajes con 8 estados animables. Integración chat. Triggers eventos. Flujos: Onboarding, Diario comidas, Receta IA, Carrito, Finanzas, Inventario, Feed. APIs: /api/food-search, /api/recipe-ai, /api/chat, /api/sync, /api/auth. Performance: Debounce 1800ms, Caché, Image optimization, Code splitting, Memoization. Testing: Jest (cálculos) | React Testing Library (componentes) | Cypress (E2E). Deployment: Vercel + Supabase Cloud. Monitoreo: Supabase Analytics, Vercel Analytics, Error tracking, Rate limiting. Roadmap: MVP (hoy) | Fase 2 (Nordigen, water_log) | Fase 3 (rutinas, comunidad) | Fase 4+ (premium, marketplace).

Sección 19: Análisis Técnico de FoodOS

FoodOS es una plataforma unificada que integra gestión de inventario, nutrición personalizada y finanzas personales. Desarrollada con Next.js 14 (App Router), React 19 (Server Components), TypeScript 5.3+, Tailwind CSS 3, y Supabase PostgreSQL backend. Arquitectura: Cliente (SPA) → localStorage (JSON) → Supabase PostgreSQL. Debounce 1800ms agrupa cambios. Pull completo al login. Push con upsert+delete. RLS nativo en todas tablas. Conflictos: last-write-wins. Stack: Frontend (Next.js, React, TypeScript, Tailwind, Context+immer) | Backend (Supabase, PostgreSQL 15+, Auth) | APIs (Open Food Facts, Gemini, OpenAI, Anthropic, Nordigen, USDA) | Hosting (Vercel, Supabase Cloud). Base de Datos:

39 tablas PostgreSQL con RLS. Categorías: catálogos (mascots), usuarios (user_profiles), inventario (almacenes, inventory_items), recetas (recipes, recipe_ingredients), nutrición (nutrition_goals, food_log), compras (shopping_lists), finanzas (gastos, ingresos), feed (feed_posts), IA (ai_events, ai_recipe_cache), otros. Seguridad: Auth (Google OAuth, Email/Password bcrypt, Magic Link). RLS en todas tablas. Keys IA en localStorage (nunca servidor). Validación TypeScript + server. CORS policy. Auditoría ai_events. Cálculos Nutricionales: TMB Mifflin-St Jeor (10×kg + 6.25×cm - 5×edad ± 5/161). TDEE = TMB × factor (1.2-1.9). Peso base = ESPEN ajuste. Macros: fat_loss 80%/2.0g/kg, muscle_gain 105%/1.8g/kg, recomp 83-90%/2.0g/kg (ciclado), maintain 100%/1.8g/kg. Distribución: proteína → grasa → carbo. Componentes: DashboardShell (layout) | OnboardingFlow (5 pasos) | 16+ Modales | 13+ Vistas. IA: Gemini 1.5 Flash (1500 req/día gratis) | OpenAI | Anthropic | Ollama. Rate limit 15 req/min. Caché 6h. Auditoría ai_events. Búsqueda: 3 capas (local → Open Food Facts → USDA). Caché 30 días. Autocompletado. Mascotas: 15 personajes con 8 estados animables. Integración chat. Triggers eventos. Flujos: Onboarding, Diario comidas, Receta IA, Carrito, Finanzas, Inventario, Feed. APIs: /api/food-search, /api/recipe-ai, /api/chat, /api/sync, /api/auth. Performance: Debounce 1800ms, Caché, Image optimization, Code splitting, Memoization. Testing: Jest (cálculos) | React Testing Library (componentes) | Cypress (E2E). Deployment: Vercel + Supabase Cloud. Monitoreo: Supabase Analytics, Vercel Analytics, Error tracking, Rate limiting. Roadmap: MVP (hoy) | Fase 2 (Nordigen, water_log) | Fase 3 (rutinas, comunidad) | Fase 4+ (premium, marketplace).

Sección 20: Análisis Técnico de FoodOS

FoodOS es una plataforma unificada que integra gestión de inventario, nutrición personalizada y finanzas personales. Desarrollada con Next.js 14 (App Router), React 19 (Server Components), TypeScript 5.3+, Tailwind CSS 3, y Supabase PostgreSQL backend. Arquitectura: Cliente (SPA) → localStorage (JSON) → Supabase PostgreSQL. Debounce 1800ms agrupa cambios. Pull completo al login. Push con upsert+delete. RLS nativo en todas tablas. Conflictos: last-write-wins. Stack: Frontend (Next.js, React, TypeScript, Tailwind, Context+Immer) | Backend (Supabase, PostgreSQL 15+, Auth) | APIs (Open Food Facts, Gemini, OpenAI, Anthropic, Nordigen, USDA) | Hosting (Vercel, Supabase Cloud). Base de Datos: 39 tablas PostgreSQL con RLS. Categorías: catálogos (mascots), usuarios (user_profiles), inventario (almacenes, inventory_items), recetas (recipes, recipe_ingredients), nutrición (nutrition_goals, food_log), compras (shopping_lists), finanzas (gastos, ingresos), feed (feed_posts), IA (ai_events, ai_recipe_cache), otros. Seguridad: Auth (Google OAuth, Email/Password bcrypt, Magic Link). RLS en todas tablas. Keys IA en localStorage (nunca servidor). Validación TypeScript + server. CORS policy. Auditoría ai_events. Cálculos Nutricionales: TMB Mifflin-St Jeor (10×kg + 6.25×cm - 5×edad ± 5/161). TDEE = TMB × factor (1.2-1.9). Peso base = ESPEN ajuste. Macros: fat_loss 80%/2.0g/kg, muscle_gain 105%/1.8g/kg, recomp 83-90%/2.0g/kg (ciclado), maintain 100%/1.8g/kg. Distribución: proteína → grasa → carbo. Componentes: DashboardShell (layout) | OnboardingFlow (5 pasos) | 16+ Modales | 13+ Vistas. IA: Gemini 1.5 Flash (1500 req/día gratis) | OpenAI | Anthropic | Ollama. Rate limit 15 req/min. Caché 6h. Auditoría ai_events. Búsqueda: 3 capas (local → Open Food Facts → USDA). Caché 30 días. Autocompletado. Mascotas: 15 personajes con 8 estados animables. Integración chat. Triggers eventos. Flujos: Onboarding, Diario comidas, Receta IA, Carrito, Finanzas, Inventario, Feed. APIs: /api/food-search, /api/recipe-ai, /api/chat, /api/sync, /api/auth. Performance: Debounce 1800ms, Caché, Image optimization, Code splitting, Memoization. Testing: Jest (cálculos) | React Testing Library (componentes) | Cypress (E2E). Deployment: Vercel + Supabase Cloud. Monitoreo: Supabase Analytics, Vercel Analytics, Error tracking, Rate limiting. Roadmap: MVP (hoy) | Fase 2 (Nordigen, water_log) | Fase 3 (rutinas, comunidad) | Fase 4+ (premium, marketplace).

Sección 21: Análisis Técnico de FoodOS

FoodOS es una plataforma unificada que integra gestión de inventario, nutrición personalizada y finanzas personales. Desarrollada con Next.js 14 (App Router), React 19 (Server Components), TypeScript 5.3+, Tailwind CSS 3, y Supabase PostgreSQL backend. Arquitectura: Cliente (SPA) → localStorage (JSON) → Supabase PostgreSQL. Debounce 1800ms agrupa cambios. Pull completo al login. Push con upsert+delete. RLS nativo en todas tablas. Conflictos: last-write-wins. Stack: Frontend (Next.js, React, TypeScript, Tailwind, Context+Immer) | Backend (Supabase, PostgreSQL 15+, Auth) | APIs (Open Food Facts, Gemini, OpenAI, Anthropic, Nordigen, USDA) | Hosting (Vercel, Supabase Cloud). Base de Datos: 39 tablas PostgreSQL con RLS. Categorías: catálogos (mascots), usuarios (user_profiles), inventario (almacenes, inventory_items), recetas (recipes, recipe_ingredients), nutrición (nutrition_goals, food_log), compras (shopping_lists), finanzas (gastos, ingresos), feed (feed_posts), IA (ai_events, ai_recipe_cache), otros. Seguridad: Auth (Google OAuth, Email/Password bcrypt, Magic Link). RLS en todas tablas. Keys IA en localStorage (nunca servidor). Validación TypeScript + server. CORS policy. Auditoría ai_events. Cálculos Nutricionales: TMB Mifflin-St Jeor (10×kg + 6.25×cm - 5×edad ± 5/161). TDEE = TMB × factor (1.2-1.9). Peso base = ESPEN ajuste. Macros: fat_loss 80%/2.0g/kg, muscle_gain 105%/1.8g/kg, recomp 83-90%/2.0g/kg (ciclado), maintain 100%/1.8g/kg. Distribución: proteína → grasa → carbo. Componentes: DashboardShell (layout) | OnboardingFlow (5 pasos) | 16+ Modales | 13+ Vistas. IA: Gemini 1.5 Flash (1500 req/día gratis) | OpenAI | Anthropic | Ollama. Rate limit 15 req/min. Caché 6h. Auditoría ai_events. Búsqueda: 3 capas (local → Open Food Facts → USDA). Caché 30 días. Autocompletado. Mascotas: 15 personajes con 8 estados animables. Integración chat. Triggers eventos. Flujos: Onboarding, Diario comidas, Receta IA, Carrito, Finanzas, Inventario, Feed. APIs: /api/food-search, /api/recipe-ai, /api/chat, /api/sync, /api/auth. Performance: Debounce 1800ms, Caché, Image optimization, Code splitting, Memoization. Testing: Jest (cálculos) | React Testing Library (componentes) | Cypress (E2E). Deployment: Vercel + Supabase Cloud. Monitoreo: Supabase Analytics, Vercel Analytics, Error tracking, Rate limiting. Roadmap: MVP (hoy) | Fase 2 (Nordigen, water_log) | Fase 3 (rutinas, comunidad) | Fase 4+ (premium, marketplace).

Sección 22: Análisis Técnico de FoodOS

FoodOS es una plataforma unificada que integra gestión de inventario, nutrición personalizada y finanzas personales. Desarrollada con Next.js 14 (App Router), React 19 (Server Components), TypeScript 5.3+, Tailwind CSS 3, y Supabase PostgreSQL backend. Arquitectura: Cliente (SPA) → localStorage (JSON) → Supabase PostgreSQL. Debounce 1800ms agrupa cambios. Pull completo al login. Push con upsert+delete. RLS nativo en todas tablas. Conflictos: last-write-wins. Stack: Frontend (Next.js, React, TypeScript, Tailwind, Context+Immer) | Backend (Supabase, PostgreSQL 15+, Auth) | APIs (Open Food Facts, Gemini, OpenAI, Anthropic, Nordigen, USDA) | Hosting (Vercel, Supabase Cloud). Base de Datos: 39 tablas PostgreSQL con RLS. Categorías: catálogos (mascots), usuarios (user_profiles), inventario (almacenes, inventory_items), recetas (recipes, recipe_ingredients), nutrición (nutrition_goals, food_log), compras (shopping_lists), finanzas (gastos, ingresos), feed (feed_posts), IA (ai_events, ai_recipe_cache), otros. Seguridad: Auth (Google OAuth, Email/Password bcrypt, Magic Link). RLS en todas tablas. Keys IA en localStorage (nunca servidor). Validación TypeScript + server. CORS policy. Auditoría ai_events. Cálculos Nutricionales: TMB Mifflin-St Jeor (10×kg + 6.25×cm - 5×edad ± 5/161). TDEE = TMB × factor (1.2-1.9). Peso base = ESPEN ajuste. Macros: fat_loss 80%/2.0g/kg, muscle_gain 105%/1.8g/kg, recomp 83-90%/2.0g/kg (ciclado), maintain 100%/1.8g/kg. Distribución: proteína → grasa → carbo. Componentes: DashboardShell (layout) | OnboardingFlow (5 pasos) | 16+ Modales | 13+ Vistas. IA: Gemini 1.5 Flash (1500 req/día gratis) | OpenAI | Anthropic | Ollama. Rate limit 15 req/min. Caché 6h. Auditoría ai_events. Búsqueda: 3 capas (local → Open Food Facts → USDA). Caché 30 días. Autocompletado. Mascotas: 15 personajes con 8 estados animables. Integración chat. Triggers eventos. Flujos: Onboarding, Diario comidas, Receta IA, Carrito, Finanzas, Inventario, Feed. APIs: /api/food-search, /api/recipe-ai, /api/chat, /api/sync, /api/auth. Performance: Debounce 1800ms, Caché, Image optimization, Code splitting, Memoization. Testing: Jest (cálculos) | React Testing Library (componentes) | Cypress (E2E). Deployment: Vercel + Supabase Cloud. Monitoreo: Supabase Analytics, Vercel Analytics, Error tracking, Rate limiting. Roadmap: MVP (hoy) | Fase 2 (Nordigen, water_log) | Fase 3 (rutinas, comunidad) | Fase 4+ (premium, marketplace).

Sección 23: Análisis Técnico de FoodOS

FoodOS es una plataforma unificada que integra gestión de inventario, nutrición personalizada y finanzas personales. Desarrollada con Next.js 14 (App Router), React 19 (Server Components), TypeScript 5.3+, Tailwind CSS 3, y Supabase PostgreSQL backend. Arquitectura: Cliente (SPA) → localStorage (JSON) → Supabase PostgreSQL. Debounce 1800ms agrupa cambios. Pull completo al login. Push con upsert+delete. RLS nativo en todas tablas. Conflictos: last-write-wins. Stack: Frontend (Next.js, React, TypeScript, Tailwind, Context+Immer) | Backend (Supabase, PostgreSQL 15+, Auth) | APIs (Open Food Facts, Gemini, OpenAI, Anthropic, Nordigen, USDA) | Hosting (Vercel, Supabase Cloud). Base de Datos: 39 tablas PostgreSQL con RLS. Categorías: catálogos (mascots), usuarios (user_profiles), inventario (almacenes, inventory_items), recetas (recipes, recipe_ingredients), nutrición (nutrition_goals, food_log), compras (shopping_lists), finanzas (gastos, ingresos), feed (feed_posts), IA (ai_events, ai_recipe_cache), otros. Seguridad: Auth (Google OAuth, Email/Password bcrypt, Magic Link). RLS en todas tablas. Keys IA en localStorage (nunca servidor). Validación TypeScript + server. CORS policy. Auditoría ai_events. Cálculos Nutricionales: TMB Mifflin-St Jeor (10×kg + 6.25×cm - 5×edad ± 5/161). TDEE = TMB × factor (1.2-1.9). Peso base = ESPEN ajuste. Macros: fat_loss 80%/2.0g/kg, muscle_gain 105%/1.8g/kg, recomp 83-90%/2.0g/kg (ciclado), maintain 100%/1.8g/kg. Distribución: proteína → grasa → carbo. Componentes: DashboardShell (layout) | OnboardingFlow (5 pasos) | 16+ Modales | 13+ Vistas. IA: Gemini 1.5 Flash (1500 req/día gratis) | OpenAI | Anthropic | Ollama. Rate limit 15 req/min. Caché 6h. Auditoría ai_events. Búsqueda: 3 capas (local → Open Food Facts → USDA). Caché 30 días. Autocompletado. Mascotas: 15 personajes con 8 estados animables. Integración chat. Triggers eventos. Flujos: Onboarding, Diario comidas, Receta IA, Carrito, Finanzas, Inventario, Feed. APIs: /api/food-search, /api/recipe-ai, /api/chat, /api/sync, /api/auth. Performance: Debounce 1800ms, Caché, Image optimization, Code splitting, Memoization. Testing: Jest (cálculos) | React Testing Library (componentes) | Cypress (E2E). Deployment: Vercel + Supabase Cloud. Monitoreo: Supabase Analytics, Vercel Analytics, Error tracking, Rate limiting. Roadmap: MVP (hoy) | Fase 2 (Nordigen, water_log) | Fase 3 (rutinas, comunidad) | Fase 4+ (premium, marketplace).

Sección 24: Análisis Técnico de FoodOS

FoodOS es una plataforma unificada que integra gestión de inventario, nutrición personalizada y finanzas personales. Desarrollada con Next.js 14 (App Router), React 19 (Server Components), TypeScript 5.3+, Tailwind CSS 3, y Supabase PostgreSQL backend. Arquitectura: Cliente (SPA) → localStorage (JSON) → Supabase PostgreSQL. Debounce 1800ms agrupa cambios. Pull completo al login. Push con upsert+delete. RLS nativo en todas tablas. Conflictos: last-write-wins. Stack: Frontend (Next.js, React, TypeScript, Tailwind, Context+Immer) | Backend (Supabase, PostgreSQL 15+, Auth) | APIs (Open Food Facts, Gemini, OpenAI, Anthropic, Nordigen, USDA) | Hosting (Vercel, Supabase Cloud). Base de Datos: 39 tablas PostgreSQL con RLS. Categorías: catálogos (mascots), usuarios (user_profiles), inventario (almacenes, inventory_items), recetas (recipes, recipe_ingredients), nutrición (nutrition_goals, food_log), compras (shopping_lists), finanzas (gastos, ingresos), feed (feed_posts), IA (ai_events, ai_recipe_cache), otros. Seguridad: Auth (Google OAuth, Email/Password bcrypt, Magic Link). RLS en todas tablas. Keys IA en localStorage (nunca servidor). Validación TypeScript + server. CORS policy. Auditoría ai_events. Cálculos Nutricionales: TMB Mifflin-St Jeor (10×kg + 6.25×cm - 5×edad ± 5/161). TDEE = TMB × factor (1.2-1.9). Peso base = ESPEN ajuste. Macros: fat_loss 80%/2.0g/kg, muscle_gain 105%/1.8g/kg, recomp 83-90%/2.0g/kg (ciclado), maintain 100%/1.8g/kg. Distribución: proteína → grasa → carbo. Componentes: DashboardShell (layout) | OnboardingFlow (5 pasos) | 16+ Modales | 13+ Vistas. IA: Gemini 1.5 Flash (1500 req/día gratis) | OpenAI | Anthropic | Ollama. Rate limit 15 req/min. Caché 6h. Auditoría ai_events. Búsqueda: 3 capas (local → Open Food Facts → USDA). Caché 30 días. Autocompletado. Mascotas: 15 personajes con 8 estados animables. Integración chat. Triggers eventos. Flujos: Onboarding, Diario comidas, Receta IA, Carrito, Finanzas, Inventario, Feed. APIs: /api/food-search, /api/recipe-ai, /api/chat, /api/sync, /api/auth. Performance: Debounce 1800ms, Caché, Image optimization, Code splitting, Memoization. Testing: Jest (cálculos) | React Testing Library (componentes) | Cypress (E2E). Deployment: Vercel + Supabase Cloud. Monitoreo: Supabase Analytics, Vercel Analytics, Error tracking, Rate limiting. Roadmap: MVP (hoy) | Fase 2 (Nordigen, water_log) | Fase 3 (rutinas, comunidad) | Fase 4+ (premium, marketplace).

Sección 25: Análisis Técnico de FoodOS

FoodOS es una plataforma unificada que integra gestión de inventario, nutrición personalizada y finanzas personales. Desarrollada con Next.js 14 (App Router), React 19 (Server Components), TypeScript 5.3+, Tailwind CSS 3, y Supabase PostgreSQL backend. Arquitectura: Cliente (SPA) → localStorage (JSON) → Supabase PostgreSQL. Debounce 1800ms agrupa cambios. Pull completo al login. Push con upsert+delete. RLS nativo en todas tablas. Conflictos: last-write-wins. Stack: Frontend (Next.js, React, TypeScript, Tailwind, Context+Immer) | Backend (Supabase, PostgreSQL 15+, Auth) | APIs (Open Food Facts, Gemini, OpenAI, Anthropic, Nordigen, USDA) | Hosting (Vercel, Supabase Cloud). Base de Datos: 39 tablas PostgreSQL con RLS. Categorías: catálogos (mascots), usuarios (user_profiles), inventario (almacenes, inventory_items), recetas (recipes, recipe_ingredients), nutrición (nutrition_goals, food_log), compras (shopping_lists), finanzas (gastos, ingresos), feed (feed_posts), IA (ai_events, ai_recipe_cache), otros. Seguridad: Auth (Google OAuth,

Email/Password bcrypt, Magic Link). RLS en todas tablas. Keys IA en localStorage (nunca servidor). Validación TypeScript + server. CORS policy. Auditoría ai_events. Cálculos Nutricionales: TMB Mifflin-St Jeor (10×kg + 6.25×cm - 5×edad ± 5/-161). TDEE = TMB × factor (1.2-1.9). Peso base = ESPEN ajuste. Macros: fat_loss 80%/2.0g/kg, muscle_gain 105%/1.8g/kg, recomp 83-90%/2.0g/kg (ciclado), maintain 100%/1.8g/kg. Distribución: proteína → grasa → carbo. Componentes: DashboardShell (layout) | OnboardingFlow (5 pasos) | 16+ Modales | 13+ Vistas. IA: Gemini 1.5 Flash (1500 req/día gratis) | OpenAI | Anthropic | Ollama. Rate limit 15 req/min. Caché 6h. Auditoría ai_events. Búsqueda: 3 capas (local → Open Food Facts → USDA). Caché 30 días. Autocompletado. Mascotas: 15 personajes con 8 estados animables. Integración chat. Triggers eventos. Flujos: Onboarding, Diario comidas, Receta IA, Carrito, Finanzas, Inventario, Feed. APIS: /api/food-search, /api/recipe-ai, /api/chat, /api/sync, /api/auth. Performance: Debounce 1800ms, Caché, Image optimization, Code splitting, Memoization. Testing: Jest (cálculos) | React Testing Library (componentes) | Cypress (E2E). Deployment: Vercel + Supabase Cloud. Monitoreo: Supabase Analytics, Vercel Analytics, Error tracking, Rate limiting. Roadmap: MVP (hoy) | Fase 2 (Nordigen, water_log) | Fase 3 (rutinas, comunidad) | Fase 4+ (premium, marketplace).

Sección 26: Análisis Técnico de FoodOS

FoodOS es una plataforma unificada que integra gestión de inventario, nutrición personalizada y finanzas personales. Desarrollada con Next.js 14 (App Router), React 19 (Server Components), TypeScript 5.3+, Tailwind CSS 3, y Supabase PostgreSQL backend. Arquitectura: Cliente (SPA) → localStorage (JSON) → Supabase PostgreSQL. Debounce 1800ms agrupa cambios. Pull completo al login. Push con upsert+delete. RLS nativo en todas tablas. Conflictos: last-write-wins. Stack: Frontend (Next.js, React, TypeScript, Tailwind, Context+Immer) | Backend (Supabase, PostgreSQL 15+, Auth) | APIs (Open Food Facts, Gemini, OpenAI, Anthropic, Nordigen, USDA) | Hosting (Vercel, Supabase Cloud). Base de Datos: 39 tablas PostgreSQL con RLS. Categorías: catálogos (mascots), usuarios (user_profiles), inventario (almacenes, inventory_items), recetas (recipes, recipe_ingredients), nutrición (nutrition_goals, food_log), compras (shopping_lists), finanzas (gastos, ingresos), feed (feed_posts), IA (ai_events, ai_recipe_cache), otros. Seguridad: Auth (Google OAuth, Email/Password bcrypt, Magic Link). RLS en todas tablas. Keys IA en localStorage (nunca servidor). Validación TypeScript + server. CORS policy. Auditoría ai_events. Cálculos Nutricionales: TMB Mifflin-St Jeor (10×kg + 6.25×cm - 5×edad ± 5/-161). TDEE = TMB × factor (1.2-1.9). Peso base = ESPEN ajuste. Macros: fat_loss 80%/2.0g/kg, muscle_gain 105%/1.8g/kg, recomp 83-90%/2.0g/kg (ciclado), maintain 100%/1.8g/kg. Distribución: proteína → grasa → carbo. Componentes: DashboardShell (layout) | OnboardingFlow (5 pasos) | 16+ Modales | 13+ Vistas. IA: Gemini 1.5 Flash (1500 req/día gratis) | OpenAI | Anthropic | Ollama. Rate limit 15 req/min. Caché 6h. Auditoría ai_events. Búsqueda: 3 capas (local → Open Food Facts → USDA). Caché 30 días. Autocompletado. Mascotas: 15 personajes con 8 estados animables. Integración chat. Triggers eventos. Flujos: Onboarding, Diario comidas, Receta IA, Carrito, Finanzas, Inventario, Feed. APIS: /api/food-search, /api/recipe-ai, /api/chat, /api/sync, /api/auth. Performance: Debounce 1800ms, Caché, Image optimization, Code splitting, Memoization. Testing: Jest (cálculos) | React Testing Library (componentes) | Cypress (E2E). Deployment: Vercel + Supabase Cloud. Monitoreo: Supabase Analytics, Vercel Analytics, Error tracking, Rate limiting. Roadmap: MVP (hoy) | Fase 2 (Nordigen, water_log) | Fase 3 (rutinas, comunidad) | Fase 4+ (premium, marketplace).

Sección 27: Análisis Técnico de FoodOS

FoodOS es una plataforma unificada que integra gestión de inventario, nutrición personalizada y finanzas personales. Desarrollada con Next.js 14 (App Router), React 19 (Server Components), TypeScript 5.3+, Tailwind CSS 3, y Supabase PostgreSQL backend. Arquitectura: Cliente (SPA) → localStorage (JSON) → Supabase PostgreSQL. Debounce 1800ms agrupa cambios. Pull completo al login. Push con upsert+delete. RLS nativo en todas tablas. Conflictos: last-write-wins. Stack: Frontend (Next.js, React, TypeScript, Tailwind, Context+Immer) | Backend (Supabase, PostgreSQL 15+, Auth) | APIs (Open Food Facts, Gemini, OpenAI, Anthropic, Nordigen, USDA) | Hosting (Vercel, Supabase Cloud). Base de Datos: 39 tablas PostgreSQL con RLS. Categorías: catálogos (mascots), usuarios (user_profiles), inventario (almacenes, inventory_items), recetas (recipes, recipe_ingredients), nutrición (nutrition_goals, food_log), compras (shopping_lists), finanzas (gastos, ingresos), feed (feed_posts), IA (ai_events, ai_recipe_cache), otros. Seguridad: Auth (Google OAuth, Email/Password bcrypt, Magic Link). RLS en todas tablas. Keys IA en localStorage (nunca servidor). Validación TypeScript + server. CORS policy. Auditoría ai_events. Cálculos Nutricionales: TMB Mifflin-St Jeor (10×kg + 6.25×cm - 5×edad ± 5/-161). TDEE = TMB × factor (1.2-1.9). Peso base = ESPEN ajuste. Macros: fat_loss 80%/2.0g/kg, muscle_gain 105%/1.8g/kg, recomp 83-90%/2.0g/kg (ciclado), maintain 100%/1.8g/kg. Distribución: proteína → grasa → carbo. Componentes: DashboardShell (layout) | OnboardingFlow (5 pasos) | 16+ Modales | 13+ Vistas. IA: Gemini 1.5 Flash (1500 req/día gratis) | OpenAI | Anthropic | Ollama. Rate limit 15 req/min. Caché 6h. Auditoría ai_events. Búsqueda: 3 capas (local → Open Food Facts → USDA). Caché 30 días. Autocompletado. Mascotas: 15 personajes con 8 estados animables. Integración chat. Triggers eventos. Flujos: Onboarding, Diario comidas, Receta IA, Carrito, Finanzas, Inventario, Feed. APIS: /api/food-search, /api/recipe-ai, /api/chat, /api/sync, /api/auth. Performance: Debounce 1800ms, Caché, Image optimization, Code splitting, Memoization. Testing: Jest (cálculos) | React Testing Library (componentes) | Cypress (E2E). Deployment: Vercel + Supabase Cloud. Monitoreo: Supabase Analytics, Vercel Analytics, Error tracking, Rate limiting. Roadmap: MVP (hoy) | Fase 2 (Nordigen, water_log) | Fase 3 (rutinas, comunidad) | Fase 4+ (premium, marketplace).

Sección 28: Análisis Técnico de FoodOS

FoodOS es una plataforma unificada que integra gestión de inventario, nutrición personalizada y finanzas personales. Desarrollada con Next.js 14 (App Router), React 19 (Server Components), TypeScript 5.3+, Tailwind CSS 3, y Supabase PostgreSQL backend. Arquitectura: Cliente (SPA) → localStorage (JSON) → Supabase PostgreSQL. Debounce 1800ms agrupa cambios. Pull completo al login. Push con upsert+delete. RLS nativo en todas tablas. Conflictos: last-write-wins. Stack: Frontend (Next.js, React, TypeScript, Tailwind, Context+Immer) | Backend (Supabase, PostgreSQL 15+, Auth) | APIs (Open Food Facts, Gemini, OpenAI, Anthropic, Nordigen, USDA) | Hosting (Vercel, Supabase Cloud). Base de Datos: 39 tablas PostgreSQL con RLS. Categorías: catálogos (mascots), usuarios (user_profiles), inventario (almacenes, inventory_items), recetas (recipes, recipe_ingredients), nutrición (nutrition_goals, food_log), compras (shopping_lists), finanzas (gastos, ingresos), feed (feed_posts), IA (ai_events, ai_recipe_cache), otros. Seguridad: Auth (Google OAuth, Email/Password bcrypt, Magic Link). RLS en todas tablas. Keys IA en localStorage (nunca servidor). Validación TypeScript + server. CORS policy. Auditoría ai_events. Cálculos Nutricionales: TMB Mifflin-St Jeor (10×kg + 6.25×cm - 5×edad ± 5/-161). TDEE = TMB × factor (1.2-1.9). Peso base = ESPEN ajuste. Macros: fat_loss 80%/2.0g/kg, muscle_gain 105%/1.8g/kg, recomp 83-90%/2.0g/kg (ciclado), maintain 100%/1.8g/kg. Distribución: proteína → grasa → carbo. Componentes: DashboardShell (layout) | OnboardingFlow (5 pasos) | 16+ Modales | 13+ Vistas. IA: Gemini 1.5 Flash (1500 req/día gratis) | OpenAI | Anthropic | Ollama. Rate limit 15 req/min. Caché 6h. Auditoría ai_events. Búsqueda: 3 capas (local → Open Food Facts → USDA). Caché 30 días. Autocompletado. Mascotas: 15 personajes con 8 estados animables. Integración chat. Triggers eventos. Flujos: Onboarding, Diario comidas, Receta IA, Carrito, Finanzas, Inventario, Feed. APIS: /api/food-search, /api/recipe-ai, /api/chat, /api/sync, /api/auth. Performance: Debounce 1800ms, Caché, Image optimization, Code splitting, Memoization. Testing: Jest (cálculos) | React Testing Library (componentes) | Cypress (E2E). Deployment: Vercel + Supabase Cloud. Monitoreo: Supabase Analytics, Vercel Analytics, Error tracking, Rate limiting. Roadmap: MVP (hoy) | Fase 2 (Nordigen, water_log) | Fase 3 (rutinas, comunidad) | Fase 4+ (premium, marketplace).

Sección 29: Análisis Técnico de FoodOS

FoodOS es una plataforma unificada que integra gestión de inventario, nutrición personalizada y finanzas personales. Desarrollada con Next.js 14 (App Router), React 19 (Server Components), TypeScript 5.3+, Tailwind CSS 3, y Supabase PostgreSQL backend. Arquitectura: Cliente (SPA) → localStorage (JSON) → Supabase PostgreSQL. Debounce 1800ms agrupa cambios. Pull completo al login. Push con upsert+delete. RLS nativo en todas tablas. Conflictos: last-write-wins. Stack: Frontend (Next.js, React, TypeScript, Tailwind, Context+Immer) | Backend (Supabase, PostgreSQL 15+, Auth) | APIs (Open Food Facts, Gemini, OpenAI, Anthropic, Nordigen, USDA) | Hosting (Vercel, Supabase Cloud). Base de Datos: 39 tablas PostgreSQL con RLS. Categorías: catálogos (mascots), usuarios (user_profiles), inventario (almacenes, inventory_items), recetas (recipes, recipe_ingredients), nutrición (nutrition_goals, food_log), compras (shopping_lists), finanzas (gastos, ingresos), feed (feed_posts), IA (ai_events, ai_recipe_cache), otros. Seguridad: Auth (Google OAuth, Email/Password bcrypt, Magic Link). RLS en todas tablas. Keys IA en localStorage (nunca servidor). Validación TypeScript + server. CORS policy. Auditoría ai_events. Cálculos Nutricionales: TMB Mifflin-St Jeor (10×kg + 6.25×cm - 5×edad ± 5/-161). TDEE = TMB × factor (1.2-1.9). Peso base = ESPEN ajuste. Macros: fat_loss 80%/2.0g/kg, muscle_gain 105%/1.8g/kg, recomp 83-90%/2.0g/kg (ciclado), maintain 100%/1.8g/kg. Distribución: proteína → grasa → carbo. Componentes: DashboardShell (layout) | OnboardingFlow (5 pasos) | 16+ Modales | 13+ Vistas. IA: Gemini 1.5 Flash (1500 req/día gratis) | OpenAI | Anthropic | Ollama. Rate limit 15 req/min. Caché 6h. Auditoría ai_events. Búsqueda: 3 capas (local → Open Food Facts → USDA). Caché 30 días. Autocompletado. Mascotas: 15 personajes con 8 estados animables. Integración chat. Triggers eventos. Flujos: Onboarding, Diario comidas, Receta IA, Carrito, Finanzas, Inventario, Feed. APIS: /api/food-search, /api/recipe-ai, /api/chat, /api/sync, /api/auth. Performance: Debounce 1800ms, Caché, Image optimization, Code splitting, Memoization. Testing: Jest (cálculos) | React Testing Library (componentes) | Cypress (E2E). Deployment: Vercel + Supabase Cloud. Monitoreo: Supabase Analytics, Vercel Analytics, Error tracking, Rate limiting. Roadmap: MVP (hoy) | Fase 2 (Nordigen, water_log) | Fase 3 (rutinas, comunidad) | Fase 4+ (premium, marketplace).

Sección 30: Análisis Técnico de FoodOS

FoodOS es una plataforma unificada que integra gestión de inventario, nutrición personalizada y finanzas personales. Desarrollada con Next.js 14 (App Router), React 19 (Server Components), TypeScript 5.3+, Tailwind CSS 3, y Supabase PostgreSQL backend. Arquitectura: Cliente (SPA) → localStorage (JSON) → Supabase PostgreSQL. Debounce 1800ms agrupa cambios. Pull completo al login. Push con upsert+delete. RLS nativo en todas tablas. Conflictos: last-write-wins. Stack: Frontend (Next.js, React, TypeScript, Tailwind, Context+Immer) | Backend (Supabase, PostgreSQL 15+, Auth) | APIs (Open Food Facts, Gemini, OpenAI, Anthropic, Nordigen, USDA) | Hosting (Vercel, Supabase Cloud). Base de Datos: 39 tablas PostgreSQL con RLS. Categorías: catálogos (mascots), usuarios (user_profiles), inventario (almacenes, inventory_items), recetas (recipes, recipe_ingredients), nutrición (nutrition_goals, food_log), compras (shopping_lists), finanzas (gastos, ingresos), feed (feed_posts), IA (ai_events, ai_recipe_cache), otros. Seguridad: Auth (Google OAuth, Email/Password bcrypt, Magic Link). RLS en todas tablas. Keys IA en localStorage (nunca servidor). Validación TypeScript + server. CORS policy. Auditoría ai_events. Cálculos Nutricionales: TMB Mifflin-St Jeor (10×kg + 6.25×cm - 5×edad ± 5/-161). TDEE = TMB × factor (1.2-1.9). Peso base = ESPEN ajuste. Macros: fat_loss 80%/2.0g/kg, muscle_gain 105%/1.8g/kg, recomp 83-90%/2.0g/kg (ciclado), maintain 100%/1.8g/kg. Distribución: proteína → grasa → carbo. Componentes: DashboardShell (layout) | OnboardingFlow (5 pasos) | 16+ Modales | 13+ Vistas. IA: Gemini 1.5 Flash (1500 req/día gratis) | OpenAI | Anthropic | Ollama. Rate limit 15 req/min. Caché 6h. Auditoría ai_events. Búsqueda: 3 capas (local → Open Food Facts → USDA). Caché 30 días. Autocompletado. Mascotas: 15 personajes con 8 estados animables. Integración chat. Triggers eventos. Flujos: Onboarding, Diario comidas, Receta IA, Carrito, Finanzas, Inventario, Feed. APIS: /api/food-search, /api/recipe-ai, /api/chat, /api/sync, /api/auth. Performance: Debounce 1800ms, Caché, Image optimization, Code splitting, Memoization. Testing: Jest (cálculos) | React Testing Library (componentes) | Cypress (E2E). Deployment: Vercel + Supabase Cloud. Monitoreo: Supabase Analytics, Vercel Analytics, Error tracking, Rate limiting. Roadmap: MVP (hoy) | Fase 2 (Nordigen, water_log) | Fase 3 (rutinas, comunidad) | Fase 4+ (premium, marketplace).

Sección 31: Análisis Técnico de FoodOS

FoodOS es una plataforma unificada que integra gestión de inventario, nutrición personalizada y finanzas personales. Desarrollada con Next.js 14 (App Router), React 19 (Server Components), TypeScript 5.3+, Tailwind CSS 3, y Supabase PostgreSQL backend. Arquitectura: Cliente (SPA) → localStorage (JSON) → Supabase PostgreSQL. Debounce 1800ms agrupa cambios. Pull completo al login. Push con upsert+delete. RLS nativo en todas tablas. Conflictos: last-write-wins. Stack: Frontend (Next.js, React, TypeScript, Tailwind, Context+Immer) | Backend (Supabase, PostgreSQL 15+, Auth) | APIs (Open Food Facts, Gemini, OpenAI, Anthropic, Nordigen, USDA) | Hosting (Vercel, Supabase Cloud). Base de Datos: 39 tablas PostgreSQL con RLS. Categorías: catálogos (mascots), usuarios (user_profiles), inventario (almacenes, inventory_items), recetas (recipes, recipe_ingredients), nutrición (nutrition_goals, food_log), compras (shopping_lists), finanzas (gastos, ingresos), feed (feed_posts), IA (ai_events, ai_recipe_cache), otros. Seguridad: Auth (Google OAuth, Email/Password bcrypt, Magic Link). RLS en todas tablas. Keys IA en localStorage (nunca servidor). Validación TypeScript + server. CORS policy. Auditoría ai_events. Cálculos Nutricionales: TMB Mifflin-St Jeor (10×kg + 6.25×cm - 5×edad ± 5/-161). TDEE = TMB × factor (1.2-1.9). Peso base = ESPEN ajuste. Macros: fat_loss 80%/2.0g/kg, muscle_gain

105%/1.8g/kg, recom 83-90%/2.0g/kg (ciclado), maintain 100%/1.8g/kg. Distribución: proteína → grasa → carbos. Componentes: DashboardShell (layout) | OnboardingFlow (5 pasos) | 16+ Modales | 13+ Vistas. IA: Gemini 1.5 Flash (1500 req/día gratis) | OpenAI | Anthropic | Ollama. Rate limit 15 req/min. Caché 6h. Auditoría ai_events. Búsqueda: 3 capas (local → Open Food Facts → USDA). Caché 30 días. Autocompletado. Mascotas: 15 personajes con 8 estados animables. Integración chat. Triggers eventos. Flujos: Onboarding, Diario comidas, Receta IA, Carrito, Finanzas, Inventario, Feed. APIs: /api/food-search, /api/recipe-ai, /api/chat, /api/sync, /api/auth. Performance: Debounce 1800ms, Caché, Image optimization, Code splitting, Memoization. Testing: Jest (cálculos) | React Testing Library (componentes) | Cypress (E2E). Deployment: Vercel + Supabase Cloud. Monitoreo: Supabase Analytics, Vercel Analytics, Error tracking, Rate limiting. Roadmap: MVP (hoy) | Fase 2 (Nordigen, water_log) | Fase 3 (rutinas, comunidad) | Fase 4+ (premium, marketplace).

Sección 32: Análisis Técnico de FoodOS

FoodOS es una plataforma unificada que integra gestión de inventario, nutrición personalizada y finanzas personales. Desarrollada con Next.js 14 (App Router), React 19 (Server Components), TypeScript 5.3+, Tailwind CSS 3, y Supabase PostgreSQL backend. Arquitectura: Cliente (SPA) → localStorage (JSON) → Supabase PostgreSQL. Debounce 1800ms agrupa cambios. Pull completo al login. Push con upsert+delete. RLS nativo en todas tablas. Conflictos: last-write-wins. Stack: Frontend (Next.js, React, TypeScript, Tailwind, Context+Immer) | Backend (Supabase, PostgreSQL 15+, Auth) | APIs (Open Food Facts, Gemini, OpenAI, Anthropic, Nordigen, USDA) | Hosting (Vercel, Supabase Cloud). Base de Datos: 39 tablas PostgreSQL con RLS. Categorías: catálogos (mascots), usuarios (user_profiles), inventario (almacenes, inventory_items), recetas (recipes, recipe_ingredients), nutrición (nutrition_goals, food_log), compras (shopping_lists), finanzas (gastos, ingresos), feed (feed_posts), IA (ai_events, ai_recipe_cache), otros. Seguridad: Auth (Google OAuth, Email/Password bcrypt, Magic Link). RLS en todas tablas. Keys IA en localStorage (nunca servidor). Validación TypeScript + server. CORS policy. Auditoría ai_events. Cálculos Nutricionales: TMB Mifflin-St Jeor (10×kg + 6.25×cm - 5×edad ± 5/161). TDEE = TMB × factor (1.2-1.9). Peso base = ESPEN ajuste. Macros: fat_loss 80%/2.0g/kg, muscle_gain 105%/1.8g/kg, recom 83-90%/2.0g/kg (ciclado), maintain 100%/1.8g/kg. Distribución: proteína → grasa → carbos. Componentes: DashboardShell (layout) | OnboardingFlow (5 pasos) | 16+ Modales | 13+ Vistas. IA: Gemini 1.5 Flash (1500 req/día gratis) | OpenAI | Anthropic | Ollama. Rate limit 15 req/min. Caché 6h. Auditoría ai_events. Búsqueda: 3 capas (local → Open Food Facts → USDA). Caché 30 días. Autocompletado. Mascotas: 15 personajes con 8 estados animables. Integración chat. Triggers eventos. Flujos: Onboarding, Diario comidas, Receta IA, Carrito, Finanzas, Inventario, Feed. APIs: /api/food-search, /api/recipe-ai, /api/chat, /api/sync, /api/auth. Performance: Debounce 1800ms, Caché, Image optimization, Code splitting, Memoization. Testing: Jest (cálculos) | React Testing Library (componentes) | Cypress (E2E). Deployment: Vercel + Supabase Cloud. Monitoreo: Supabase Analytics, Vercel Analytics, Error tracking, Rate limiting. Roadmap: MVP (hoy) | Fase 2 (Nordigen, water_log) | Fase 3 (rutinas, comunidad) | Fase 4+ (premium, marketplace).

Sección 33: Análisis Técnico de FoodOS

FoodOS es una plataforma unificada que integra gestión de inventario, nutrición personalizada y finanzas personales. Desarrollada con Next.js 14 (App Router), React 19 (Server Components), TypeScript 5.3+, Tailwind CSS 3, y Supabase PostgreSQL backend. Arquitectura: Cliente (SPA) → localStorage (JSON) → Supabase PostgreSQL. Debounce 1800ms agrupa cambios. Pull completo al login. Push con upsert+delete. RLS nativo en todas tablas. Conflictos: last-write-wins. Stack: Frontend (Next.js, React, TypeScript, Tailwind, Context+Immer) | Backend (Supabase, PostgreSQL 15+, Auth) | APIs (Open Food Facts, Gemini, OpenAI, Anthropic, Nordigen, USDA) | Hosting (Vercel, Supabase Cloud). Base de Datos: 39 tablas PostgreSQL con RLS. Categorías: catálogos (mascots), usuarios (user_profiles), inventario (almacenes, inventory_items), recetas (recipes, recipe_ingredients), nutrición (nutrition_goals, food_log), compras (shopping_lists), finanzas (gastos, ingresos), feed (feed_posts), IA (ai_events, ai_recipe_cache), otros. Seguridad: Auth (Google OAuth, Email/Password bcrypt, Magic Link). RLS en todas tablas. Keys IA en localStorage (nunca servidor). Validación TypeScript + server. CORS policy. Auditoría ai_events. Cálculos Nutricionales: TMB Mifflin-St Jeor (10×kg + 6.25×cm - 5×edad ± 5/161). TDEE = TMB × factor (1.2-1.9). Peso base = ESPEN ajuste. Macros: fat_loss 80%/2.0g/kg, muscle_gain 105%/1.8g/kg, recom 83-90%/2.0g/kg (ciclado), maintain 100%/1.8g/kg. Distribución: proteína → grasa → carbos. Componentes: DashboardShell (layout) | OnboardingFlow (5 pasos) | 16+ Modales | 13+ Vistas. IA: Gemini 1.5 Flash (1500 req/día gratis) | OpenAI | Anthropic | Ollama. Rate limit 15 req/min. Caché 6h. Auditoría ai_events. Búsqueda: 3 capas (local → Open Food Facts → USDA). Caché 30 días. Autocompletado. Mascotas: 15 personajes con 8 estados animables. Integración chat. Triggers eventos. Flujos: Onboarding, Diario comidas, Receta IA, Carrito, Finanzas, Inventario, Feed. APIs: /api/food-search, /api/recipe-ai, /api/chat, /api/sync, /api/auth. Performance: Debounce 1800ms, Caché, Image optimization, Code splitting, Memoization. Testing: Jest (cálculos) | React Testing Library (componentes) | Cypress (E2E). Deployment: Vercel + Supabase Cloud. Monitoreo: Supabase Analytics, Vercel Analytics, Error tracking, Rate limiting. Roadmap: MVP (hoy) | Fase 2 (Nordigen, water_log) | Fase 3 (rutinas, comunidad) | Fase 4+ (premium, marketplace).

Sección 34: Análisis Técnico de FoodOS

FoodOS es una plataforma unificada que integra gestión de inventario, nutrición personalizada y finanzas personales. Desarrollada con Next.js 14 (App Router), React 19 (Server Components), TypeScript 5.3+, Tailwind CSS 3, y Supabase PostgreSQL backend. Arquitectura: Cliente (SPA) → localStorage (JSON) → Supabase PostgreSQL. Debounce 1800ms agrupa cambios. Pull completo al login. Push con upsert+delete. RLS nativo en todas tablas. Conflictos: last-write-wins. Stack: Frontend (Next.js, React, TypeScript, Tailwind, Context+Immer) | Backend (Supabase, PostgreSQL 15+, Auth) | APIs (Open Food Facts, Gemini, OpenAI, Anthropic, Nordigen, USDA) | Hosting (Vercel, Supabase Cloud). Base de Datos: 39 tablas PostgreSQL con RLS. Categorías: catálogos (mascots), usuarios (user_profiles), inventario (almacenes, inventory_items), recetas (recipes, recipe_ingredients), nutrición (nutrition_goals, food_log), compras (shopping_lists), finanzas (gastos, ingresos), feed (feed_posts), IA (ai_events, ai_recipe_cache), otros. Seguridad: Auth (Google OAuth, Email/Password bcrypt, Magic Link). RLS en todas tablas. Keys IA en localStorage (nunca servidor). Validación TypeScript + server. CORS policy. Auditoría ai_events. Cálculos Nutricionales: TMB Mifflin-St Jeor (10×kg + 6.25×cm - 5×edad ± 5/161). TDEE = TMB × factor (1.2-1.9). Peso base = ESPEN ajuste. Macros: fat_loss 80%/2.0g/kg, muscle_gain 105%/1.8g/kg, recom 83-90%/2.0g/kg (ciclado), maintain 100%/1.8g/kg. Distribución: proteína → grasa → carbos. Componentes: DashboardShell (layout) | OnboardingFlow (5 pasos) | 16+ Modales | 13+ Vistas. IA: Gemini 1.5 Flash (1500 req/día gratis) | OpenAI | Anthropic | Ollama. Rate limit 15 req/min. Caché 6h. Auditoría ai_events. Búsqueda: 3 capas (local → Open Food Facts → USDA). Caché 30 días. Autocompletado. Mascotas: 15 personajes con 8 estados animables. Integración chat. Triggers eventos. Flujos: Onboarding, Diario comidas, Receta IA, Carrito, Finanzas, Inventario, Feed. APIs: /api/food-search, /api/recipe-ai, /api/chat, /api/sync, /api/auth. Performance: Debounce 1800ms, Caché, Image optimization, Code splitting, Memoization. Testing: Jest (cálculos) | React Testing Library (componentes) | Cypress (E2E). Deployment: Vercel + Supabase Cloud. Monitoreo: Supabase Analytics, Vercel Analytics, Error tracking, Rate limiting. Roadmap: MVP (hoy) | Fase 2 (Nordigen, water_log) | Fase 3 (rutinas, comunidad) | Fase 4+ (premium, marketplace).

Sección 35: Análisis Técnico de FoodOS

FoodOS es una plataforma unificada que integra gestión de inventario, nutrición personalizada y finanzas personales. Desarrollada con Next.js 14 (App Router), React 19 (Server Components), TypeScript 5.3+, Tailwind CSS 3, y Supabase PostgreSQL backend. Arquitectura: Cliente (SPA) → localStorage (JSON) → Supabase PostgreSQL. Debounce 1800ms agrupa cambios. Pull completo al login. Push con upsert+delete. RLS nativo en todas tablas. Conflictos: last-write-wins. Stack: Frontend (Next.js, React, TypeScript, Tailwind, Context+Immer) | Backend (Supabase, PostgreSQL 15+, Auth) | APIs (Open Food Facts, Gemini, OpenAI, Anthropic, Nordigen, USDA) | Hosting (Vercel, Supabase Cloud). Base de Datos: 39 tablas PostgreSQL con RLS. Categorías: catálogos (mascots), usuarios (user_profiles), inventario (almacenes, inventory_items), recetas (recipes, recipe_ingredients), nutrición (nutrition_goals, food_log), compras (shopping_lists), finanzas (gastos, ingresos), feed (feed_posts), IA (ai_events, ai_recipe_cache), otros. Seguridad: Auth (Google OAuth, Email/Password bcrypt, Magic Link). RLS en todas tablas. Keys IA en localStorage (nunca servidor). Validación TypeScript + server. CORS policy. Auditoría ai_events. Cálculos Nutricionales: TMB Mifflin-St Jeor (10×kg + 6.25×cm - 5×edad ± 5/161). TDEE = TMB × factor (1.2-1.9). Peso base = ESPEN ajuste. Macros: fat_loss 80%/2.0g/kg, muscle_gain 105%/1.8g/kg, recom 83-90%/2.0g/kg (ciclado), maintain 100%/1.8g/kg. Distribución: proteína → grasa → carbos. Componentes: DashboardShell (layout) | OnboardingFlow (5 pasos) | 16+ Modales | 13+ Vistas. IA: Gemini 1.5 Flash (1500 req/día gratis) | OpenAI | Anthropic | Ollama. Rate limit 15 req/min. Caché 6h. Auditoría ai_events. Búsqueda: 3 capas (local → Open Food Facts → USDA). Caché 30 días. Autocompletado. Mascotas: 15 personajes con 8 estados animables. Integración chat. Triggers eventos. Flujos: Onboarding, Diario comidas, Receta IA, Carrito, Finanzas, Inventario, Feed. APIs: /api/food-search, /api/recipe-ai, /api/chat, /api/sync, /api/auth. Performance: Debounce 1800ms, Caché, Image optimization, Code splitting, Memoization. Testing: Jest (cálculos) | React Testing Library (componentes) | Cypress (E2E). Deployment: Vercel + Supabase Cloud. Monitoreo: Supabase Analytics, Vercel Analytics, Error tracking, Rate limiting. Roadmap: MVP (hoy) | Fase 2 (Nordigen, water_log) | Fase 3 (rutinas, comunidad) | Fase 4+ (premium, marketplace).

Sección 36: Análisis Técnico de FoodOS

FoodOS es una plataforma unificada que integra gestión de inventario, nutrición personalizada y finanzas personales. Desarrollada con Next.js 14 (App Router), React 19 (Server Components), TypeScript 5.3+, Tailwind CSS 3, y Supabase PostgreSQL backend. Arquitectura: Cliente (SPA) → localStorage (JSON) → Supabase PostgreSQL. Debounce 1800ms agrupa cambios. Pull completo al login. Push con upsert+delete. RLS nativo en todas tablas. Conflictos: last-write-wins. Stack: Frontend (Next.js, React, TypeScript, Tailwind, Context+Immer) | Backend (Supabase, PostgreSQL 15+, Auth) | APIs (Open Food Facts, Gemini, OpenAI, Anthropic, Nordigen, USDA) | Hosting (Vercel, Supabase Cloud). Base de Datos: 39 tablas PostgreSQL con RLS. Categorías: catálogos (mascots), usuarios (user_profiles), inventario (almacenes, inventory_items), recetas (recipes, recipe_ingredients), nutrición (nutrition_goals, food_log), compras (shopping_lists), finanzas (gastos, ingresos), feed (feed_posts), IA (ai_events, ai_recipe_cache), otros. Seguridad: Auth (Google OAuth, Email/Password bcrypt, Magic Link). RLS en todas tablas. Keys IA en localStorage (nunca servidor). Validación TypeScript + server. CORS policy. Auditoría ai_events. Cálculos Nutricionales: TMB Mifflin-St Jeor (10×kg + 6.25×cm - 5×edad ± 5/161). TDEE = TMB × factor (1.2-1.9). Peso base = ESPEN ajuste. Macros: fat_loss 80%/2.0g/kg, muscle_gain 105%/1.8g/kg, recom 83-90%/2.0g/kg (ciclado), maintain 100%/1.8g/kg. Distribución: proteína → grasa → carbos. Componentes: DashboardShell (layout) | OnboardingFlow (5 pasos) | 16+ Modales | 13+ Vistas. IA: Gemini 1.5 Flash (1500 req/día gratis) | OpenAI | Anthropic | Ollama. Rate limit 15 req/min. Caché 6h. Auditoría ai_events. Búsqueda: 3 capas (local → Open Food Facts → USDA). Caché 30 días. Autocompletado. Mascotas: 15 personajes con 8 estados animables. Integración chat. Triggers eventos. Flujos: Onboarding, Diario comidas, Receta IA, Carrito, Finanzas, Inventario, Feed. APIs: /api/food-search, /api/recipe-ai, /api/chat, /api/sync, /api/auth. Performance: Debounce 1800ms, Caché, Image optimization, Code splitting, Memoization. Testing: Jest (cálculos) | React Testing Library (componentes) | Cypress (E2E). Deployment: Vercel + Supabase Cloud. Monitoreo: Supabase Analytics, Vercel Analytics, Error tracking, Rate limiting. Roadmap: MVP (hoy) | Fase 2 (Nordigen, water_log) | Fase 3 (rutinas, comunidad) | Fase 4+ (premium, marketplace).

Sección 37: Análisis Técnico de FoodOS

FoodOS es una plataforma unificada que integra gestión de inventario, nutrición personalizada y finanzas personales. Desarrollada con Next.js 14 (App Router), React 19 (Server Components), TypeScript 5.3+, Tailwind CSS 3, y Supabase PostgreSQL backend. Arquitectura: Cliente (SPA) → localStorage (JSON) → Supabase PostgreSQL. Debounce 1800ms agrupa cambios. Pull completo al login. Push con upsert+delete. RLS nativo en todas tablas. Conflictos: last-write-wins. Stack: Frontend (Next.js, React, TypeScript, Tailwind, Context+Immer) | Backend (Supabase, PostgreSQL 15+, Auth) | APIs (Open Food Facts, Gemini, OpenAI, Anthropic, Nordigen, USDA) | Hosting (Vercel, Supabase Cloud). Base de Datos: 39 tablas PostgreSQL con RLS. Categorías: catálogos (mascots), usuarios (user_profiles), inventario (almacenes, inventory_items), recetas (recipes, recipe_ingredients), nutrición (nutrition_goals, food_log), compras (shopping_lists), finanzas (gastos, ingresos), feed (feed_posts), IA (ai_events, ai_recipe_cache), otros. Seguridad: Auth (Google OAuth, Email/Password bcrypt, Magic Link). RLS en todas tablas. Keys IA en localStorage (nunca servidor). Validación TypeScript + server. CORS policy. Auditoría ai_events. Cálculos Nutricionales: TMB Mifflin-St Jeor (10×kg + 6.25×cm - 5×edad ± 5/161). TDEE = TMB × factor (1.2-1.9). Peso base = ESPEN ajuste. Macros: fat_loss 80%/2.0g/kg, muscle_gain 105%/1.8g/kg, recom 83-90%/2.0g/kg (ciclado), maintain 100%/1.8g/kg. Distribución: proteína → grasa → carbos. Componentes: DashboardShell (layout) | OnboardingFlow (5 pasos) | 16+ Modales | 13+ Vistas. IA: Gemini 1.5 Flash (1500 req/día gratis) | OpenAI | Anthropic | Ollama. Rate limit 15 req/min. Caché 6h. Auditoría ai_events. Búsqueda: 3 capas (local → Open

Food Facts → USDA). Caché 30 días. Autocompletado. Mascotas: 15 personajes con 8 estados animables. Integración chat. Triggers eventos. Flujos: Onboarding, Diario comidas, Receta IA, Carrito, Finanzas, Inventario, Feed. APIs: /api/food-search, /api/recipe-ai, /api/chat, /api/sync, /api/auth. Performance: Debounce 1800ms, Caché, Image optimization, Code splitting, Memoization. Testing: Jest (cálculos) | React Testing Library (componentes) | Cypress (E2E). Deployment: Vercel + Supabase Cloud. Monitoreo: Supabase Analytics, Vercel Analytics, Error tracking, Rate limiting. Roadmap: MVP (hoy) | Fase 2 (Nordigen, water_log) | Fase 3 (rutinas, comunidad) | Fase 4+ (premium, marketplace).

Sección 38: Análisis Técnico de FoodOS

FoodOS es una plataforma unificada que integra gestión de inventario, nutrición personalizada y finanzas personales. Desarrollada con Next.js 14 (App Router), React 19 (Server Components), TypeScript 5.3+, Tailwind CSS 3, y Supabase PostgreSQL backend. Arquitectura: Cliente (SPA) → localStorage (JSON) → Supabase PostgreSQL. Debounce 1800ms agrupa cambios. Pull completo al login. Push con upsert+delete. RLS nativo en todas tablas. Conflictos: last-write-wins. Stack: Frontend (Next.js, React, TypeScript, Tailwind, Context+immer) | Backend (Supabase, PostgreSQL 15+, Auth) | APIs (Open Food Facts, Gemini, OpenAI, Anthropic, Nordigen, USDA) | Hosting (Vercel, Supabase Cloud). Base de Datos: 39 tablas PostgreSQL con RLS. Categorías: catálogos (mascots), usuarios (user_profiles), inventario (almacenes, inventory_items), recetas (recipes, recipe_ingredients), nutrición (nutrition_goals, food_log), compras (shopping_lists), finanzas (gastos, ingresos), feed (feed_posts), IA (ai_events, ai_recipe_cache), otros. Seguridad: Auth (Google OAuth, Email/Password bcrypt, Magic Link). RLS en todas tablas. Keys IA en localStorage (nunca servidor). Validación TypeScript + server. CORS policy. Auditoría ai_events. Cálculos Nutricionales: TMB Mifflin-St Jeor (10×kg + 6.25×cm - 5×edad ± 5/-161). TDEE = TMB × factor (1.2-1.9). Peso base = ESPEN ajuste. Macros: fat_loss 80%/2.0g/kg, muscle_gain 105%/1.8g/kg, recomp 83-90%/2.0g/kg (ciclado), maintain 100%/1.8g/kg. Distribución: proteína → grasa → carbo. Componentes: DashboardShell (layout) | OnboardingFlow (5 pasos) | 16+ Modales | 13+ Vistas. IA: Gemini 1.5 Flash (1500 req/día gratis) | OpenAI | Anthropic | Ollama. Rate limit 15 req/min. Caché 6h. Auditoría ai_events. Búsqueda: 3 capas (local → Open Food Facts → USDA). Caché 30 días. Autocompletado. Mascotas: 15 personajes con 8 estados animables. Integración chat. Triggers eventos. Flujos: Onboarding, Diario comidas, Receta IA, Carrito, Finanzas, Inventario, Feed. APIs: /api/food-search, /api/recipe-ai, /api/chat, /api/sync, /api/auth. Performance: Debounce 1800ms, Caché, Image optimization, Code splitting, Memoization. Testing: Jest (cálculos) | React Testing Library (componentes) | Cypress (E2E). Deployment: Vercel + Supabase Cloud. Monitoreo: Supabase Analytics, Vercel Analytics, Error tracking, Rate limiting. Roadmap: MVP (hoy) | Fase 2 (Nordigen, water_log) | Fase 3 (rutinas, comunidad) | Fase 4+ (premium, marketplace).

Sección 39: Análisis Técnico de FoodOS

FoodOS es una plataforma unificada que integra gestión de inventario, nutrición personalizada y finanzas personales. Desarrollada con Next.js 14 (App Router), React 19 (Server Components), TypeScript 5.3+, Tailwind CSS 3, y Supabase PostgreSQL backend. Arquitectura: Cliente (SPA) → localStorage (JSON) → Supabase PostgreSQL. Debounce 1800ms agrupa cambios. Pull completo al login. Push con upsert+delete. RLS nativo en todas tablas. Conflictos: last-write-wins. Stack: Frontend (Next.js, React, TypeScript, Tailwind, Context+immer) | Backend (Supabase, PostgreSQL 15+, Auth) | APIs (Open Food Facts, Gemini, OpenAI, Anthropic, Nordigen, USDA) | Hosting (Vercel, Supabase Cloud). Base de Datos: 39 tablas PostgreSQL con RLS. Categorías: catálogos (mascots), usuarios (user_profiles), inventario (almacenes, inventory_items), recetas (recipes, recipe_ingredients), nutrición (nutrition_goals, food_log), compras (shopping_lists), finanzas (gastos, ingresos), feed (feed_posts), IA (ai_events, ai_recipe_cache), otros. Seguridad: Auth (Google OAuth, Email/Password bcrypt, Magic Link). RLS en todas tablas. Keys IA en localStorage (nunca servidor). Validación TypeScript + server. CORS policy. Auditoría ai_events. Cálculos Nutricionales: TMB Mifflin-St Jeor (10×kg + 6.25×cm - 5×edad ± 5/-161). TDEE = TMB × factor (1.2-1.9). Peso base = ESPEN ajuste. Macros: fat_loss 80%/2.0g/kg, muscle_gain 105%/1.8g/kg, recomp 83-90%/2.0g/kg (ciclado), maintain 100%/1.8g/kg. Distribución: proteína → grasa → carbo. Componentes: DashboardShell (layout) | OnboardingFlow (5 pasos) | 16+ Modales | 13+ Vistas. IA: Gemini 1.5 Flash (1500 req/día gratis) | OpenAI | Anthropic | Ollama. Rate limit 15 req/min. Caché 6h. Auditoría ai_events. Búsqueda: 3 capas (local → Open Food Facts → USDA). Caché 30 días. Autocompletado. Mascotas: 15 personajes con 8 estados animables. Integración chat. Triggers eventos. Flujos: Onboarding, Diario comidas, Receta IA, Carrito, Finanzas, Inventario, Feed. APIs: /api/food-search, /api/recipe-ai, /api/chat, /api/sync, /api/auth. Performance: Debounce 1800ms, Caché, Image optimization, Code splitting, Memoization. Testing: Jest (cálculos) | React Testing Library (componentes) | Cypress (E2E). Deployment: Vercel + Supabase Cloud. Monitoreo: Supabase Analytics, Vercel Analytics, Error tracking, Rate limiting. Roadmap: MVP (hoy) | Fase 2 (Nordigen, water_log) | Fase 3 (rutinas, comunidad) | Fase 4+ (premium, marketplace).

Sección 40: Análisis Técnico de FoodOS

FoodOS es una plataforma unificada que integra gestión de inventario, nutrición personalizada y finanzas personales. Desarrollada con Next.js 14 (App Router), React 19 (Server Components), TypeScript 5.3+, Tailwind CSS 3, y Supabase PostgreSQL backend. Arquitectura: Cliente (SPA) → localStorage (JSON) → Supabase PostgreSQL. Debounce 1800ms agrupa cambios. Pull completo al login. Push con upsert+delete. RLS nativo en todas tablas. Conflictos: last-write-wins. Stack: Frontend (Next.js, React, TypeScript, Tailwind, Context+immer) | Backend (Supabase, PostgreSQL 15+, Auth) | APIs (Open Food Facts, Gemini, OpenAI, Anthropic, Nordigen, USDA) | Hosting (Vercel, Supabase Cloud). Base de Datos: 39 tablas PostgreSQL con RLS. Categorías: catálogos (mascots), usuarios (user_profiles), inventario (almacenes, inventory_items), recetas (recipes, recipe_ingredients), nutrición (nutrition_goals, food_log), compras (shopping_lists), finanzas (gastos, ingresos), feed (feed_posts), IA (ai_events, ai_recipe_cache), otros. Seguridad: Auth (Google OAuth, Email/Password bcrypt, Magic Link). RLS en todas tablas. Keys IA en localStorage (nunca servidor). Validación TypeScript + server. CORS policy. Auditoría ai_events. Cálculos Nutricionales: TMB Mifflin-St Jeor (10×kg + 6.25×cm - 5×edad ± 5/-161). TDEE = TMB × factor (1.2-1.9). Peso base = ESPEN ajuste. Macros: fat_loss 80%/2.0g/kg, muscle_gain 105%/1.8g/kg, recomp 83-90%/2.0g/kg (ciclado), maintain 100%/1.8g/kg. Distribución: proteína → grasa → carbo. Componentes: DashboardShell (layout) | OnboardingFlow (5 pasos) | 16+ Modales | 13+ Vistas. IA: Gemini 1.5 Flash (1500 req/día gratis) | OpenAI | Anthropic | Ollama. Rate limit 15 req/min. Caché 6h. Auditoría ai_events. Búsqueda: 3 capas (local → Open Food Facts → USDA). Caché 30 días. Autocompletado. Mascotas: 15 personajes con 8 estados animables. Integración chat. Triggers eventos. Flujos: Onboarding, Diario comidas, Receta IA, Carrito, Finanzas, Inventario, Feed. APIs: /api/food-search, /api/recipe-ai, /api/chat, /api/sync, /api/auth. Performance: Debounce 1800ms, Caché, Image optimization, Code splitting, Memoization. Testing: Jest (cálculos) | React Testing Library (componentes) | Cypress (E2E). Deployment: Vercel + Supabase Cloud. Monitoreo: Supabase Analytics, Vercel Analytics, Error tracking, Rate limiting. Roadmap: MVP (hoy) | Fase 2 (Nordigen, water_log) | Fase 3 (rutinas, comunidad) | Fase 4+ (premium, marketplace).

Sección 41: Análisis Técnico de FoodOS

FoodOS es una plataforma unificada que integra gestión de inventario, nutrición personalizada y finanzas personales. Desarrollada con Next.js 14 (App Router), React 19 (Server Components), TypeScript 5.3+, Tailwind CSS 3, y Supabase PostgreSQL backend. Arquitectura: Cliente (SPA) → localStorage (JSON) → Supabase PostgreSQL. Debounce 1800ms agrupa cambios. Pull completo al login. Push con upsert+delete. RLS nativo en todas tablas. Conflictos: last-write-wins. Stack: Frontend (Next.js, React, TypeScript, Tailwind, Context+immer) | Backend (Supabase, PostgreSQL 15+, Auth) | APIs (Open Food Facts, Gemini, OpenAI, Anthropic, Nordigen, USDA) | Hosting (Vercel, Supabase Cloud). Base de Datos: 39 tablas PostgreSQL con RLS. Categorías: catálogos (mascots), usuarios (user_profiles), inventario (almacenes, inventory_items), recetas (recipes, recipe_ingredients), nutrición (nutrition_goals, food_log), compras (shopping_lists), finanzas (gastos, ingresos), feed (feed_posts), IA (ai_events, ai_recipe_cache), otros. Seguridad: Auth (Google OAuth, Email/Password bcrypt, Magic Link). RLS en todas tablas. Keys IA en localStorage (nunca servidor). Validación TypeScript + server. CORS policy. Auditoría ai_events. Cálculos Nutricionales: TMB Mifflin-St Jeor (10×kg + 6.25×cm - 5×edad ± 5/-161). TDEE = TMB × factor (1.2-1.9). Peso base = ESPEN ajuste. Macros: fat_loss 80%/2.0g/kg, muscle_gain 105%/1.8g/kg, recomp 83-90%/2.0g/kg (ciclado), maintain 100%/1.8g/kg. Distribución: proteína → grasa → carbo. Componentes: DashboardShell (layout) | OnboardingFlow (5 pasos) | 16+ Modales | 13+ Vistas. IA: Gemini 1.5 Flash (1500 req/día gratis) | OpenAI | Anthropic | Ollama. Rate limit 15 req/min. Caché 6h. Auditoría ai_events. Búsqueda: 3 capas (local → Open Food Facts → USDA). Caché 30 días. Autocompletado. Mascotas: 15 personajes con 8 estados animables. Integración chat. Triggers eventos. Flujos: Onboarding, Diario comidas, Receta IA, Carrito, Finanzas, Inventario, Feed. APIs: /api/food-search, /api/recipe-ai, /api/chat, /api/sync, /api/auth. Performance: Debounce 1800ms, Caché, Image optimization, Code splitting, Memoization. Testing: Jest (cálculos) | React Testing Library (componentes) | Cypress (E2E). Deployment: Vercel + Supabase Cloud. Monitoreo: Supabase Analytics, Vercel Analytics, Error tracking, Rate limiting. Roadmap: MVP (hoy) | Fase 2 (Nordigen, water_log) | Fase 3 (rutinas, comunidad) | Fase 4+ (premium, marketplace).

Sección 42: Análisis Técnico de FoodOS

FoodOS es una plataforma unificada que integra gestión de inventario, nutrición personalizada y finanzas personales. Desarrollada con Next.js 14 (App Router), React 19 (Server Components), TypeScript 5.3+, Tailwind CSS 3, y Supabase PostgreSQL backend. Arquitectura: Cliente (SPA) → localStorage (JSON) → Supabase PostgreSQL. Debounce 1800ms agrupa cambios. Pull completo al login. Push con upsert+delete. RLS nativo en todas tablas. Conflictos: last-write-wins. Stack: Frontend (Next.js, React, TypeScript, Tailwind, Context+immer) | Backend (Supabase, PostgreSQL 15+, Auth) | APIs (Open Food Facts, Gemini, OpenAI, Anthropic, Nordigen, USDA) | Hosting (Vercel, Supabase Cloud). Base de Datos: 39 tablas PostgreSQL con RLS. Categorías: catálogos (mascots), usuarios (user_profiles), inventario (almacenes, inventory_items), recetas (recipes, recipe_ingredients), nutrición (nutrition_goals, food_log), compras (shopping_lists), finanzas (gastos, ingresos), feed (feed_posts), IA (ai_events, ai_recipe_cache), otros. Seguridad: Auth (Google OAuth, Email/Password bcrypt, Magic Link). RLS en todas tablas. Keys IA en localStorage (nunca servidor). Validación TypeScript + server. CORS policy. Auditoría ai_events. Cálculos Nutricionales: TMB Mifflin-St Jeor (10×kg + 6.25×cm - 5×edad ± 5/-161). TDEE = TMB × factor (1.2-1.9). Peso base = ESPEN ajuste. Macros: fat_loss 80%/2.0g/kg, muscle_gain 105%/1.8g/kg, recomp 83-90%/2.0g/kg (ciclado), maintain 100%/1.8g/kg. Distribución: proteína → grasa → carbo. Componentes: DashboardShell (layout) | OnboardingFlow (5 pasos) | 16+ Modales | 13+ Vistas. IA: Gemini 1.5 Flash (1500 req/día gratis) | OpenAI | Anthropic | Ollama. Rate limit 15 req/min. Caché 6h. Auditoría ai_events. Búsqueda: 3 capas (local → Open Food Facts → USDA). Caché 30 días. Autocompletado. Mascotas: 15 personajes con 8 estados animables. Integración chat. Triggers eventos. Flujos: Onboarding, Diario comidas, Receta IA, Carrito, Finanzas, Inventario, Feed. APIs: /api/food-search, /api/recipe-ai, /api/chat, /api/sync, /api/auth. Performance: Debounce 1800ms, Caché, Image optimization, Code splitting, Memoization. Testing: Jest (cálculos) | React Testing Library (componentes) | Cypress (E2E). Deployment: Vercel + Supabase Cloud. Monitoreo: Supabase Analytics, Vercel Analytics, Error tracking, Rate limiting. Roadmap: MVP (hoy) | Fase 2 (Nordigen, water_log) | Fase 3 (rutinas, comunidad) | Fase 4+ (premium, marketplace).

Sección 43: Análisis Técnico de FoodOS

FoodOS es una plataforma unificada que integra gestión de inventario, nutrición personalizada y finanzas personales. Desarrollada con Next.js 14 (App Router), React 19 (Server Components), TypeScript 5.3+, Tailwind CSS 3, y Supabase PostgreSQL backend. Arquitectura: Cliente (SPA) → localStorage (JSON) → Supabase PostgreSQL. Debounce 1800ms agrupa cambios. Pull completo al login. Push con upsert+delete. RLS nativo en todas tablas. Conflictos: last-write-wins. Stack: Frontend (Next.js, React, TypeScript, Tailwind, Context+immer) | Backend (Supabase, PostgreSQL 15+, Auth) | APIs (Open Food Facts, Gemini, OpenAI, Anthropic, Nordigen, USDA) | Hosting (Vercel, Supabase Cloud). Base de Datos: 39 tablas PostgreSQL con RLS. Categorías: catálogos (mascots), usuarios (user_profiles), inventario (almacenes, inventory_items), recetas (recipes, recipe_ingredients), nutrición (nutrition_goals, food_log), compras (shopping_lists), finanzas (gastos, ingresos), feed (feed_posts), IA (ai_events, ai_recipe_cache), otros. Seguridad: Auth (Google OAuth, Email/Password bcrypt, Magic Link). RLS en todas tablas. Keys IA en localStorage (nunca servidor). Validación TypeScript + server. CORS policy. Auditoría ai_events. Cálculos Nutricionales: TMB Mifflin-St Jeor (10×kg + 6.25×cm - 5×edad ± 5/-161). TDEE = TMB × factor (1.2-1.9). Peso base = ESPEN ajuste. Macros: fat_loss 80%/2.0g/kg, muscle_gain 105%/1.8g/kg, recomp 83-90%/2.0g/kg (ciclado), maintain 100%/1.8g/kg. Distribución: proteína → grasa → carbo. Componentes: DashboardShell (layout) | OnboardingFlow (5 pasos) | 16+ Modales | 13+ Vistas. IA: Gemini 1.5 Flash (1500 req/día gratis) | OpenAI | Anthropic | Ollama. Rate limit 15 req/min. Caché 6h. Auditoría ai_events. Búsqueda: 3 capas (local → Open Food Facts → USDA). Caché 30 días. Autocompletado. Mascotas: 15 personajes con 8 estados animables. Integración chat. Triggers eventos. Flujos: Onboarding, Diario comidas, Receta IA, Carrito, Finanzas, Inventario, Feed. APIs: /api/food-search, /api/recipe-ai, /api/chat, /api/sync, /api/auth. Performance: Debounce 1800ms, Caché, Image optimization, Code splitting,

Memoization. Testing: Jest (cálculos) | React Testing Library (componentes) | Cypress (E2E). Deployment: Vercel + Supabase Cloud. Monitoreo: Supabase Analytics, Vercel Analytics, Error tracking, Rate limiting. Roadmap: MVP (hoy) | Fase 2 (Nordigen, water_log) | Fase 3 (rutinas, comunidad) | Fase 4+ (premium, marketplace).

Sección 44: Análisis Técnico de FoodOS

FoodOS es una plataforma unificada que integra gestión de inventario, nutrición personalizada y finanzas personales. Desarrollada con Next.js 14 (App Router), React 19 (Server Components), TypeScript 5.3+, Tailwind CSS 3, y Supabase PostgreSQL backend. Arquitectura: Cliente (SPA) → localStorage (JSON) → Supabase PostgreSQL. Debounce 1800ms agrupa cambios. Pull completo al login. Push con upsert+delete. RLS nativo en todas tablas. Conflictos: last-write-wins. Stack: Frontend (Next.js, React, TypeScript, Tailwind, Context+Immer) | Backend (Supabase, PostgreSQL 15+, Auth) | APIs (Open Food Facts, Gemini, OpenAI, Anthropic, Nordigen, USDA) | Hosting (Vercel, Supabase Cloud). Base de Datos: 39 tablas PostgreSQL con RLS. Categorías: catálogos (mascots), usuarios (user_profiles), inventario (almacenes, inventory_items), recetas (recipes, recipe_ingredients), nutrición (nutrition_goals, food_log), compras (shopping_lists), finanzas (gastos, ingresos), feed (feed_posts), IA (ai_events, ai_recipe_cache), otros. Seguridad: Auth (Google OAuth, Email/Password bcrypt, Magic Link). RLS en todas tablas. Keys IA en localStorage (nunca servidor). Validación TypeScript + server. CORS policy. Auditoría ai_events. Cálculos Nutricionales: TMB Mifflin-St Jeor (10×kg + 6.25×cm - 5×edad ± 5/161). TDEE = TMB × factor (1.2-1.9). Peso base = ESPEN ajuste. Macros: fat_loss 80%/2.0g/kg, muscle_gain 105%/1.8g/kg, recomp 83-90%/2.0g/kg (ciclado), maintain 100%/1.8g/kg. Distribución: proteína → grasa → carbos. Componentes: DashboardShell (layout) | OnboardingFlow (5 pasos) | 16+ Modales | 13+ Vistas. IA: Gemini 1.5 Flash (1500 req/día gratis) | OpenAI | Anthropic | Ollama. Rate limit 15 req/min. Caché 6h. Auditoría ai_events. Búsqueda: 3 capas (local → Open Food Facts → USDA). Caché 30 días. Autocompletado. Mascotas: 15 personajes con 8 estados animables. Integración chat. Triggers eventos. Flujos: Onboarding, Diario comidas, Receta IA, Carrito, Finanzas, Inventario, Feed. APIs: /api/food-search, /api/recipe-ai, /api/chat, /api/sync, /api/auth. Performance: Debounce 1800ms, Caché, Image optimization, Code splitting, Memoization. Testing: Jest (cálculos) | React Testing Library (componentes) | Cypress (E2E). Deployment: Vercel + Supabase Cloud. Monitoreo: Supabase Analytics, Vercel Analytics, Error tracking, Rate limiting. Roadmap: MVP (hoy) | Fase 2 (Nordigen, water_log) | Fase 3 (rutinas, comunidad) | Fase 4+ (premium, marketplace).

Sección 45: Análisis Técnico de FoodOS

FoodOS es una plataforma unificada que integra gestión de inventario, nutrición personalizada y finanzas personales. Desarrollada con Next.js 14 (App Router), React 19 (Server Components), TypeScript 5.3+, Tailwind CSS 3, y Supabase PostgreSQL backend. Arquitectura: Cliente (SPA) → localStorage (JSON) → Supabase PostgreSQL. Debounce 1800ms agrupa cambios. Pull completo al login. Push con upsert+delete. RLS nativo en todas tablas. Conflictos: last-write-wins. Stack: Frontend (Next.js, React, TypeScript, Tailwind, Context+Immer) | Backend (Supabase, PostgreSQL 15+, Auth) | APIs (Open Food Facts, Gemini, OpenAI, Anthropic, Nordigen, USDA) | Hosting (Vercel, Supabase Cloud). Base de Datos: 39 tablas PostgreSQL con RLS. Categorías: catálogos (mascots), usuarios (user_profiles), inventario (almacenes, inventory_items), recetas (recipes, recipe_ingredients), nutrición (nutrition_goals, food_log), compras (shopping_lists), finanzas (gastos, ingresos), feed (feed_posts), IA (ai_events, ai_recipe_cache), otros. Seguridad: Auth (Google OAuth, Email/Password bcrypt, Magic Link). RLS en todas tablas. Keys IA en localStorage (nunca servidor). Validación TypeScript + server. CORS policy. Auditoría ai_events. Cálculos Nutricionales: TMB Mifflin-St Jeor (10×kg + 6.25×cm - 5×edad ± 5/161). TDEE = TMB × factor (1.2-1.9). Peso base = ESPEN ajuste. Macros: fat_loss 80%/2.0g/kg, muscle_gain 105%/1.8g/kg, recomp 83-90%/2.0g/kg (ciclado), maintain 100%/1.8g/kg. Distribución: proteína → grasa → carbos. Componentes: DashboardShell (layout) | OnboardingFlow (5 pasos) | 16+ Modales | 13+ Vistas. IA: Gemini 1.5 Flash (1500 req/día gratis) | OpenAI | Anthropic | Ollama. Rate limit 15 req/min. Caché 6h. Auditoría ai_events. Búsqueda: 3 capas (local → Open Food Facts → USDA). Caché 30 días. Autocompletado. Mascotas: 15 personajes con 8 estados animables. Integración chat. Triggers eventos. Flujos: Onboarding, Diario comidas, Receta IA, Carrito, Finanzas, Inventario, Feed. APIs: /api/food-search, /api/recipe-ai, /api/chat, /api/sync, /api/auth. Performance: Debounce 1800ms, Caché, Image optimization, Code splitting, Memoization. Testing: Jest (cálculos) | React Testing Library (componentes) | Cypress (E2E). Deployment: Vercel + Supabase Cloud. Monitoreo: Supabase Analytics, Vercel Analytics, Error tracking, Rate limiting. Roadmap: MVP (hoy) | Fase 2 (Nordigen, water_log) | Fase 3 (rutinas, comunidad) | Fase 4+ (premium, marketplace).

Sección 46: Análisis Técnico de FoodOS

FoodOS es una plataforma unificada que integra gestión de inventario, nutrición personalizada y finanzas personales. Desarrollada con Next.js 14 (App Router), React 19 (Server Components), TypeScript 5.3+, Tailwind CSS 3, y Supabase PostgreSQL backend. Arquitectura: Cliente (SPA) → localStorage (JSON) → Supabase PostgreSQL. Debounce 1800ms agrupa cambios. Pull completo al login. Push con upsert+delete. RLS nativo en todas tablas. Conflictos: last-write-wins. Stack: Frontend (Next.js, React, TypeScript, Tailwind, Context+Immer) | Backend (Supabase, PostgreSQL 15+, Auth) | APIs (Open Food Facts, Gemini, OpenAI, Anthropic, Nordigen, USDA) | Hosting (Vercel, Supabase Cloud). Base de Datos: 39 tablas PostgreSQL con RLS. Categorías: catálogos (mascots), usuarios (user_profiles), inventario (almacenes, inventory_items), recetas (recipes, recipe_ingredients), nutrición (nutrition_goals, food_log), compras (shopping_lists), finanzas (gastos, ingresos), feed (feed_posts), IA (ai_events, ai_recipe_cache), otros. Seguridad: Auth (Google OAuth, Email/Password bcrypt, Magic Link). RLS en todas tablas. Keys IA en localStorage (nunca servidor). Validación TypeScript + server. CORS policy. Auditoría ai_events. Cálculos Nutricionales: TMB Mifflin-St Jeor (10×kg + 6.25×cm - 5×edad ± 5/161). TDEE = TMB × factor (1.2-1.9). Peso base = ESPEN ajuste. Macros: fat_loss 80%/2.0g/kg, muscle_gain 105%/1.8g/kg, recomp 83-90%/2.0g/kg (ciclado), maintain 100%/1.8g/kg. Distribución: proteína → grasa → carbos. Componentes: DashboardShell (layout) | OnboardingFlow (5 pasos) | 16+ Modales | 13+ Vistas. IA: Gemini 1.5 Flash (1500 req/día gratis) | OpenAI | Anthropic | Ollama. Rate limit 15 req/min. Caché 6h. Auditoría ai_events. Búsqueda: 3 capas (local → Open Food Facts → USDA). Caché 30 días. Autocompletado. Mascotas: 15 personajes con 8 estados animables. Integración chat. Triggers eventos. Flujos: Onboarding, Diario comidas, Receta IA, Carrito, Finanzas, Inventario, Feed. APIs: /api/food-search, /api/recipe-ai, /api/chat, /api/sync, /api/auth. Performance: Debounce 1800ms, Caché, Image optimization, Code splitting, Memoization. Testing: Jest (cálculos) | React Testing Library (componentes) | Cypress (E2E). Deployment: Vercel + Supabase Cloud. Monitoreo: Supabase Analytics, Vercel Analytics, Error tracking, Rate limiting. Roadmap: MVP (hoy) | Fase 2 (Nordigen, water_log) | Fase 3 (rutinas, comunidad) | Fase 4+ (premium, marketplace).

Sección 47: Análisis Técnico de FoodOS

FoodOS es una plataforma unificada que integra gestión de inventario, nutrición personalizada y finanzas personales. Desarrollada con Next.js 14 (App Router), React 19 (Server Components), TypeScript 5.3+, Tailwind CSS 3, y Supabase PostgreSQL backend. Arquitectura: Cliente (SPA) → localStorage (JSON) → Supabase PostgreSQL. Debounce 1800ms agrupa cambios. Pull completo al login. Push con upsert+delete. RLS nativo en todas tablas. Conflictos: last-write-wins. Stack: Frontend (Next.js, React, TypeScript, Tailwind, Context+Immer) | Backend (Supabase, PostgreSQL 15+, Auth) | APIs (Open Food Facts, Gemini, OpenAI, Anthropic, Nordigen, USDA) | Hosting (Vercel, Supabase Cloud). Base de Datos: 39 tablas PostgreSQL con RLS. Categorías: catálogos (mascots), usuarios (user_profiles), inventario (almacenes, inventory_items), recetas (recipes, recipe_ingredients), nutrición (nutrition_goals, food_log), compras (shopping_lists), finanzas (gastos, ingresos), feed (feed_posts), IA (ai_events, ai_recipe_cache), otros. Seguridad: Auth (Google OAuth, Email/Password bcrypt, Magic Link). RLS en todas tablas. Keys IA en localStorage (nunca servidor). Validación TypeScript + server. CORS policy. Auditoría ai_events. Cálculos Nutricionales: TMB Mifflin-St Jeor (10×kg + 6.25×cm - 5×edad ± 5/161). TDEE = TMB × factor (1.2-1.9). Peso base = ESPEN ajuste. Macros: fat_loss 80%/2.0g/kg, muscle_gain 105%/1.8g/kg, recomp 83-90%/2.0g/kg (ciclado), maintain 100%/1.8g/kg. Distribución: proteína → grasa → carbos. Componentes: DashboardShell (layout) | OnboardingFlow (5 pasos) | 16+ Modales | 13+ Vistas. IA: Gemini 1.5 Flash (1500 req/día gratis) | OpenAI | Anthropic | Ollama. Rate limit 15 req/min. Caché 6h. Auditoría ai_events. Búsqueda: 3 capas (local → Open Food Facts → USDA). Caché 30 días. Autocompletado. Mascotas: 15 personajes con 8 estados animables. Integración chat. Triggers eventos. Flujos: Onboarding, Diario comidas, Receta IA, Carrito, Finanzas, Inventario, Feed. APIs: /api/food-search, /api/recipe-ai, /api/chat, /api/sync, /api/auth. Performance: Debounce 1800ms, Caché, Image optimization, Code splitting, Memoization. Testing: Jest (cálculos) | React Testing Library (componentes) | Cypress (E2E). Deployment: Vercel + Supabase Cloud. Monitoreo: Supabase Analytics, Vercel Analytics, Error tracking, Rate limiting. Roadmap: MVP (hoy) | Fase 2 (Nordigen, water_log) | Fase 3 (rutinas, comunidad) | Fase 4+ (premium, marketplace).

Sección 48: Análisis Técnico de FoodOS

FoodOS es una plataforma unificada que integra gestión de inventario, nutrición personalizada y finanzas personales. Desarrollada con Next.js 14 (App Router), React 19 (Server Components), TypeScript 5.3+, Tailwind CSS 3, y Supabase PostgreSQL backend. Arquitectura: Cliente (SPA) → localStorage (JSON) → Supabase PostgreSQL. Debounce 1800ms agrupa cambios. Pull completo al login. Push con upsert+delete. RLS nativo en todas tablas. Conflictos: last-write-wins. Stack: Frontend (Next.js, React, TypeScript, Tailwind, Context+Immer) | Backend (Supabase, PostgreSQL 15+, Auth) | APIs (Open Food Facts, Gemini, OpenAI, Anthropic, Nordigen, USDA) | Hosting (Vercel, Supabase Cloud). Base de Datos: 39 tablas PostgreSQL con RLS. Categorías: catálogos (mascots), usuarios (user_profiles), inventario (almacenes, inventory_items), recetas (recipes, recipe_ingredients), nutrición (nutrition_goals, food_log), compras (shopping_lists), finanzas (gastos, ingresos), feed (feed_posts), IA (ai_events, ai_recipe_cache), otros. Seguridad: Auth (Google OAuth, Email/Password bcrypt, Magic Link). RLS en todas tablas. Keys IA en localStorage (nunca servidor). Validación TypeScript + server. CORS policy. Auditoría ai_events. Cálculos Nutricionales: TMB Mifflin-St Jeor (10×kg + 6.25×cm - 5×edad ± 5/161). TDEE = TMB × factor (1.2-1.9). Peso base = ESPEN ajuste. Macros: fat_loss 80%/2.0g/kg, muscle_gain 105%/1.8g/kg, recomp 83-90%/2.0g/kg (ciclado), maintain 100%/1.8g/kg. Distribución: proteína → grasa → carbos. Componentes: DashboardShell (layout) | OnboardingFlow (5 pasos) | 16+ Modales | 13+ Vistas. IA: Gemini 1.5 Flash (1500 req/día gratis) | OpenAI | Anthropic | Ollama. Rate limit 15 req/min. Caché 6h. Auditoría ai_events. Búsqueda: 3 capas (local → Open Food Facts → USDA). Caché 30 días. Autocompletado. Mascotas: 15 personajes con 8 estados animables. Integración chat. Triggers eventos. Flujos: Onboarding, Diario comidas, Receta IA, Carrito, Finanzas, Inventario, Feed. APIs: /api/food-search, /api/recipe-ai, /api/chat, /api/sync, /api/auth. Performance: Debounce 1800ms, Caché, Image optimization, Code splitting, Memoization. Testing: Jest (cálculos) | React Testing Library (componentes) | Cypress (E2E). Deployment: Vercel + Supabase Cloud. Monitoreo: Supabase Analytics, Vercel Analytics, Error tracking, Rate limiting. Roadmap: MVP (hoy) | Fase 2 (Nordigen, water_log) | Fase 3 (rutinas, comunidad) | Fase 4+ (premium, marketplace).

Sección 49: Análisis Técnico de FoodOS

FoodOS es una plataforma unificada que integra gestión de inventario, nutrición personalizada y finanzas personales. Desarrollada con Next.js 14 (App Router), React 19 (Server Components), TypeScript 5.3+, Tailwind CSS 3, y Supabase PostgreSQL backend. Arquitectura: Cliente (SPA) → localStorage (JSON) → Supabase PostgreSQL. Debounce 1800ms agrupa cambios. Pull completo al login. Push con upsert+delete. RLS nativo en todas tablas. Conflictos: last-write-wins. Stack: Frontend (Next.js, React, TypeScript, Tailwind, Context+Immer) | Backend (Supabase, PostgreSQL 15+, Auth) | APIs (Open Food Facts, Gemini, OpenAI, Anthropic, Nordigen, USDA) | Hosting (Vercel, Supabase Cloud). Base de Datos: 39 tablas PostgreSQL con RLS. Categorías: catálogos (mascots), usuarios (user_profiles), inventario (almacenes, inventory_items), recetas (recipes, recipe_ingredients), nutrición (nutrition_goals, food_log), compras (shopping_lists), finanzas (gastos, ingresos), feed (feed_posts), IA (ai_events, ai_recipe_cache), otros. Seguridad: Auth (Google OAuth, Email/Password bcrypt, Magic Link). RLS en todas tablas. Keys IA en localStorage (nunca servidor). Validación TypeScript + server. CORS policy. Auditoría ai_events. Cálculos Nutricionales: TMB Mifflin-St Jeor (10×kg + 6.25×cm - 5×edad ± 5/161). TDEE = TMB × factor (1.2-1.9). Peso base = ESPEN ajuste. Macros: fat_loss 80%/2.0g/kg, muscle_gain 105%/1.8g/kg, recomp 83-90%/2.0g/kg (ciclado), maintain 100%/1.8g/kg. Distribución: proteína → grasa → carbos. Componentes: DashboardShell (layout) | OnboardingFlow (5 pasos) | 16+ Modales | 13+ Vistas. IA: Gemini 1.5 Flash (1500 req/día gratis) | OpenAI | Anthropic | Ollama. Rate limit 15 req/min. Caché 6h. Auditoría ai_events. Búsqueda: 3 capas (local → Open Food Facts → USDA). Caché 30 días. Autocompletado. Mascotas: 15 personajes con 8 estados animables. Integración chat. Triggers eventos. Flujos: Onboarding, Diario comidas, Receta IA, Carrito, Finanzas, Inventario, Feed. APIs: /api/food-search, /api/recipe-ai, /api/chat, /api/sync, /api/auth. Performance: Debounce 1800ms, Caché, Image optimization, Code splitting, Memoization. Testing: Jest (cálculos) | React Testing Library (componentes) | Cypress (E2E). Deployment: Vercel + Supabase Cloud. Monitoreo: Supabase Analytics, Vercel Analytics, Error tracking, Rate limiting. Roadmap: MVP (hoy) | Fase 2 (Nordigen, water_log) | Fase 3 (rutinas, comunidad) | Fase 4+ (premium, marketplace).

Sección 50: Análisis Técnico de FoodOS

FoodOS es una plataforma unificada que integra gestión de inventario, nutrición personalizada y finanzas personales. Desarrollada con Next.js 14 (App Router), React 19 (Server Components), TypeScript 5.3+, Tailwind CSS 3, y Supabase PostgreSQL backend. Arquitectura: Cliente (SPA) → localStorage (JSON) → Supabase PostgreSQL. Debounce 1800ms agrupa cambios. Pull completo al login. Push con upsert+delete. RLS nativo en todas tablas. Conflictos: last-write-wins. Stack: Frontend (Next.js, React, TypeScript, Tailwind, Context+Immer) | Backend (Supabase, PostgreSQL 15+, Auth) | APIs (Open Food Facts, Gemini, OpenAI, Anthropic, Nordigen, USDA) | Hosting (Vercel, Supabase Cloud). Base de Datos: 39 tablas PostgreSQL con RLS. Categorías: catálogos (mascots), usuarios (user_profiles), inventario (almacenes, inventory_items), recetas (recipes, recipe_ingredients), nutrición (nutrition_goals, food_log), compras (shopping_lists), finanzas (gastos, ingresos), feed (feed_posts), IA (ai_events, ai_recipe_cache), otros. Seguridad: Auth (Google OAuth, Email/Password bcrypt, Magic Link). RLS en todas tablas. Keys IA en localStorage (nunca servidor). Validación TypeScript + server. CORS policy. Auditoría ai_events. Cálculos Nutricionales: TMB Mifflin-St Jeor (10×kg + 6.25×cm - 5×edad ± 5/-161). TDEE = TMB × factor (1.2-1.9). Peso base = ESPEN ajuste. Macros: fat_loss 80%/2.0g/kg, muscle_gain 105%/1.8g/kg, recomp 83-90%/2.0g/kg (ciclado), maintain 100%/1.8g/kg. Distribución: proteína → grasa → carbos. Componentes: DashboardShell (layout) | OnboardingFlow (5 pasos) | 16+ Modales | 13+ Vistas. IA: Gemini 1.5 Flash (1500 req/día gratis) | OpenAI | Anthropic | Ollama. Rate limit 15 req/min. Caché 6h. Auditoría ai_events. Búsqueda: 3 capas (local → Open Food Facts → USDA). Caché 30 días. Autocompletado. Mascotas: 15 personajes con 8 estados animables. Integración chat. Triggers eventos. Flujos: Onboarding, Diario comidas, Receta IA, Carrito, Finanzas, Inventario, Feed. APIs: /api/food-search, /api/recipe-ai, /api/chat, /api/sync, /api/auth. Performance: Debounce 1800ms, Caché, Image optimization, Code splitting, Memoization. Testing: Jest (cálculos) | React Testing Library (componentes) | Cypress (E2E). Deployment: Vercel + Supabase Cloud. Monitoreo: Supabase Analytics, Vercel Analytics, Error tracking, Rate limiting. Roadmap: MVP (hoy) | Fase 2 (Nordigen, water_log) | Fase 3 (rutinas, comunidad) | Fase 4+ (premium, marketplace).

Sección 51: Análisis Técnico de FoodOS

FoodOS es una plataforma unificada que integra gestión de inventario, nutrición personalizada y finanzas personales. Desarrollada con Next.js 14 (App Router), React 19 (Server Components), TypeScript 5.3+, Tailwind CSS 3, y Supabase PostgreSQL backend. Arquitectura: Cliente (SPA) → localStorage (JSON) → Supabase PostgreSQL. Debounce 1800ms agrupa cambios. Pull completo al login. Push con upsert+delete. RLS nativo en todas tablas. Conflictos: last-write-wins. Stack: Frontend (Next.js, React, TypeScript, Tailwind, Context+Immer) | Backend (Supabase, PostgreSQL 15+, Auth) | APIs (Open Food Facts, Gemini, OpenAI, Anthropic, Nordigen, USDA) | Hosting (Vercel, Supabase Cloud). Base de Datos: 39 tablas PostgreSQL con RLS. Categorías: catálogos (mascots), usuarios (user_profiles), inventario (almacenes, inventory_items), recetas (recipes, recipe_ingredients), nutrición (nutrition_goals, food_log), compras (shopping_lists), finanzas (gastos, ingresos), feed (feed_posts), IA (ai_events, ai_recipe_cache), otros. Seguridad: Auth (Google OAuth, Email/Password bcrypt, Magic Link). RLS en todas tablas. Keys IA en localStorage (nunca servidor). Validación TypeScript + server. CORS policy. Auditoría ai_events. Cálculos Nutricionales: TMB Mifflin-St Jeor (10×kg + 6.25×cm - 5×edad ± 5/-161). TDEE = TMB × factor (1.2-1.9). Peso base = ESPEN ajuste. Macros: fat_loss 80%/2.0g/kg, muscle_gain 105%/1.8g/kg, recomp 83-90%/2.0g/kg (ciclado), maintain 100%/1.8g/kg. Distribución: proteína → grasa → carbos. Componentes: DashboardShell (layout) | OnboardingFlow (5 pasos) | 16+ Modales | 13+ Vistas. IA: Gemini 1.5 Flash (1500 req/día gratis) | OpenAI | Anthropic | Ollama. Rate limit 15 req/min. Caché 6h. Auditoría ai_events. Búsqueda: 3 capas (local → Open Food Facts → USDA). Caché 30 días. Autocompletado. Mascotas: 15 personajes con 8 estados animables. Integración chat. Triggers eventos. Flujos: Onboarding, Diario comidas, Receta IA, Carrito, Finanzas, Inventario, Feed. APIs: /api/food-search, /api/recipe-ai, /api/chat, /api/sync, /api/auth. Performance: Debounce 1800ms, Caché, Image optimization, Code splitting, Memoization. Testing: Jest (cálculos) | React Testing Library (componentes) | Cypress (E2E). Deployment: Vercel + Supabase Cloud. Monitoreo: Supabase Analytics, Vercel Analytics, Error tracking, Rate limiting. Roadmap: MVP (hoy) | Fase 2 (Nordigen, water_log) | Fase 3 (rutinas, comunidad) | Fase 4+ (premium, marketplace).

Sección 52: Análisis Técnico de FoodOS

FoodOS es una plataforma unificada que integra gestión de inventario, nutrición personalizada y finanzas personales. Desarrollada con Next.js 14 (App Router), React 19 (Server Components), TypeScript 5.3+, Tailwind CSS 3, y Supabase PostgreSQL backend. Arquitectura: Cliente (SPA) → localStorage (JSON) → Supabase PostgreSQL. Debounce 1800ms agrupa cambios. Pull completo al login. Push con upsert+delete. RLS nativo en todas tablas. Conflictos: last-write-wins. Stack: Frontend (Next.js, React, TypeScript, Tailwind, Context+Immer) | Backend (Supabase, PostgreSQL 15+, Auth) | APIs (Open Food Facts, Gemini, OpenAI, Anthropic, Nordigen, USDA) | Hosting (Vercel, Supabase Cloud). Base de Datos: 39 tablas PostgreSQL con RLS. Categorías: catálogos (mascots), usuarios (user_profiles), inventario (almacenes, inventory_items), recetas (recipes, recipe_ingredients), nutrición (nutrition_goals, food_log), compras (shopping_lists), finanzas (gastos, ingresos), feed (feed_posts), IA (ai_events, ai_recipe_cache), otros. Seguridad: Auth (Google OAuth, Email/Password bcrypt, Magic Link). RLS en todas tablas. Keys IA en localStorage (nunca servidor). Validación TypeScript + server. CORS policy. Auditoría ai_events. Cálculos Nutricionales: TMB Mifflin-St Jeor (10×kg + 6.25×cm - 5×edad ± 5/-161). TDEE = TMB × factor (1.2-1.9). Peso base = ESPEN ajuste. Macros: fat_loss 80%/2.0g/kg, muscle_gain 105%/1.8g/kg, recomp 83-90%/2.0g/kg (ciclado), maintain 100%/1.8g/kg. Distribución: proteína → grasa → carbos. Componentes: DashboardShell (layout) | OnboardingFlow (5 pasos) | 16+ Modales | 13+ Vistas. IA: Gemini 1.5 Flash (1500 req/día gratis) | OpenAI | Anthropic | Ollama. Rate limit 15 req/min. Caché 6h. Auditoría ai_events. Búsqueda: 3 capas (local → Open Food Facts → USDA). Caché 30 días. Autocompletado. Mascotas: 15 personajes con 8 estados animables. Integración chat. Triggers eventos. Flujos: Onboarding, Diario comidas, Receta IA, Carrito, Finanzas, Inventario, Feed. APIs: /api/food-search, /api/recipe-ai, /api/chat, /api/sync, /api/auth. Performance: Debounce 1800ms, Caché, Image optimization, Code splitting, Memoization. Testing: Jest (cálculos) | React Testing Library (componentes) | Cypress (E2E). Deployment: Vercel + Supabase Cloud. Monitoreo: Supabase Analytics, Vercel Analytics, Error tracking, Rate limiting. Roadmap: MVP (hoy) | Fase 2 (Nordigen, water_log) | Fase 3 (rutinas, comunidad) | Fase 4+ (premium, marketplace).

Sección 53: Análisis Técnico de FoodOS

FoodOS es una plataforma unificada que integra gestión de inventario, nutrición personalizada y finanzas personales. Desarrollada con Next.js 14 (App Router), React 19 (Server Components), TypeScript 5.3+, Tailwind CSS 3, y Supabase PostgreSQL backend. Arquitectura: Cliente (SPA) → localStorage (JSON) → Supabase PostgreSQL. Debounce 1800ms agrupa cambios. Pull completo al login. Push con upsert+delete. RLS nativo en todas tablas. Conflictos: last-write-wins. Stack: Frontend (Next.js, React, TypeScript, Tailwind, Context+Immer) | Backend (Supabase, PostgreSQL 15+, Auth) | APIs (Open Food Facts, Gemini, OpenAI, Anthropic, Nordigen, USDA) | Hosting (Vercel, Supabase Cloud). Base de Datos: 39 tablas PostgreSQL con RLS. Categorías: catálogos (mascots), usuarios (user_profiles), inventario (almacenes, inventory_items), recetas (recipes, recipe_ingredients), nutrición (nutrition_goals, food_log), compras (shopping_lists), finanzas (gastos, ingresos), feed (feed_posts), IA (ai_events, ai_recipe_cache), otros. Seguridad: Auth (Google OAuth, Email/Password bcrypt, Magic Link). RLS en todas tablas. Keys IA en localStorage (nunca servidor). Validación TypeScript + server. CORS policy. Auditoría ai_events. Cálculos Nutricionales: TMB Mifflin-St Jeor (10×kg + 6.25×cm - 5×edad ± 5/-161). TDEE = TMB × factor (1.2-1.9). Peso base = ESPEN ajuste. Macros: fat_loss 80%/2.0g/kg, muscle_gain 105%/1.8g/kg, recomp 83-90%/2.0g/kg (ciclado), maintain 100%/1.8g/kg. Distribución: proteína → grasa → carbos. Componentes: DashboardShell (layout) | OnboardingFlow (5 pasos) | 16+ Modales | 13+ Vistas. IA: Gemini 1.5 Flash (1500 req/día gratis) | OpenAI | Anthropic | Ollama. Rate limit 15 req/min. Caché 6h. Auditoría ai_events. Búsqueda: 3 capas (local → Open Food Facts → USDA). Caché 30 días. Autocompletado. Mascotas: 15 personajes con 8 estados animables. Integración chat. Triggers eventos. Flujos: Onboarding, Diario comidas, Receta IA, Carrito, Finanzas, Inventario, Feed. APIs: /api/food-search, /api/recipe-ai, /api/chat, /api/sync, /api/auth. Performance: Debounce 1800ms, Caché, Image optimization, Code splitting, Memoization. Testing: Jest (cálculos) | React Testing Library (componentes) | Cypress (E2E). Deployment: Vercel + Supabase Cloud. Monitoreo: Supabase Analytics, Vercel Analytics, Error tracking, Rate limiting. Roadmap: MVP (hoy) | Fase 2 (Nordigen, water_log) | Fase 3 (rutinas, comunidad) | Fase 4+ (premium, marketplace).

Sección 54: Análisis Técnico de FoodOS

FoodOS es una plataforma unificada que integra gestión de inventario, nutrición personalizada y finanzas personales. Desarrollada con Next.js 14 (App Router), React 19 (Server Components), TypeScript 5.3+, Tailwind CSS 3, y Supabase PostgreSQL backend. Arquitectura: Cliente (SPA) → localStorage (JSON) → Supabase PostgreSQL. Debounce 1800ms agrupa cambios. Pull completo al login. Push con upsert+delete. RLS nativo en todas tablas. Conflictos: last-write-wins. Stack: Frontend (Next.js, React, TypeScript, Tailwind, Context+Immer) | Backend (Supabase, PostgreSQL 15+, Auth) | APIs (Open Food Facts, Gemini, OpenAI, Anthropic, Nordigen, USDA) | Hosting (Vercel, Supabase Cloud). Base de Datos: 39 tablas PostgreSQL con RLS. Categorías: catálogos (mascots), usuarios (user_profiles), inventario (almacenes, inventory_items), recetas (recipes, recipe_ingredients), nutrición (nutrition_goals, food_log), compras (shopping_lists), finanzas (gastos, ingresos), feed (feed_posts), IA (ai_events, ai_recipe_cache), otros. Seguridad: Auth (Google OAuth, Email/Password bcrypt, Magic Link). RLS en todas tablas. Keys IA en localStorage (nunca servidor). Validación TypeScript + server. CORS policy. Auditoría ai_events. Cálculos Nutricionales: TMB Mifflin-St Jeor (10×kg + 6.25×cm - 5×edad ± 5/-161). TDEE = TMB × factor (1.2-1.9). Peso base = ESPEN ajuste. Macros: fat_loss 80%/2.0g/kg, muscle_gain 105%/1.8g/kg, recomp 83-90%/2.0g/kg (ciclado), maintain 100%/1.8g/kg. Distribución: proteína → grasa → carbos. Componentes: DashboardShell (layout) | OnboardingFlow (5 pasos) | 16+ Modales | 13+ Vistas. IA: Gemini 1.5 Flash (1500 req/día gratis) | OpenAI | Anthropic | Ollama. Rate limit 15 req/min. Caché 6h. Auditoría ai_events. Búsqueda: 3 capas (local → Open Food Facts → USDA). Caché 30 días. Autocompletado. Mascotas: 15 personajes con 8 estados animables. Integración chat. Triggers eventos. Flujos: Onboarding, Diario comidas, Receta IA, Carrito, Finanzas, Inventario, Feed. APIs: /api/food-search, /api/recipe-ai, /api/chat, /api/sync, /api/auth. Performance: Debounce 1800ms, Caché, Image optimization, Code splitting, Memoization. Testing: Jest (cálculos) | React Testing Library (componentes) | Cypress (E2E). Deployment: Vercel + Supabase Cloud. Monitoreo: Supabase Analytics, Vercel Analytics, Error tracking, Rate limiting. Roadmap: MVP (hoy) | Fase 2 (Nordigen, water_log) | Fase 3 (rutinas, comunidad) | Fase 4+ (premium, marketplace).

Sección 55: Análisis Técnico de FoodOS

FoodOS es una plataforma unificada que integra gestión de inventario, nutrición personalizada y finanzas personales. Desarrollada con Next.js 14 (App Router), React 19 (Server Components), TypeScript 5.3+, Tailwind CSS 3, y Supabase PostgreSQL backend. Arquitectura: Cliente (SPA) → localStorage (JSON) → Supabase PostgreSQL. Debounce 1800ms agrupa cambios. Pull completo al login. Push con upsert+delete. RLS nativo en todas tablas. Conflictos: last-write-wins. Stack: Frontend (Next.js, React, TypeScript, Tailwind, Context+Immer) | Backend (Supabase, PostgreSQL 15+, Auth) | APIs (Open Food Facts, Gemini, OpenAI, Anthropic, Nordigen, USDA) | Hosting (Vercel, Supabase Cloud). Base de Datos: 39 tablas PostgreSQL con RLS. Categorías: catálogos (mascots), usuarios (user_profiles), inventario (almacenes, inventory_items), recetas (recipes, recipe_ingredients), nutrición (nutrition_goals, food_log), compras (shopping_lists), finanzas (gastos, ingresos), feed (feed_posts), IA (ai_events, ai_recipe_cache), otros. Seguridad: Auth (Google OAuth, Email/Password bcrypt, Magic Link). RLS en todas tablas. Keys IA en localStorage (nunca servidor). Validación TypeScript + server. CORS policy. Auditoría ai_events. Cálculos Nutricionales: TMB Mifflin-St Jeor (10×kg + 6.25×cm - 5×edad ± 5/-161). TDEE = TMB × factor (1.2-1.9). Peso base = ESPEN ajuste. Macros: fat_loss 80%/2.0g/kg, muscle_gain 105%/1.8g/kg, recomp 83-90%/2.0g/kg (ciclado), maintain 100%/1.8g/kg. Distribución: proteína → grasa → carbos. Componentes: DashboardShell (layout) | OnboardingFlow (5 pasos) | 16+ Modales | 13+ Vistas. IA: Gemini 1.5 Flash (1500 req/día gratis) | OpenAI | Anthropic | Ollama. Rate limit 15 req/min. Caché 6h. Auditoría ai_events. Búsqueda: 3 capas (local → Open Food Facts → USDA). Caché 30 días. Autocompletado. Mascotas: 15 personajes con 8 estados animables. Integración chat. Triggers eventos. Flujos: Onboarding, Diario comidas, Receta IA, Carrito, Finanzas, Inventario, Feed. APIs: /api/food-search, /api/recipe-ai, /api/chat, /api/sync, /api/auth. Performance: Debounce 1800ms, Caché, Image optimization, Code splitting, Memoization. Testing: Jest (cálculos) | React Testing Library (componentes) | Cypress (E2E). Deployment: Vercel + Supabase Cloud. Monitoreo: Supabase Analytics, Vercel Analytics, Error tracking, Rate limiting. Roadmap: MVP (hoy) | Fase 2 (Nordigen, water_log) | Fase 3 (rutinas, comunidad) | Fase 4+ (premium, marketplace).

Sección 56: Análisis Técnico de FoodOS

FoodOS es una plataforma unificada que integra gestión de inventario, nutrición personalizada y finanzas personales. Desarrollada con Next.js 14 (App Router), React 19 (Server Components), TypeScript 5.3+, Tailwind CSS 3, y Supabase PostgreSQL backend. Arquitectura: Cliente (SPA) → localStorage (JSON) → Supabase PostgreSQL. Debounce 1800ms agrupa cambios. Pull completo al login. Push con upsert+delete. RLS nativo en todas tablas. Conflictos: last-write-wins. Stack: Frontend (Next.js, React, TypeScript, Tailwind, Context+Immer) | Backend (Supabase, PostgreSQL 15+, Auth) | APIs (Open Food Facts, Gemini, OpenAI, Anthropic, Nordigen, USDA) | Hosting (Vercel, Supabase Cloud). Base de Datos: 39 tablas PostgreSQL con RLS. Categorías: catálogos (mascots), usuarios (user_profiles), inventario (almacenes, inventory_items), recetas (recipes, recipe_ingredients), nutrición (nutrition_goals, food_log), compras (shopping_lists), finanzas (gastos, ingresos), feed (feed_posts), IA (ai_events, ai_recipe_cache), otros. Seguridad: Auth (Google OAuth, Email/Password bcrypt, Magic Link). RLS en todas tablas. Keys IA en localStorage (nunca servidor). Validación TypeScript + server. CORS policy. Auditoría ai_events. Cálculos Nutricionales: TMB Mifflin-St Jeor (10×kg + 6.25×cm - 5×edad ± 5/-161). TDEE = TMB × factor (1.2-1.9). Peso base = ESPEN ajuste. Macros: fat_loss 80%/2.0g/kg, muscle_gain 105%/1.8g/kg, recomp 83-90%/2.0g/kg (ciclado), maintain 100%/1.8g/kg. Distribución: proteína → grasa → carbos. Componentes: DashboardShell (layout) | OnboardingFlow (5 pasos) | 16+ Modales | 13+ Vistas. IA: Gemini 1.5 Flash (1500 req/día gratis) | OpenAI | Anthropic | Ollama. Rate limit 15 req/min. Caché 6h. Auditoría ai_events. Búsqueda: 3 capas (local → Open Food Facts → USDA). Caché 30 días. Autocompletado. Mascotas: 15 personajes con 8 estados animables. Integración chat. Triggers eventos. Flujos: Onboarding, Diario comidas, Receta IA, Carrito, Finanzas, Inventario, Feed. APIs: /api/food-search, /api/recipe-ai, /api/chat, /api/sync, /api/auth. Performance: Debounce 1800ms, Caché, Image optimization, Code splitting, Memoization. Testing: Jest (cálculos) | React Testing Library (componentes) | Cypress (E2E). Deployment: Vercel + Supabase Cloud. Monitoreo: Supabase Analytics, Vercel Analytics, Error tracking, Rate limiting. Roadmap: MVP (hoy) | Fase 2 (Nordigen, water_log) | Fase 3 (rutinas, comunidad) | Fase 4+ (premium, marketplace).

Sección 57: Análisis Técnico de FoodOS

FoodOS es una plataforma unificada que integra gestión de inventario, nutrición personalizada y finanzas personales. Desarrollada con Next.js 14 (App Router), React 19 (Server Components), TypeScript 5.3+, Tailwind CSS 3, y Supabase PostgreSQL backend. Arquitectura: Cliente (SPA) → localStorage (JSON) → Supabase PostgreSQL. Debounce 1800ms agrupa cambios. Pull completo al login. Push con upsert+delete. RLS nativo en todas tablas. Conflictos: last-write-wins. Stack: Frontend (Next.js, React, TypeScript, Tailwind, Context+Immer) | Backend (Supabase, PostgreSQL 15+, Auth) | APIs (Open Food Facts, Gemini, OpenAI, Anthropic, Nordigen, USDA) | Hosting (Vercel, Supabase Cloud). Base de Datos: 39 tablas PostgreSQL con RLS. Categorías: catálogos (mascots), usuarios (user_profiles), inventario (almacenes, inventory_items), recetas (recipes, recipe_ingredients), nutrición (nutrition_goals, food_log), compras (shopping_lists), finanzas (gastos, ingresos), feed (feed_posts), IA (ai_events, ai_recipe_cache), otros. Seguridad: Auth (Google OAuth, Email/Password bcrypt, Magic Link). RLS en todas tablas. Keys IA en localStorage (nunca servidor). Validación TypeScript + server. CORS policy. Auditoría ai_events. Cálculos Nutricionales: TMB Mifflin-St Jeor (10×kg + 6.25×cm - 5×edad ± 5/-161). TDEE = TMB × factor (1.2-1.9). Peso base = ESPEN ajuste. Macros: fat_loss 80%/2.0g/kg, muscle_gain 105%/1.8g/kg, recomp 83-90%/2.0g/kg (ciclado), maintain 100%/1.8g/kg. Distribución: proteína → grasa → carbos. Componentes: DashboardShell (layout) | OnboardingFlow (5 pasos) | 16+ Modales | 13+ Vistas. IA: Gemini 1.5 Flash (1500 req/día gratis) | OpenAI | Anthropic | Ollama. Rate limit 15 req/min. Caché 6h. Auditoría ai_events. Búsqueda: 3 capas (local → Open Food Facts → USDA). Caché 30 días. Autocompletado. Mascotas: 15 personajes con 8 estados animables. Integración chat. Triggers eventos. Flujos: Onboarding, Diario comidas, Receta IA, Carrito, Finanzas, Inventario, Feed. APIs: /api/food-search, /api/recipe-ai, /api/chat, /api/sync, /api/auth. Performance: Debounce 1800ms, Caché, Image optimization, Code splitting, Memoization. Testing: Jest (cálculos) | React Testing Library (componentes) | Cypress (E2E). Deployment: Vercel + Supabase Cloud. Monitoreo: Supabase Analytics, Vercel Analytics, Error tracking, Rate limiting. Roadmap: MVP (hoy) | Fase 2 (Nordigen, water_log) | Fase 3 (rutinas, comunidad) | Fase 4+ (premium, marketplace).

Sección 58: Análisis Técnico de FoodOS

FoodOS es una plataforma unificada que integra gestión de inventario, nutrición personalizada y finanzas personales. Desarrollada con Next.js 14 (App Router), React 19 (Server Components), TypeScript 5.3+, Tailwind CSS 3, y Supabase PostgreSQL backend. Arquitectura: Cliente (SPA) → localStorage (JSON) → Supabase PostgreSQL. Debounce 1800ms agrupa cambios. Pull completo al login. Push con upsert+delete. RLS nativo en todas tablas. Conflictos: last-write-wins. Stack: Frontend (Next.js, React, TypeScript, Tailwind, Context+Immer) | Backend (Supabase, PostgreSQL 15+, Auth) | APIs (Open Food Facts, Gemini, OpenAI, Anthropic, Nordigen, USDA) | Hosting (Vercel, Supabase Cloud). Base de Datos: 39 tablas PostgreSQL con RLS. Categorías: catálogos (mascots), usuarios (user_profiles), inventario (almacenes, inventory_items), recetas (recipes, recipe_ingredients), nutrición (nutrition_goals, food_log), compras (shopping_lists), finanzas (gastos, ingresos), feed (feed_posts), IA (ai_events, ai_recipe_cache), otros. Seguridad: Auth (Google OAuth, Email/Password bcrypt, Magic Link). RLS en todas tablas. Keys IA en localStorage (nunca servidor). Validación TypeScript + server. CORS policy. Auditoría ai_events. Cálculos Nutricionales: TMB Mifflin-St Jeor (10×kg + 6.25×cm - 5×edad ± 5/-161). TDEE = TMB × factor (1.2-1.9). Peso base = ESPEN ajuste. Macros: fat_loss 80%/2.0g/kg, muscle_gain 105%/1.8g/kg, recomp 83-90%/2.0g/kg (ciclado), maintain 100%/1.8g/kg. Distribución: proteína → grasa → carbos. Componentes: DashboardShell (layout) | OnboardingFlow (5 pasos) | 16+ Modales | 13+ Vistas. IA: Gemini 1.5 Flash (1500 req/día gratis) | OpenAI | Anthropic | Ollama. Rate limit 15 req/min. Caché 6h. Auditoría ai_events. Búsqueda: 3 capas (local → Open Food Facts → USDA). Caché 30 días. Autocompletado. Mascotas: 15 personajes con 8 estados animables. Integración chat. Triggers eventos. Flujos: Onboarding, Diario comidas, Receta IA, Carrito, Finanzas, Inventario, Feed. APIs: /api/food-search, /api/recipe-ai, /api/chat, /api/sync, /api/auth. Performance: Debounce 1800ms, Caché, Image optimization, Code splitting, Memoization. Testing: Jest (cálculos) | React Testing Library (componentes) | Cypress (E2E). Deployment: Vercel + Supabase Cloud. Monitoreo: Supabase Analytics, Vercel Analytics, Error tracking, Rate limiting. Roadmap: MVP (hoy) | Fase 2 (Nordigen, water_log) | Fase 3 (rutinas, comunidad) | Fase 4+ (premium, marketplace).

Sección 59: Análisis Técnico de FoodOS

FoodOS es una plataforma unificada que integra gestión de inventario, nutrición personalizada y finanzas personales. Desarrollada con Next.js 14 (App Router), React 19 (Server Components), TypeScript 5.3+, Tailwind CSS 3, y Supabase PostgreSQL backend. Arquitectura: Cliente (SPA) → localStorage (JSON) → Supabase PostgreSQL. Debounce 1800ms agrupa cambios. Pull completo al login. Push con upsert+delete. RLS nativo en todas tablas. Conflictos: last-write-wins. Stack: Frontend (Next.js, React, TypeScript, Tailwind, Context+Immer) | Backend (Supabase, PostgreSQL 15+, Auth) | APIs (Open Food Facts, Gemini, OpenAI, Anthropic, Nordigen, USDA) | Hosting (Vercel, Supabase Cloud). Base de Datos: 39 tablas PostgreSQL con RLS. Categorías: catálogos (mascots), usuarios (user_profiles), inventario (almacenes, inventory_items), recetas (recipes, recipe_ingredients), nutrición (nutrition_goals, food_log), compras (shopping_lists), finanzas (gastos, ingresos), feed (feed_posts), IA (ai_events, ai_recipe_cache), otros. Seguridad: Auth (Google OAuth, Email/Password bcrypt, Magic Link). RLS en todas tablas. Keys IA en localStorage (nunca servidor). Validación TypeScript + server. CORS policy. Auditoría ai_events. Cálculos Nutricionales: TMB Mifflin-St Jeor (10×kg + 6.25×cm - 5×edad ± 5/-161). TDEE = TMB × factor (1.2-1.9). Peso base = ESPEN ajuste. Macros: fat_loss 80%/2.0g/kg, muscle_gain 105%/1.8g/kg, recomp 83-90%/2.0g/kg (ciclado), maintain 100%/1.8g/kg. Distribución: proteína → grasa → carbos. Componentes: DashboardShell (layout) | OnboardingFlow (5 pasos) | 16+ Modales | 13+ Vistas. IA: Gemini 1.5 Flash (1500 req/día gratis) | OpenAI | Anthropic | Ollama. Rate limit 15 req/min. Caché 6h. Auditoría ai_events. Búsqueda: 3 capas (local → Open Food Facts → USDA). Caché 30 días. Autocompletado. Mascotas: 15 personajes con 8 estados animables. Integración chat. Triggers eventos. Flujos: Onboarding, Diario comidas, Receta IA, Carrito, Finanzas, Inventario, Feed. APIs: /api/food-search, /api/recipe-ai, /api/chat, /api/sync, /api/auth. Performance: Debounce 1800ms, Caché, Image optimization, Code splitting, Memoization. Testing: Jest (cálculos) | React Testing Library (componentes) | Cypress (E2E). Deployment: Vercel + Supabase Cloud. Monitoreo: Supabase Analytics, Vercel Analytics, Error tracking, Rate limiting. Roadmap: MVP (hoy) | Fase 2 (Nordigen, water_log) | Fase 3 (rutinas, comunidad) | Fase 4+ (premium, marketplace).

Sección 60: Análisis Técnico de FoodOS

FoodOS es una plataforma unificada que integra gestión de inventario, nutrición personalizada y finanzas personales. Desarrollada con Next.js 14 (App Router), React 19 (Server Components), TypeScript 5.3+, Tailwind CSS 3, y Supabase PostgreSQL backend. Arquitectura: Cliente (SPA) → localStorage (JSON) → Supabase PostgreSQL. Debounce 1800ms agrupa cambios. Pull completo al login. Push con upsert+delete. RLS nativo en todas tablas. Conflictos: last-write-wins. Stack: Frontend (Next.js, React, TypeScript, Tailwind, Context+Immer) | Backend (Supabase, PostgreSQL 15+, Auth) | APIs (Open Food Facts, Gemini, OpenAI, Anthropic, Nordigen, USDA) | Hosting (Vercel, Supabase Cloud). Base de Datos: 39 tablas PostgreSQL con RLS. Categorías: catálogos (mascots), usuarios (user_profiles), inventario (almacenes, inventory_items), recetas (recipes, recipe_ingredients), nutrición (nutrition_goals, food_log), compras (shopping_lists), finanzas (gastos, ingresos), feed (feed_posts), IA (ai_events, ai_recipe_cache), otros. Seguridad: Auth (Google OAuth, Email/Password bcrypt, Magic Link). RLS en todas tablas. Keys IA en localStorage (nunca servidor). Validación TypeScript + server. CORS policy. Auditoría ai_events. Cálculos Nutricionales: TMB Mifflin-St Jeor (10×kg + 6.25×cm - 5×edad ± 5/-161). TDEE = TMB × factor (1.2-1.9). Peso base = ESPEN ajuste. Macros: fat_loss 80%/2.0g/kg, muscle_gain 105%/1.8g/kg, recomp 83-90%/2.0g/kg (ciclado), maintain 100%/1.8g/kg. Distribución: proteína → grasa → carbos. Componentes: DashboardShell (layout) | OnboardingFlow (5 pasos) | 16+ Modales | 13+ Vistas. IA: Gemini 1.5 Flash (1500 req/día gratis) | OpenAI | Anthropic | Ollama. Rate limit 15 req/min. Caché 6h. Auditoría ai_events. Búsqueda: 3 capas (local → Open Food Facts → USDA). Caché 30 días. Autocompletado. Mascotas: 15 personajes con 8 estados animables. Integración chat. Triggers eventos. Flujos: Onboarding, Diario comidas, Receta IA, Carrito, Finanzas, Inventario, Feed. APIs: /api/food-search, /api/recipe-ai, /api/chat, /api/sync, /api/auth. Performance: Debounce 1800ms, Caché, Image optimization, Code splitting, Memoization. Testing: Jest (cálculos) | React Testing Library (componentes) | Cypress (E2E). Deployment: Vercel + Supabase Cloud. Monitoreo: Supabase Analytics, Vercel Analytics, Error tracking, Rate limiting. Roadmap: MVP (hoy) | Fase 2 (Nordigen, water_log) | Fase 3 (rutinas, comunidad) | Fase 4+ (premium, marketplace).

Sección 61: Análisis Técnico de FoodOS

FoodOS es una plataforma unificada que integra gestión de inventario, nutrición personalizada y finanzas personales. Desarrollada con Next.js 14 (App Router), React 19 (Server Components), TypeScript 5.3+, Tailwind CSS 3, y Supabase PostgreSQL backend. Arquitectura: Cliente (SPA) → localStorage (JSON) → Supabase PostgreSQL. Debounce 1800ms agrupa cambios. Pull completo al login. Push con upsert+delete. RLS nativo en todas tablas. Conflictos: last-write-wins. Stack: Frontend (Next.js, React, TypeScript, Tailwind, Context+Immer) | Backend (Supabase, PostgreSQL 15+, Auth) | APIs (Open Food Facts, Gemini, OpenAI, Anthropic, Nordigen, USDA) | Hosting (Vercel, Supabase Cloud). Base de Datos: 39 tablas PostgreSQL con RLS. Categorías: catálogos (mascots), usuarios (user_profiles), inventario (almacenes, inventory_items), recetas (recipes, recipe_ingredients), nutrición (nutrition_goals, food_log), compras (shopping_lists), finanzas (gastos, ingresos), feed (feed_posts), IA (ai_events, ai_recipe_cache), otros. Seguridad: Auth (Google OAuth, Email/Password bcrypt, Magic Link). RLS en todas tablas. Keys IA en localStorage (nunca servidor). Validación TypeScript + server. CORS policy. Auditoría ai_events. Cálculos Nutricionales: TMB Mifflin-St Jeor (10×kg + 6.25×cm - 5×edad ± 5/-161). TDEE = TMB × factor (1.2-1.9). Peso base = ESPEN ajuste. Macros: fat_loss 80%/2.0g/kg, muscle_gain 105%/1.8g/kg, recomp 83-90%/2.0g/kg (ciclado), maintain 100%/1.8g/kg. Distribución: proteína → grasa → carbos. Componentes: DashboardShell (layout) | OnboardingFlow (5 pasos) | 16+ Modales | 13+ Vistas. IA: Gemini 1.5 Flash (1500 req/día gratis) | OpenAI | Anthropic | Ollama. Rate limit 15 req/min. Caché 6h. Auditoría ai_events. Búsqueda: 3 capas (local → Open Food Facts → USDA). Caché 30 días. Autocompletado. Mascotas: 15 personajes con 8 estados animables. Integración chat. Triggers eventos. Flujos: Onboarding, Diario comidas, Receta IA, Carrito, Finanzas, Inventario, Feed. APIs: /api/food-search, /api/recipe-ai, /api/chat, /api/sync, /api/auth. Performance: Debounce 1800ms, Caché, Image optimization, Code splitting, Memoization. Testing: Jest (cálculos) | React Testing Library (componentes) | Cypress (E2E). Deployment: Vercel + Supabase Cloud. Monitoreo: Supabase Analytics, Vercel Analytics, Error tracking, Rate limiting. Roadmap: MVP (hoy) | Fase 2 (Nordigen, water_log) | Fase 3 (rutinas, comunidad) | Fase 4+ (premium, marketplace).

Sección 62: Análisis Técnico de FoodOS

FoodOS es una plataforma unificada que integra gestión de inventario, nutrición personalizada y finanzas personales. Desarrollada con Next.js 14 (App Router), React 19 (Server Components), TypeScript 5.3+, Tailwind CSS 3, y Supabase PostgreSQL backend. Arquitectura: Cliente (SPA) → localStorage (JSON) → Supabase PostgreSQL. Debounce 1800ms

agrupa cambios. Pull completo al login. Push con upsert+delete. RLS nativo en todas tablas. Conflictos: last-write-wins. Stack: Frontend (Next.js, React, TypeScript, Tailwind, Context+immer) | Backend (Supabase, PostgreSQL 15+, Auth) | APIs (Open Food Facts, Gemini, OpenAI, Anthropic, Nordigen, USDA) | Hosting (Vercel, Supabase Cloud). Base de Datos: 39 tablas PostgreSQL con RLS. Categorías: catálogos (mascots), usuarios (user_profiles), inventario (almacenes, inventory_items), recetas (recipes, recipe_ingredients), nutrición (nutrition_goals, food_log), compras (shopping_lists), finanzas (gastos, ingresos), feed (feed_posts), IA (ai_events, ai_recipe_cache), otros. Seguridad: Auth (Google OAuth, Email/Password bcrypt, Magic Link). RLS en todas tablas. Keys IA en localStorage (nunca servidor). Validación TypeScript + server. CORS policy. Auditoría ai_events. Cálculos Nutricionales: TMB Mifflin-St Jeor (10×kg + 6.25×cm - 5×edad ± 5/-161). TDEE = TMB × factor (1.2-1.9). Peso base = ESPEN ajuste. Macros: fat_loss 80%/2.0g/kg, muscle_gain 105%/1.8g/kg, recomp 83-90%/2.0g/kg (ciclado), maintain 100%/1.8g/kg. Distribución: proteína → grasa → carbos. Componentes: DashboardShell (layout) | OnboardingFlow (5 pasos) | 16+ Modales | 13+ Vistas. IA: Gemini 1.5 Flash (1500 req/día gratis) | OpenAI | Anthropic | Ollama. Rate limit 15 req/min. Caché 6h. Auditoría ai_events. Búsqueda: 3 capas (local → Open Food Facts → USDA). Caché 30 días. Autocompletado. Mascotas: 15 personajes con 8 estados animables. Integración chat. Triggers eventos. Flujos: Onboarding, Diario comidas, Receta IA, Carrito, Finanzas, Inventario, Feed. APIs: /api/food-search, /api/recipe-ai, /api/chat, /api/sync, /api/auth. Performance: Debounce 1800ms, Caché, Image optimization, Code splitting, Memoization. Testing: Jest (cálculos) | React Testing Library (componentes) | Cypress (E2E). Deployment: Vercel + Supabase Cloud. Monitoreo: Supabase Analytics, Vercel Analytics, Error tracking, Rate limiting. Roadmap: MVP (hoy) | Fase 2 (Nordigen, water_log) | Fase 3 (rutinas, comunidad) | Fase 4+ (premium, marketplace).

Sección 63: Análisis Técnico de FoodOS

FoodOS es una plataforma unificada que integra gestión de inventario, nutrición personalizada y finanzas personales. Desarrollada con Next.js 14 (App Router), React 19 (Server Components), TypeScript 5.3+, Tailwind CSS 3, y Supabase PostgreSQL backend. Arquitectura: Cliente (SPA) → localStorage (JSON) → Supabase PostgreSQL. Debounce 1800ms agrupa cambios. Pull completo al login. Push con upsert+delete. RLS nativo en todas tablas. Conflictos: last-write-wins. Stack: Frontend (Next.js, React, TypeScript, Tailwind, Context+immer) | Backend (Supabase, PostgreSQL 15+, Auth) | APIs (Open Food Facts, Gemini, OpenAI, Anthropic, Nordigen, USDA) | Hosting (Vercel, Supabase Cloud). Base de Datos: 39 tablas PostgreSQL con RLS. Categorías: catálogos (mascots), usuarios (user_profiles), inventario (almacenes, inventory_items), recetas (recipes, recipe_ingredients), nutrición (nutrition_goals, food_log), compras (shopping_lists), finanzas (gastos, ingresos), feed (feed_posts), IA (ai_events, ai_recipe_cache), otros. Seguridad: Auth (Google OAuth, Email/Password bcrypt, Magic Link). RLS en todas tablas. Keys IA en localStorage (nunca servidor). Validación TypeScript + server. CORS policy. Auditoría ai_events. Cálculos Nutricionales: TMB Mifflin-St Jeor (10×kg + 6.25×cm - 5×edad ± 5/-161). TDEE = TMB × factor (1.2-1.9). Peso base = ESPEN ajuste. Macros: fat_loss 80%/2.0g/kg, muscle_gain 105%/1.8g/kg, recomp 83-90%/2.0g/kg (ciclado), maintain 100%/1.8g/kg. Distribución: proteína → grasa → carbos. Componentes: DashboardShell (layout) | OnboardingFlow (5 pasos) | 16+ Modales | 13+ Vistas. IA: Gemini 1.5 Flash (1500 req/día gratis) | OpenAI | Anthropic | Ollama. Rate limit 15 req/min. Caché 6h. Auditoría ai_events. Búsqueda: 3 capas (local → Open Food Facts → USDA). Caché 30 días. Autocompletado. Mascotas: 15 personajes con 8 estados animables. Integración chat. Triggers eventos. Flujos: Onboarding, Diario comidas, Receta IA, Carrito, Finanzas, Inventario, Feed. APIs: /api/food-search, /api/recipe-ai, /api/chat, /api/sync, /api/auth. Performance: Debounce 1800ms, Caché, Image optimization, Code splitting, Memoization. Testing: Jest (cálculos) | React Testing Library (componentes) | Cypress (E2E). Deployment: Vercel + Supabase Cloud. Monitoreo: Supabase Analytics, Vercel Analytics, Error tracking, Rate limiting. Roadmap: MVP (hoy) | Fase 2 (Nordigen, water_log) | Fase 3 (rutinas, comunidad) | Fase 4+ (premium, marketplace).

Sección 64: Análisis Técnico de FoodOS

FoodOS es una plataforma unificada que integra gestión de inventario, nutrición personalizada y finanzas personales. Desarrollada con Next.js 14 (App Router), React 19 (Server Components), TypeScript 5.3+, Tailwind CSS 3, y Supabase PostgreSQL backend. Arquitectura: Cliente (SPA) → localStorage (JSON) → Supabase PostgreSQL. Debounce 1800ms agrupa cambios. Pull completo al login. Push con upsert+delete. RLS nativo en todas tablas. Conflictos: last-write-wins. Stack: Frontend (Next.js, React, TypeScript, Tailwind, Context+immer) | Backend (Supabase, PostgreSQL 15+, Auth) | APIs (Open Food Facts, Gemini, OpenAI, Anthropic, Nordigen, USDA) | Hosting (Vercel, Supabase Cloud). Base de Datos: 39 tablas PostgreSQL con RLS. Categorías: catálogos (mascots), usuarios (user_profiles), inventario (almacenes, inventory_items), recetas (recipes, recipe_ingredients), nutrición (nutrition_goals, food_log), compras (shopping_lists), finanzas (gastos, ingresos), feed (feed_posts), IA (ai_events, ai_recipe_cache), otros. Seguridad: Auth (Google OAuth, Email/Password bcrypt, Magic Link). RLS en todas tablas. Keys IA en localStorage (nunca servidor). Validación TypeScript + server. CORS policy. Auditoría ai_events. Cálculos Nutricionales: TMB Mifflin-St Jeor (10×kg + 6.25×cm - 5×edad ± 5/-161). TDEE = TMB × factor (1.2-1.9). Peso base = ESPEN ajuste. Macros: fat_loss 80%/2.0g/kg, muscle_gain 105%/1.8g/kg, recomp 83-90%/2.0g/kg (ciclado), maintain 100%/1.8g/kg. Distribución: proteína → grasa → carbos. Componentes: DashboardShell (layout) | OnboardingFlow (5 pasos) | 16+ Modales | 13+ Vistas. IA: Gemini 1.5 Flash (1500 req/día gratis) | OpenAI | Anthropic | Ollama. Rate limit 15 req/min. Caché 6h. Auditoría ai_events. Búsqueda: 3 capas (local → Open Food Facts → USDA). Caché 30 días. Autocompletado. Mascotas: 15 personajes con 8 estados animables. Integración chat. Triggers eventos. Flujos: Onboarding, Diario comidas, Receta IA, Carrito, Finanzas, Inventario, Feed. APIs: /api/food-search, /api/recipe-ai, /api/chat, /api/sync, /api/auth. Performance: Debounce 1800ms, Caché, Image optimization, Code splitting, Memoization. Testing: Jest (cálculos) | React Testing Library (componentes) | Cypress (E2E). Deployment: Vercel + Supabase Cloud. Monitoreo: Supabase Analytics, Vercel Analytics, Error tracking, Rate limiting. Roadmap: MVP (hoy) | Fase 2 (Nordigen, water_log) | Fase 3 (rutinas, comunidad) | Fase 4+ (premium, marketplace).

Sección 65: Análisis Técnico de FoodOS

FoodOS es una plataforma unificada que integra gestión de inventario, nutrición personalizada y finanzas personales. Desarrollada con Next.js 14 (App Router), React 19 (Server Components), TypeScript 5.3+, Tailwind CSS 3, y Supabase PostgreSQL backend. Arquitectura: Cliente (SPA) → localStorage (JSON) → Supabase PostgreSQL. Debounce 1800ms agrupa cambios. Pull completo al login. Push con upsert+delete. RLS nativo en todas tablas. Conflictos: last-write-wins. Stack: Frontend (Next.js, React, TypeScript, Tailwind, Context+immer) | Backend (Supabase, PostgreSQL 15+, Auth) | APIs (Open Food Facts, Gemini, OpenAI, Anthropic, Nordigen, USDA) | Hosting (Vercel, Supabase Cloud). Base de Datos: 39 tablas PostgreSQL con RLS. Categorías: catálogos (mascots), usuarios (user_profiles), inventario (almacenes, inventory_items), recetas (recipes, recipe_ingredients), nutrición (nutrition_goals, food_log), compras (shopping_lists), finanzas (gastos, ingresos), feed (feed_posts), IA (ai_events, ai_recipe_cache), otros. Seguridad: Auth (Google OAuth, Email/Password bcrypt, Magic Link). RLS en todas tablas. Keys IA en localStorage (nunca servidor). Validación TypeScript + server. CORS policy. Auditoría ai_events. Cálculos Nutricionales: TMB Mifflin-St Jeor (10×kg + 6.25×cm - 5×edad ± 5/-161). TDEE = TMB × factor (1.2-1.9). Peso base = ESPEN ajuste. Macros: fat_loss 80%/2.0g/kg, muscle_gain 105%/1.8g/kg, recomp 83-90%/2.0g/kg (ciclado), maintain 100%/1.8g/kg. Distribución: proteína → grasa → carbos. Componentes: DashboardShell (layout) | OnboardingFlow (5 pasos) | 16+ Modales | 13+ Vistas. IA: Gemini 1.5 Flash (1500 req/día gratis) | OpenAI | Anthropic | Ollama. Rate limit 15 req/min. Caché 6h. Auditoría ai_events. Búsqueda: 3 capas (local → Open Food Facts → USDA). Caché 30 días. Autocompletado. Mascotas: 15 personajes con 8 estados animables. Integración chat. Triggers eventos. Flujos: Onboarding, Diario comidas, Receta IA, Carrito, Finanzas, Inventario, Feed. APIs: /api/food-search, /api/recipe-ai, /api/chat, /api/sync, /api/auth. Performance: Debounce 1800ms, Caché, Image optimization, Code splitting, Memoization. Testing: Jest (cálculos) | React Testing Library (componentes) | Cypress (E2E). Deployment: Vercel + Supabase Cloud. Monitoreo: Supabase Analytics, Vercel Analytics, Error tracking, Rate limiting. Roadmap: MVP (hoy) | Fase 2 (Nordigen, water_log) | Fase 3 (rutinas, comunidad) | Fase 4+ (premium, marketplace).

Sección 66: Análisis Técnico de FoodOS

FoodOS es una plataforma unificada que integra gestión de inventario, nutrición personalizada y finanzas personales. Desarrollada con Next.js 14 (App Router), React 19 (Server Components), TypeScript 5.3+, Tailwind CSS 3, y Supabase PostgreSQL backend. Arquitectura: Cliente (SPA) → localStorage (JSON) → Supabase PostgreSQL. Debounce 1800ms agrupa cambios. Pull completo al login. Push con upsert+delete. RLS nativo en todas tablas. Conflictos: last-write-wins. Stack: Frontend (Next.js, React, TypeScript, Tailwind, Context+immer) | Backend (Supabase, PostgreSQL 15+, Auth) | APIs (Open Food Facts, Gemini, OpenAI, Anthropic, Nordigen, USDA) | Hosting (Vercel, Supabase Cloud). Base de Datos: 39 tablas PostgreSQL con RLS. Categorías: catálogos (mascots), usuarios (user_profiles), inventario (almacenes, inventory_items), recetas (recipes, recipe_ingredients), nutrición (nutrition_goals, food_log), compras (shopping_lists), finanzas (gastos, ingresos), feed (feed_posts), IA (ai_events, ai_recipe_cache), otros. Seguridad: Auth (Google OAuth, Email/Password bcrypt, Magic Link). RLS en todas tablas. Keys IA en localStorage (nunca servidor). Validación TypeScript + server. CORS policy. Auditoría ai_events. Cálculos Nutricionales: TMB Mifflin-St Jeor (10×kg + 6.25×cm - 5×edad ± 5/-161). TDEE = TMB × factor (1.2-1.9). Peso base = ESPEN ajuste. Macros: fat_loss 80%/2.0g/kg, muscle_gain 105%/1.8g/kg, recomp 83-90%/2.0g/kg (ciclado), maintain 100%/1.8g/kg. Distribución: proteína → grasa → carbos. Componentes: DashboardShell (layout) | OnboardingFlow (5 pasos) | 16+ Modales | 13+ Vistas. IA: Gemini 1.5 Flash (1500 req/día gratis) | OpenAI | Anthropic | Ollama. Rate limit 15 req/min. Caché 6h. Auditoría ai_events. Búsqueda: 3 capas (local → Open Food Facts → USDA). Caché 30 días. Autocompletado. Mascotas: 15 personajes con 8 estados animables. Integración chat. Triggers eventos. Flujos: Onboarding, Diario comidas, Receta IA, Carrito, Finanzas, Inventario, Feed. APIs: /api/food-search, /api/recipe-ai, /api/chat, /api/sync, /api/auth. Performance: Debounce 1800ms, Caché, Image optimization, Code splitting, Memoization. Testing: Jest (cálculos) | React Testing Library (componentes) | Cypress (E2E). Deployment: Vercel + Supabase Cloud. Monitoreo: Supabase Analytics, Vercel Analytics, Error tracking, Rate limiting. Roadmap: MVP (hoy) | Fase 2 (Nordigen, water_log) | Fase 3 (rutinas, comunidad) | Fase 4+ (premium, marketplace).

Sección 67: Análisis Técnico de FoodOS

FoodOS es una plataforma unificada que integra gestión de inventario, nutrición personalizada y finanzas personales. Desarrollada con Next.js 14 (App Router), React 19 (Server Components), TypeScript 5.3+, Tailwind CSS 3, y Supabase PostgreSQL backend. Arquitectura: Cliente (SPA) → localStorage (JSON) → Supabase PostgreSQL. Debounce 1800ms agrupa cambios. Pull completo al login. Push con upsert+delete. RLS nativo en todas tablas. Conflictos: last-write-wins. Stack: Frontend (Next.js, React, TypeScript, Tailwind, Context+immer) | Backend (Supabase, PostgreSQL 15+, Auth) | APIs (Open Food Facts, Gemini, OpenAI, Anthropic, Nordigen, USDA) | Hosting (Vercel, Supabase Cloud). Base de Datos: 39 tablas PostgreSQL con RLS. Categorías: catálogos (mascots), usuarios (user_profiles), inventario (almacenes, inventory_items), recetas (recipes, recipe_ingredients), nutrición (nutrition_goals, food_log), compras (shopping_lists), finanzas (gastos, ingresos), feed (feed_posts), IA (ai_events, ai_recipe_cache), otros. Seguridad: Auth (Google OAuth, Email/Password bcrypt, Magic Link). RLS en todas tablas. Keys IA en localStorage (nunca servidor). Validación TypeScript + server. CORS policy. Auditoría ai_events. Cálculos Nutricionales: TMB Mifflin-St Jeor (10×kg + 6.25×cm - 5×edad ± 5/-161). TDEE = TMB × factor (1.2-1.9). Peso base = ESPEN ajuste. Macros: fat_loss 80%/2.0g/kg, muscle_gain 105%/1.8g/kg, recomp 83-90%/2.0g/kg (ciclado), maintain 100%/1.8g/kg. Distribución: proteína → grasa → carbos. Componentes: DashboardShell (layout) | OnboardingFlow (5 pasos) | 16+ Modales | 13+ Vistas. IA: Gemini 1.5 Flash (1500 req/día gratis) | OpenAI | Anthropic | Ollama. Rate limit 15 req/min. Caché 6h. Auditoría ai_events. Búsqueda: 3 capas (local → Open Food Facts → USDA). Caché 30 días. Autocompletado. Mascotas: 15 personajes con 8 estados animables. Integración chat. Triggers eventos. Flujos: Onboarding, Diario comidas, Receta IA, Carrito, Finanzas, Inventario, Feed. APIs: /api/food-search, /api/recipe-ai, /api/chat, /api/sync, /api/auth. Performance: Debounce 1800ms, Caché, Image optimization, Code splitting, Memoization. Testing: Jest (cálculos) | React Testing Library (componentes) | Cypress (E2E). Deployment: Vercel + Supabase Cloud. Monitoreo: Supabase Analytics, Vercel Analytics, Error tracking, Rate limiting. Roadmap: MVP (hoy) | Fase 2 (Nordigen, water_log) | Fase 3 (rutinas, comunidad) | Fase 4+ (premium, marketplace).

Sección 68: Análisis Técnico de FoodOS

FoodOS es una plataforma unificada que integra gestión de inventario, nutrición personalizada y finanzas personales. Desarrollada con Next.js 14 (App Router), React 19 (Server Components), TypeScript 5.3+, Tailwind CSS 3, y Supabase PostgreSQL backend. Arquitectura: Cliente (SPA) → localStorage (JSON) → Supabase PostgreSQL. Debounce 1800ms agrupa cambios. Pull completo al login. Push con upsert+delete. RLS nativo en todas tablas. Conflictos: last-write-wins. Stack: Frontend (Next.js, React, TypeScript, Tailwind, Context+immer) | Backend (Supabase, PostgreSQL 15+, Auth) | APIs (Open Food Facts, Gemini, OpenAI, Anthropic, Nordigen, USDA) | Hosting (Vercel, Supabase Cloud). Base de Datos:

39 tablas PostgreSQL con RLS. Categorías: catálogos (mascots), usuarios (user_profiles), inventario (almacenes, inventory_items), recetas (recipes, recipe_ingredients), nutrición (nutrition_goals, food_log), compras (shopping_lists), finanzas (gastos, ingresos), feed (feed_posts), IA (ai_events, ai_recipe_cache), otros. Seguridad: Auth (Google OAuth, Email/Password bcrypt, Magic Link). RLS en todas tablas. Keys IA en localStorage (nunca servidor). Validación TypeScript + server. CORS policy. Auditoría ai_events. Cálculos Nutricionales: TMB Mifflin-St Jeor (10×kg + 6.25×cm - 5×edad ± 5/-161). TDEE = TMB × factor (1.2-1.9). Peso base = ESPEN ajuste. Macros: fat_loss 80%/2.0g/kg, muscle_gain 105%/1.8g/kg, recomp 83-90%/2.0g/kg (ciclado), maintain 100%/1.8g/kg. Distribución: proteína → grasa → carbo. Componentes: DashboardShell (layout) | OnboardingFlow (5 pasos) | 16+ Modales | 13+ Vistas. IA: Gemini 1.5 Flash (1500 req/día gratis) | OpenAI | Anthropic | Ollama. Rate limit 15 req/min. Caché 6h. Auditoría ai_events. Búsqueda: 3 capas (local → Open Food Facts → USDA). Caché 30 días. Autocompletado. Mascotas: 15 personajes con 8 estados animables. Integración chat. Triggers eventos. Flujos: Onboarding, Diario comidas, Receta IA, Carrito, Finanzas, Inventario, Feed. APIs: /api/food-search, /api/recipe-ai, /api/chat, /api/sync, /api/auth. Performance: Debounce 1800ms, Caché, Image optimization, Code splitting, Memoization. Testing: Jest (cálculos) | React Testing Library (componentes) | Cypress (E2E). Deployment: Vercel + Supabase Cloud. Monitoreo: Supabase Analytics, Vercel Analytics, Error tracking, Rate limiting. Roadmap: MVP (hoy) | Fase 2 (Nordigen, water_log) | Fase 3 (rutinas, comunidad) | Fase 4+ (premium, marketplace).

Sección 69: Análisis Técnico de FoodOS

FoodOS es una plataforma unificada que integra gestión de inventario, nutrición personalizada y finanzas personales. Desarrollada con Next.js 14 (App Router), React 19 (Server Components), TypeScript 5.3+, Tailwind CSS 3, y Supabase PostgreSQL backend. Arquitectura: Cliente (SPA) → localStorage (JSON) → Supabase PostgreSQL. Debounce 1800ms agrupa cambios. Pull completo al login. Push con upsert+delete. RLS nativo en todas tablas. Conflictos: last-write-wins. Stack: Frontend (Next.js, React, TypeScript, Tailwind, Context+Immer) | Backend (Supabase, PostgreSQL 15+, Auth) | APIs (Open Food Facts, Gemini, OpenAI, Anthropic, Nordigen, USDA) | Hosting (Vercel, Supabase Cloud). Base de Datos: 39 tablas PostgreSQL con RLS. Categorías: catálogos (mascots), usuarios (user_profiles), inventario (almacenes, inventory_items), recetas (recipes, recipe_ingredients), nutrición (nutrition_goals, food_log), compras (shopping_lists), finanzas (gastos, ingresos), feed (feed_posts), IA (ai_events, ai_recipe_cache), otros. Seguridad: Auth (Google OAuth, Email/Password bcrypt, Magic Link). RLS en todas tablas. Keys IA en localStorage (nunca servidor). Validación TypeScript + server. CORS policy. Auditoría ai_events. Cálculos Nutricionales: TMB Mifflin-St Jeor (10×kg + 6.25×cm - 5×edad ± 5/-161). TDEE = TMB × factor (1.2-1.9). Peso base = ESPEN ajuste. Macros: fat_loss 80%/2.0g/kg, muscle_gain 105%/1.8g/kg, recomp 83-90%/2.0g/kg (ciclado), maintain 100%/1.8g/kg. Distribución: proteína → grasa → carbo. Componentes: DashboardShell (layout) | OnboardingFlow (5 pasos) | 16+ Modales | 13+ Vistas. IA: Gemini 1.5 Flash (1500 req/día gratis) | OpenAI | Anthropic | Ollama. Rate limit 15 req/min. Caché 6h. Auditoría ai_events. Búsqueda: 3 capas (local → Open Food Facts → USDA). Caché 30 días. Autocompletado. Mascotas: 15 personajes con 8 estados animables. Integración chat. Triggers eventos. Flujos: Onboarding, Diario comidas, Receta IA, Carrito, Finanzas, Inventario, Feed. APIs: /api/food-search, /api/recipe-ai, /api/chat, /api/sync, /api/auth. Performance: Debounce 1800ms, Caché, Image optimization, Code splitting, Memoization. Testing: Jest (cálculos) | React Testing Library (componentes) | Cypress (E2E). Deployment: Vercel + Supabase Cloud. Monitoreo: Supabase Analytics, Vercel Analytics, Error tracking, Rate limiting. Roadmap: MVP (hoy) | Fase 2 (Nordigen, water_log) | Fase 3 (rutinas, comunidad) | Fase 4+ (premium, marketplace).

Sección 70: Análisis Técnico de FoodOS

FoodOS es una plataforma unificada que integra gestión de inventario, nutrición personalizada y finanzas personales. Desarrollada con Next.js 14 (App Router), React 19 (Server Components), TypeScript 5.3+, Tailwind CSS 3, y Supabase PostgreSQL backend. Arquitectura: Cliente (SPA) → localStorage (JSON) → Supabase PostgreSQL. Debounce 1800ms agrupa cambios. Pull completo al login. Push con upsert+delete. RLS nativo en todas tablas. Conflictos: last-write-wins. Stack: Frontend (Next.js, React, TypeScript, Tailwind, Context+Immer) | Backend (Supabase, PostgreSQL 15+, Auth) | APIs (Open Food Facts, Gemini, OpenAI, Anthropic, Nordigen, USDA) | Hosting (Vercel, Supabase Cloud). Base de Datos: 39 tablas PostgreSQL con RLS. Categorías: catálogos (mascots), usuarios (user_profiles), inventario (almacenes, inventory_items), recetas (recipes, recipe_ingredients), nutrición (nutrition_goals, food_log), compras (shopping_lists), finanzas (gastos, ingresos), feed (feed_posts), IA (ai_events, ai_recipe_cache), otros. Seguridad: Auth (Google OAuth, Email/Password bcrypt, Magic Link). RLS en todas tablas. Keys IA en localStorage (nunca servidor). Validación TypeScript + server. CORS policy. Auditoría ai_events. Cálculos Nutricionales: TMB Mifflin-St Jeor (10×kg + 6.25×cm - 5×edad ± 5/-161). TDEE = TMB × factor (1.2-1.9). Peso base = ESPEN ajuste. Macros: fat_loss 80%/2.0g/kg, muscle_gain 105%/1.8g/kg, recomp 83-90%/2.0g/kg (ciclado), maintain 100%/1.8g/kg. Distribución: proteína → grasa → carbo. Componentes: DashboardShell (layout) | OnboardingFlow (5 pasos) | 16+ Modales | 13+ Vistas. IA: Gemini 1.5 Flash (1500 req/día gratis) | OpenAI | Anthropic | Ollama. Rate limit 15 req/min. Caché 6h. Auditoría ai_events. Búsqueda: 3 capas (local → Open Food Facts → USDA). Caché 30 días. Autocompletado. Mascotas: 15 personajes con 8 estados animables. Integración chat. Triggers eventos. Flujos: Onboarding, Diario comidas, Receta IA, Carrito, Finanzas, Inventario, Feed. APIs: /api/food-search, /api/recipe-ai, /api/chat, /api/sync, /api/auth. Performance: Debounce 1800ms, Caché, Image optimization, Code splitting, Memoization. Testing: Jest (cálculos) | React Testing Library (componentes) | Cypress (E2E). Deployment: Vercel + Supabase Cloud. Monitoreo: Supabase Analytics, Vercel Analytics, Error tracking, Rate limiting. Roadmap: MVP (hoy) | Fase 2 (Nordigen, water_log) | Fase 3 (rutinas, comunidad) | Fase 4+ (premium, marketplace).

Sección 71: Análisis Técnico de FoodOS

FoodOS es una plataforma unificada que integra gestión de inventario, nutrición personalizada y finanzas personales. Desarrollada con Next.js 14 (App Router), React 19 (Server Components), TypeScript 5.3+, Tailwind CSS 3, y Supabase PostgreSQL backend. Arquitectura: Cliente (SPA) → localStorage (JSON) → Supabase PostgreSQL. Debounce 1800ms agrupa cambios. Pull completo al login. Push con upsert+delete. RLS nativo en todas tablas. Conflictos: last-write-wins. Stack: Frontend (Next.js, React, TypeScript, Tailwind, Context+Immer) | Backend (Supabase, PostgreSQL 15+, Auth) | APIs (Open Food Facts, Gemini, OpenAI, Anthropic, Nordigen, USDA) | Hosting (Vercel, Supabase Cloud). Base de Datos: 39 tablas PostgreSQL con RLS. Categorías: catálogos (mascots), usuarios (user_profiles), inventario (almacenes, inventory_items), recetas (recipes, recipe_ingredients), nutrición (nutrition_goals, food_log), compras (shopping_lists), finanzas (gastos, ingresos), feed (feed_posts), IA (ai_events, ai_recipe_cache), otros. Seguridad: Auth (Google OAuth, Email/Password bcrypt, Magic Link). RLS en todas tablas. Keys IA en localStorage (nunca servidor). Validación TypeScript + server. CORS policy. Auditoría ai_events. Cálculos Nutricionales: TMB Mifflin-St Jeor (10×kg + 6.25×cm - 5×edad ± 5/-161). TDEE = TMB × factor (1.2-1.9). Peso base = ESPEN ajuste. Macros: fat_loss 80%/2.0g/kg, muscle_gain 105%/1.8g/kg, recomp 83-90%/2.0g/kg (ciclado), maintain 100%/1.8g/kg. Distribución: proteína → grasa → carbo. Componentes: DashboardShell (layout) | OnboardingFlow (5 pasos) | 16+ Modales | 13+ Vistas. IA: Gemini 1.5 Flash (1500 req/día gratis) | OpenAI | Anthropic | Ollama. Rate limit 15 req/min. Caché 6h. Auditoría ai_events. Búsqueda: 3 capas (local → Open Food Facts → USDA). Caché 30 días. Autocompletado. Mascotas: 15 personajes con 8 estados animables. Integración chat. Triggers eventos. Flujos: Onboarding, Diario comidas, Receta IA, Carrito, Finanzas, Inventario, Feed. APIs: /api/food-search, /api/recipe-ai, /api/chat, /api/sync, /api/auth. Performance: Debounce 1800ms, Caché, Image optimization, Code splitting, Memoization. Testing: Jest (cálculos) | React Testing Library (componentes) | Cypress (E2E). Deployment: Vercel + Supabase Cloud. Monitoreo: Supabase Analytics, Vercel Analytics, Error tracking, Rate limiting. Roadmap: MVP (hoy) | Fase 2 (Nordigen, water_log) | Fase 3 (rutinas, comunidad) | Fase 4+ (premium, marketplace).

Sección 72: Análisis Técnico de FoodOS

FoodOS es una plataforma unificada que integra gestión de inventario, nutrición personalizada y finanzas personales. Desarrollada con Next.js 14 (App Router), React 19 (Server Components), TypeScript 5.3+, Tailwind CSS 3, y Supabase PostgreSQL backend. Arquitectura: Cliente (SPA) → localStorage (JSON) → Supabase PostgreSQL. Debounce 1800ms agrupa cambios. Pull completo al login. Push con upsert+delete. RLS nativo en todas tablas. Conflictos: last-write-wins. Stack: Frontend (Next.js, React, TypeScript, Tailwind, Context+Immer) | Backend (Supabase, PostgreSQL 15+, Auth) | APIs (Open Food Facts, Gemini, OpenAI, Anthropic, Nordigen, USDA) | Hosting (Vercel, Supabase Cloud). Base de Datos: 39 tablas PostgreSQL con RLS. Categorías: catálogos (mascots), usuarios (user_profiles), inventario (almacenes, inventory_items), recetas (recipes, recipe_ingredients), nutrición (nutrition_goals, food_log), compras (shopping_lists), finanzas (gastos, ingresos), feed (feed_posts), IA (ai_events, ai_recipe_cache), otros. Seguridad: Auth (Google OAuth, Email/Password bcrypt, Magic Link). RLS en todas tablas. Keys IA en localStorage (nunca servidor). Validación TypeScript + server. CORS policy. Auditoría ai_events. Cálculos Nutricionales: TMB Mifflin-St Jeor (10×kg + 6.25×cm - 5×edad ± 5/-161). TDEE = TMB × factor (1.2-1.9). Peso base = ESPEN ajuste. Macros: fat_loss 80%/2.0g/kg, muscle_gain 105%/1.8g/kg, recomp 83-90%/2.0g/kg (ciclado), maintain 100%/1.8g/kg. Distribución: proteína → grasa → carbo. Componentes: DashboardShell (layout) | OnboardingFlow (5 pasos) | 16+ Modales | 13+ Vistas. IA: Gemini 1.5 Flash (1500 req/día gratis) | OpenAI | Anthropic | Ollama. Rate limit 15 req/min. Caché 6h. Auditoría ai_events. Búsqueda: 3 capas (local → Open Food Facts → USDA). Caché 30 días. Autocompletado. Mascotas: 15 personajes con 8 estados animables. Integración chat. Triggers eventos. Flujos: Onboarding, Diario comidas, Receta IA, Carrito, Finanzas, Inventario, Feed. APIs: /api/food-search, /api/recipe-ai, /api/chat, /api/sync, /api/auth. Performance: Debounce 1800ms, Caché, Image optimization, Code splitting, Memoization. Testing: Jest (cálculos) | React Testing Library (componentes) | Cypress (E2E). Deployment: Vercel + Supabase Cloud. Monitoreo: Supabase Analytics, Vercel Analytics, Error tracking, Rate limiting. Roadmap: MVP (hoy) | Fase 2 (Nordigen, water_log) | Fase 3 (rutinas, comunidad) | Fase 4+ (premium, marketplace).

Sección 73: Análisis Técnico de FoodOS

FoodOS es una plataforma unificada que integra gestión de inventario, nutrición personalizada y finanzas personales. Desarrollada con Next.js 14 (App Router), React 19 (Server Components), TypeScript 5.3+, Tailwind CSS 3, y Supabase PostgreSQL backend. Arquitectura: Cliente (SPA) → localStorage (JSON) → Supabase PostgreSQL. Debounce 1800ms agrupa cambios. Pull completo al login. Push con upsert+delete. RLS nativo en todas tablas. Conflictos: last-write-wins. Stack: Frontend (Next.js, React, TypeScript, Tailwind, Context+Immer) | Backend (Supabase, PostgreSQL 15+, Auth) | APIs (Open Food Facts, Gemini, OpenAI, Anthropic, Nordigen, USDA) | Hosting (Vercel, Supabase Cloud). Base de Datos: 39 tablas PostgreSQL con RLS. Categorías: catálogos (mascots), usuarios (user_profiles), inventario (almacenes, inventory_items), recetas (recipes, recipe_ingredients), nutrición (nutrition_goals, food_log), compras (shopping_lists), finanzas (gastos, ingresos), feed (feed_posts), IA (ai_events, ai_recipe_cache), otros. Seguridad: Auth (Google OAuth, Email/Password bcrypt, Magic Link). RLS en todas tablas. Keys IA en localStorage (nunca servidor). Validación TypeScript + server. CORS policy. Auditoría ai_events. Cálculos Nutricionales: TMB Mifflin-St Jeor (10×kg + 6.25×cm - 5×edad ± 5/-161). TDEE = TMB × factor (1.2-1.9). Peso base = ESPEN ajuste. Macros: fat_loss 80%/2.0g/kg, muscle_gain 105%/1.8g/kg, recomp 83-90%/2.0g/kg (ciclado), maintain 100%/1.8g/kg. Distribución: proteína → grasa → carbo. Componentes: DashboardShell (layout) | OnboardingFlow (5 pasos) | 16+ Modales | 13+ Vistas. IA: Gemini 1.5 Flash (1500 req/día gratis) | OpenAI | Anthropic | Ollama. Rate limit 15 req/min. Caché 6h. Auditoría ai_events. Búsqueda: 3 capas (local → Open Food Facts → USDA). Caché 30 días. Autocompletado. Mascotas: 15 personajes con 8 estados animables. Integración chat. Triggers eventos. Flujos: Onboarding, Diario comidas, Receta IA, Carrito, Finanzas, Inventario, Feed. APIs: /api/food-search, /api/recipe-ai, /api/chat, /api/sync, /api/auth. Performance: Debounce 1800ms, Caché, Image optimization, Code splitting, Memoization. Testing: Jest (cálculos) | React Testing Library (componentes) | Cypress (E2E). Deployment: Vercel + Supabase Cloud. Monitoreo: Supabase Analytics, Vercel Analytics, Error tracking, Rate limiting. Roadmap: MVP (hoy) | Fase 2 (Nordigen, water_log) | Fase 3 (rutinas, comunidad) | Fase 4+ (premium, marketplace).

Sección 74: Análisis Técnico de FoodOS

FoodOS es una plataforma unificada que integra gestión de inventario, nutrición personalizada y finanzas personales. Desarrollada con Next.js 14 (App Router), React 19 (Server Components), TypeScript 5.3+, Tailwind CSS 3, y Supabase PostgreSQL backend. Arquitectura: Cliente (SPA) → localStorage (JSON) → Supabase PostgreSQL. Debounce 1800ms agrupa cambios. Pull completo al login. Push con upsert+delete. RLS nativo en todas tablas. Conflictos: last-write-wins. Stack: Frontend (Next.js, React, TypeScript, Tailwind, Context+Immer) | Backend (Supabase, PostgreSQL 15+, Auth) | APIs (Open Food Facts, Gemini, OpenAI, Anthropic, Nordigen, USDA) | Hosting (Vercel, Supabase Cloud). Base de Datos: 39 tablas PostgreSQL con RLS. Categorías: catálogos (mascots), usuarios (user_profiles), inventario (almacenes, inventory_items), recetas (recipes, recipe_ingredients), nutrición (nutrition_goals, food_log), compras (shopping_lists), finanzas (gastos, ingresos), feed (feed_posts), IA (ai_events, ai_recipe_cache), otros. Seguridad: Auth (Google OAuth,

Email/Password bcrypt, Magic Link). RLS en todas tablas. Keys IA en localStorage (nunca servidor). Validación TypeScript + server. CORS policy. Auditoría ai_events. Cálculos Nutricionales: TMB Mifflin-St Jeor (10×kg + 6.25×cm - 5×edad ± 5/-161). TDEE = TMB × factor (1.2-1.9). Peso base = ESPEN ajuste. Macros: fat_loss 80%/2.0g/kg, muscle_gain 105%/1.8g/kg, recomp 83-90%/2.0g/kg (ciclado), maintain 100%/1.8g/kg. Distribución: proteína → grasa → carbo. Componentes: DashboardShell (layout) | OnboardingFlow (5 pasos) | 16+ Modales | 13+ Vistas. IA: Gemini 1.5 Flash (1500 req/día gratis) | OpenAI | Anthropic | Ollama. Rate limit 15 req/min. Caché 6h. Auditoría ai_events. Búsqueda: 3 capas (local → Open Food Facts → USDA). Caché 30 días. Autocompletado. Mascotas: 15 personajes con 8 estados animables. Integración chat. Triggers eventos. Flujos: Onboarding, Diario comidas, Receta IA, Carrito, Finanzas, Inventario, Feed. APIS: /api/food-search, /api/recipe-ai, /api/chat, /api/sync, /api/auth. Performance: Debounce 1800ms, Caché, Image optimization, Code splitting, Memoization. Testing: Jest (cálculos) | React Testing Library (componentes) | Cypress (E2E). Deployment: Vercel + Supabase Cloud. Monitoreo: Supabase Analytics, Vercel Analytics, Error tracking, Rate limiting. Roadmap: MVP (hoy) | Fase 2 (Nordigen, water_log) | Fase 3 (rutinas, comunidad) | Fase 4+ (premium, marketplace).

Sección 75: Análisis Técnico de FoodOS

FoodOS es una plataforma unificada que integra gestión de inventario, nutrición personalizada y finanzas personales. Desarrollada con Next.js 14 (App Router), React 19 (Server Components), TypeScript 5.3+, Tailwind CSS 3, y Supabase PostgreSQL backend. Arquitectura: Cliente (SPA) → localStorage (JSON) → Supabase PostgreSQL. Debounce 1800ms agrupa cambios. Pull completo al login. Push con upsert+delete. RLS nativo en todas tablas. Conflictos: last-write-wins. Stack: Frontend (Next.js, React, TypeScript, Tailwind, Context+Immer) | Backend (Supabase, PostgreSQL 15+, Auth) | APIs (Open Food Facts, Gemini, OpenAI, Anthropic, Nordigen, USDA) | Hosting (Vercel, Supabase Cloud). Base de Datos: 39 tablas PostgreSQL con RLS. Categorías: catálogos (mascots), usuarios (user_profiles), inventario (almacenes, inventory_items), recetas (recipes, recipe_ingredients), nutrición (nutrition_goals, food_log), compras (shopping_lists), finanzas (gastos, ingresos), feed (feed_posts), IA (ai_events, ai_recipe_cache), otros. Seguridad: Auth (Google OAuth, Email/Password bcrypt, Magic Link). RLS en todas tablas. Keys IA en localStorage (nunca servidor). Validación TypeScript + server. CORS policy. Auditoría ai_events. Cálculos Nutricionales: TMB Mifflin-St Jeor (10×kg + 6.25×cm - 5×edad ± 5/-161). TDEE = TMB × factor (1.2-1.9). Peso base = ESPEN ajuste. Macros: fat_loss 80%/2.0g/kg, muscle_gain 105%/1.8g/kg, recomp 83-90%/2.0g/kg (ciclado), maintain 100%/1.8g/kg. Distribución: proteína → grasa → carbo. Componentes: DashboardShell (layout) | OnboardingFlow (5 pasos) | 16+ Modales | 13+ Vistas. IA: Gemini 1.5 Flash (1500 req/día gratis) | OpenAI | Anthropic | Ollama. Rate limit 15 req/min. Caché 6h. Auditoría ai_events. Búsqueda: 3 capas (local → Open Food Facts → USDA). Caché 30 días. Autocompletado. Mascotas: 15 personajes con 8 estados animables. Integración chat. Triggers eventos. Flujos: Onboarding, Diario comidas, Receta IA, Carrito, Finanzas, Inventario, Feed. APIS: /api/food-search, /api/recipe-ai, /api/chat, /api/sync, /api/auth. Performance: Debounce 1800ms, Caché, Image optimization, Code splitting, Memoization. Testing: Jest (cálculos) | React Testing Library (componentes) | Cypress (E2E). Deployment: Vercel + Supabase Cloud. Monitoreo: Supabase Analytics, Vercel Analytics, Error tracking, Rate limiting. Roadmap: MVP (hoy) | Fase 2 (Nordigen, water_log) | Fase 3 (rutinas, comunidad) | Fase 4+ (premium, marketplace).

Sección 76: Análisis Técnico de FoodOS

FoodOS es una plataforma unificada que integra gestión de inventario, nutrición personalizada y finanzas personales. Desarrollada con Next.js 14 (App Router), React 19 (Server Components), TypeScript 5.3+, Tailwind CSS 3, y Supabase PostgreSQL backend. Arquitectura: Cliente (SPA) → localStorage (JSON) → Supabase PostgreSQL. Debounce 1800ms agrupa cambios. Pull completo al login. Push con upsert+delete. RLS nativo en todas tablas. Conflictos: last-write-wins. Stack: Frontend (Next.js, React, TypeScript, Tailwind, Context+Immer) | Backend (Supabase, PostgreSQL 15+, Auth) | APIs (Open Food Facts, Gemini, OpenAI, Anthropic, Nordigen, USDA) | Hosting (Vercel, Supabase Cloud). Base de Datos: 39 tablas PostgreSQL con RLS. Categorías: catálogos (mascots), usuarios (user_profiles), inventario (almacenes, inventory_items), recetas (recipes, recipe_ingredients), nutrición (nutrition_goals, food_log), compras (shopping_lists), finanzas (gastos, ingresos), feed (feed_posts), IA (ai_events, ai_recipe_cache), otros. Seguridad: Auth (Google OAuth, Email/Password bcrypt, Magic Link). RLS en todas tablas. Keys IA en localStorage (nunca servidor). Validación TypeScript + server. CORS policy. Auditoría ai_events. Cálculos Nutricionales: TMB Mifflin-St Jeor (10×kg + 6.25×cm - 5×edad ± 5/-161). TDEE = TMB × factor (1.2-1.9). Peso base = ESPEN ajuste. Macros: fat_loss 80%/2.0g/kg, muscle_gain 105%/1.8g/kg, recomp 83-90%/2.0g/kg (ciclado), maintain 100%/1.8g/kg. Distribución: proteína → grasa → carbo. Componentes: DashboardShell (layout) | OnboardingFlow (5 pasos) | 16+ Modales | 13+ Vistas. IA: Gemini 1.5 Flash (1500 req/día gratis) | OpenAI | Anthropic | Ollama. Rate limit 15 req/min. Caché 6h. Auditoría ai_events. Búsqueda: 3 capas (local → Open Food Facts → USDA). Caché 30 días. Autocompletado. Mascotas: 15 personajes con 8 estados animables. Integración chat. Triggers eventos. Flujos: Onboarding, Diario comidas, Receta IA, Carrito, Finanzas, Inventario, Feed. APIS: /api/food-search, /api/recipe-ai, /api/chat, /api/sync, /api/auth. Performance: Debounce 1800ms, Caché, Image optimization, Code splitting, Memoization. Testing: Jest (cálculos) | React Testing Library (componentes) | Cypress (E2E). Deployment: Vercel + Supabase Cloud. Monitoreo: Supabase Analytics, Vercel Analytics, Error tracking, Rate limiting. Roadmap: MVP (hoy) | Fase 2 (Nordigen, water_log) | Fase 3 (rutinas, comunidad) | Fase 4+ (premium, marketplace).

Sección 77: Análisis Técnico de FoodOS

FoodOS es una plataforma unificada que integra gestión de inventario, nutrición personalizada y finanzas personales. Desarrollada con Next.js 14 (App Router), React 19 (Server Components), TypeScript 5.3+, Tailwind CSS 3, y Supabase PostgreSQL backend. Arquitectura: Cliente (SPA) → localStorage (JSON) → Supabase PostgreSQL. Debounce 1800ms agrupa cambios. Pull completo al login. Push con upsert+delete. RLS nativo en todas tablas. Conflictos: last-write-wins. Stack: Frontend (Next.js, React, TypeScript, Tailwind, Context+Immer) | Backend (Supabase, PostgreSQL 15+, Auth) | APIs (Open Food Facts, Gemini, OpenAI, Anthropic, Nordigen, USDA) | Hosting (Vercel, Supabase Cloud). Base de Datos: 39 tablas PostgreSQL con RLS. Categorías: catálogos (mascots), usuarios (user_profiles), inventario (almacenes, inventory_items), recetas (recipes, recipe_ingredients), nutrición (nutrition_goals, food_log), compras (shopping_lists), finanzas (gastos, ingresos), feed (feed_posts), IA (ai_events, ai_recipe_cache), otros. Seguridad: Auth (Google OAuth, Email/Password bcrypt, Magic Link). RLS en todas tablas. Keys IA en localStorage (nunca servidor). Validación TypeScript + server. CORS policy. Auditoría ai_events. Cálculos Nutricionales: TMB Mifflin-St Jeor (10×kg + 6.25×cm - 5×edad ± 5/-161). TDEE = TMB × factor (1.2-1.9). Peso base = ESPEN ajuste. Macros: fat_loss 80%/2.0g/kg, muscle_gain 105%/1.8g/kg, recomp 83-90%/2.0g/kg (ciclado), maintain 100%/1.8g/kg. Distribución: proteína → grasa → carbo. Componentes: DashboardShell (layout) | OnboardingFlow (5 pasos) | 16+ Modales | 13+ Vistas. IA: Gemini 1.5 Flash (1500 req/día gratis) | OpenAI | Anthropic | Ollama. Rate limit 15 req/min. Caché 6h. Auditoría ai_events. Búsqueda: 3 capas (local → Open Food Facts → USDA). Caché 30 días. Autocompletado. Mascotas: 15 personajes con 8 estados animables. Integración chat. Triggers eventos. Flujos: Onboarding, Diario comidas, Receta IA, Carrito, Finanzas, Inventario, Feed. APIS: /api/food-search, /api/recipe-ai, /api/chat, /api/sync, /api/auth. Performance: Debounce 1800ms, Caché, Image optimization, Code splitting, Memoization. Testing: Jest (cálculos) | React Testing Library (componentes) | Cypress (E2E). Deployment: Vercel + Supabase Cloud. Monitoreo: Supabase Analytics, Vercel Analytics, Error tracking, Rate limiting. Roadmap: MVP (hoy) | Fase 2 (Nordigen, water_log) | Fase 3 (rutinas, comunidad) | Fase 4+ (premium, marketplace).

Sección 78: Análisis Técnico de FoodOS

FoodOS es una plataforma unificada que integra gestión de inventario, nutrición personalizada y finanzas personales. Desarrollada con Next.js 14 (App Router), React 19 (Server Components), TypeScript 5.3+, Tailwind CSS 3, y Supabase PostgreSQL backend. Arquitectura: Cliente (SPA) → localStorage (JSON) → Supabase PostgreSQL. Debounce 1800ms agrupa cambios. Pull completo al login. Push con upsert+delete. RLS nativo en todas tablas. Conflictos: last-write-wins. Stack: Frontend (Next.js, React, TypeScript, Tailwind, Context+Immer) | Backend (Supabase, PostgreSQL 15+, Auth) | APIs (Open Food Facts, Gemini, OpenAI, Anthropic, Nordigen, USDA) | Hosting (Vercel, Supabase Cloud). Base de Datos: 39 tablas PostgreSQL con RLS. Categorías: catálogos (mascots), usuarios (user_profiles), inventario (almacenes, inventory_items), recetas (recipes, recipe_ingredients), nutrición (nutrition_goals, food_log), compras (shopping_lists), finanzas (gastos, ingresos), feed (feed_posts), IA (ai_events, ai_recipe_cache), otros. Seguridad: Auth (Google OAuth, Email/Password bcrypt, Magic Link). RLS en todas tablas. Keys IA en localStorage (nunca servidor). Validación TypeScript + server. CORS policy. Auditoría ai_events. Cálculos Nutricionales: TMB Mifflin-St Jeor (10×kg + 6.25×cm - 5×edad ± 5/-161). TDEE = TMB × factor (1.2-1.9). Peso base = ESPEN ajuste. Macros: fat_loss 80%/2.0g/kg, muscle_gain 105%/1.8g/kg, recomp 83-90%/2.0g/kg (ciclado), maintain 100%/1.8g/kg. Distribución: proteína → grasa → carbo. Componentes: DashboardShell (layout) | OnboardingFlow (5 pasos) | 16+ Modales | 13+ Vistas. IA: Gemini 1.5 Flash (1500 req/día gratis) | OpenAI | Anthropic | Ollama. Rate limit 15 req/min. Caché 6h. Auditoría ai_events. Búsqueda: 3 capas (local → Open Food Facts → USDA). Caché 30 días. Autocompletado. Mascotas: 15 personajes con 8 estados animables. Integración chat. Triggers eventos. Flujos: Onboarding, Diario comidas, Receta IA, Carrito, Finanzas, Inventario, Feed. APIS: /api/food-search, /api/recipe-ai, /api/chat, /api/sync, /api/auth. Performance: Debounce 1800ms, Caché, Image optimization, Code splitting, Memoization. Testing: Jest (cálculos) | React Testing Library (componentes) | Cypress (E2E). Deployment: Vercel + Supabase Cloud. Monitoreo: Supabase Analytics, Vercel Analytics, Error tracking, Rate limiting. Roadmap: MVP (hoy) | Fase 2 (Nordigen, water_log) | Fase 3 (rutinas, comunidad) | Fase 4+ (premium, marketplace).

Sección 79: Análisis Técnico de FoodOS

FoodOS es una plataforma unificada que integra gestión de inventario, nutrición personalizada y finanzas personales. Desarrollada con Next.js 14 (App Router), React 19 (Server Components), TypeScript 5.3+, Tailwind CSS 3, y Supabase PostgreSQL backend. Arquitectura: Cliente (SPA) → localStorage (JSON) → Supabase PostgreSQL. Debounce 1800ms agrupa cambios. Pull completo al login. Push con upsert+delete. RLS nativo en todas tablas. Conflictos: last-write-wins. Stack: Frontend (Next.js, React, TypeScript, Tailwind, Context+Immer) | Backend (Supabase, PostgreSQL 15+, Auth) | APIs (Open Food Facts, Gemini, OpenAI, Anthropic, Nordigen, USDA) | Hosting (Vercel, Supabase Cloud). Base de Datos: 39 tablas PostgreSQL con RLS. Categorías: catálogos (mascots), usuarios (user_profiles), inventario (almacenes, inventory_items), recetas (recipes, recipe_ingredients), nutrición (nutrition_goals, food_log), compras (shopping_lists), finanzas (gastos, ingresos), feed (feed_posts), IA (ai_events, ai_recipe_cache), otros. Seguridad: Auth (Google OAuth, Email/Password bcrypt, Magic Link). RLS en todas tablas. Keys IA en localStorage (nunca servidor). Validación TypeScript + server. CORS policy. Auditoría ai_events. Cálculos Nutricionales: TMB Mifflin-St Jeor (10×kg + 6.25×cm - 5×edad ± 5/-161). TDEE = TMB × factor (1.2-1.9). Peso base = ESPEN ajuste. Macros: fat_loss 80%/2.0g/kg, muscle_gain 105%/1.8g/kg, recomp 83-90%/2.0g/kg (ciclado), maintain 100%/1.8g/kg. Distribución: proteína → grasa → carbo. Componentes: DashboardShell (layout) | OnboardingFlow (5 pasos) | 16+ Modales | 13+ Vistas. IA: Gemini 1.5 Flash (1500 req/día gratis) | OpenAI | Anthropic | Ollama. Rate limit 15 req/min. Caché 6h. Auditoría ai_events. Búsqueda: 3 capas (local → Open Food Facts → USDA). Caché 30 días. Autocompletado. Mascotas: 15 personajes con 8 estados animables. Integración chat. Triggers eventos. Flujos: Onboarding, Diario comidas, Receta IA, Carrito, Finanzas, Inventario, Feed. APIS: /api/food-search, /api/recipe-ai, /api/chat, /api/sync, /api/auth. Performance: Debounce 1800ms, Caché, Image optimization, Code splitting, Memoization. Testing: Jest (cálculos) | React Testing Library (componentes) | Cypress (E2E). Deployment: Vercel + Supabase Cloud. Monitoreo: Supabase Analytics, Vercel Analytics, Error tracking, Rate limiting. Roadmap: MVP (hoy) | Fase 2 (Nordigen, water_log) | Fase 3 (rutinas, comunidad) | Fase 4+ (premium, marketplace).

Sección 80: Análisis Técnico de FoodOS

FoodOS es una plataforma unificada que integra gestión de inventario, nutrición personalizada y finanzas personales. Desarrollada con Next.js 14 (App Router), React 19 (Server Components), TypeScript 5.3+, Tailwind CSS 3, y Supabase PostgreSQL backend. Arquitectura: Cliente (SPA) → localStorage (JSON) → Supabase PostgreSQL. Debounce 1800ms agrupa cambios. Pull completo al login. Push con upsert+delete. RLS nativo en todas tablas. Conflictos: last-write-wins. Stack: Frontend (Next.js, React, TypeScript, Tailwind, Context+Immer) | Backend (Supabase, PostgreSQL 15+, Auth) | APIs (Open Food Facts, Gemini, OpenAI, Anthropic, Nordigen, USDA) | Hosting (Vercel, Supabase Cloud). Base de Datos: 39 tablas PostgreSQL con RLS. Categorías: catálogos (mascots), usuarios (user_profiles), inventario (almacenes, inventory_items), recetas (recipes, recipe_ingredients), nutrición (nutrition_goals, food_log), compras (shopping_lists), finanzas (gastos, ingresos), feed (feed_posts), IA (ai_events, ai_recipe_cache), otros. Seguridad: Auth (Google OAuth, Email/Password bcrypt, Magic Link). RLS en todas tablas. Keys IA en localStorage (nunca servidor). Validación TypeScript + server. CORS policy. Auditoría ai_events. Cálculos Nutricionales: TMB Mifflin-St Jeor (10×kg + 6.25×cm - 5×edad ± 5/-161). TDEE = TMB × factor (1.2-1.9). Peso base = ESPEN ajuste. Macros: fat_loss 80%/2.0g/kg, muscle_gain

105%/1.8g/kg, recomp 83-90%/2.0g/kg (ciclado), maintain 100%/1.8g/kg. Distribución: proteína → grasa → carbos. Componentes: DashboardShell (layout) | OnboardingFlow (5 pasos) | 16+ Modales | 13+ Vistas. IA: Gemini 1.5 Flash (1500 req/día gratis) | OpenAI | Anthropic | Ollama. Rate limit 15 req/min. Caché 6h. Auditoría ai_events. Búsqueda: 3 capas (local → Open Food Facts → USDA). Caché 30 días. Autocompletado. Mascotas: 15 personajes con 8 estados animables. Integración chat. Triggers eventos. Flujos: Onboarding, Diario comidas, Receta IA, Carrito, Finanzas, Inventario, Feed. APIs: /api/food-search, /api/recipe-ai, /api/chat, /api/sync, /api/auth. Performance: Debounce 1800ms, Caché, Image optimization, Code splitting, Memoization. Testing: Jest (cálculos) | React Testing Library (componentes) | Cypress (E2E). Deployment: Vercel + Supabase Cloud. Monitoreo: Supabase Analytics, Vercel Analytics, Error tracking, Rate limiting. Roadmap: MVP (hoy) | Fase 2 (Nordigen, water_log) | Fase 3 (rutinas, comunidad) | Fase 4+ (premium, marketplace).

Sección 81: Análisis Técnico de FoodOS

FoodOS es una plataforma unificada que integra gestión de inventario, nutrición personalizada y finanzas personales. Desarrollada con Next.js 14 (App Router), React 19 (Server Components), TypeScript 5.3+, Tailwind CSS 3, y Supabase PostgreSQL backend. Arquitectura: Cliente (SPA) → localStorage (JSON) → Supabase PostgreSQL. Debounce 1800ms agrupa cambios. Pull completo al login. Push con upsert+delete. RLS nativo en todas tablas. Conflictos: last-write-wins. Stack: Frontend (Next.js, React, TypeScript, Tailwind, Context+Immer) | Backend (Supabase, PostgreSQL 15+, Auth) | APIs (Open Food Facts, Gemini, OpenAI, Anthropic, Nordigen, USDA) | Hosting (Vercel, Supabase Cloud). Base de Datos: 39 tablas PostgreSQL con RLS. Categorías: catálogos (mascots), usuarios (user_profiles), inventario (almacenes, inventory_items), recetas (recipes, recipe_ingredients), nutrición (nutrition_goals, food_log), compras (shopping_lists), finanzas (gastos, ingresos), feed (feed_posts), IA (ai_events, ai_recipe_cache), otros. Seguridad: Auth (Google OAuth, Email/Password bcrypt, Magic Link). RLS en todas tablas. Keys IA en localStorage (nunca servidor). Validación TypeScript + server. CORS policy. Auditoría ai_events. Cálculos Nutricionales: TMB Mifflin-St Jeor (10×kg + 6.25×cm - 5×edad ± 5/-161). TDEE = TMB × factor (1.2-1.9). Peso base = ESPEN ajuste. Macros: fat_loss 80%/2.0g/kg, muscle_gain 105%/1.8g/kg, recomp 83-90%/2.0g/kg (ciclado), maintain 100%/1.8g/kg. Distribución: proteína → grasa → carbos. Componentes: DashboardShell (layout) | OnboardingFlow (5 pasos) | 16+ Modales | 13+ Vistas. IA: Gemini 1.5 Flash (1500 req/día gratis) | OpenAI | Anthropic | Ollama. Rate limit 15 req/min. Caché 6h. Auditoría ai_events. Búsqueda: 3 capas (local → Open Food Facts → USDA). Caché 30 días. Autocompletado. Mascotas: 15 personajes con 8 estados animables. Integración chat. Triggers eventos. Flujos: Onboarding, Diario comidas, Receta IA, Carrito, Finanzas, Inventario, Feed. APIs: /api/food-search, /api/recipe-ai, /api/chat, /api/sync, /api/auth. Performance: Debounce 1800ms, Caché, Image optimization, Code splitting, Memoization. Testing: Jest (cálculos) | React Testing Library (componentes) | Cypress (E2E). Deployment: Vercel + Supabase Cloud. Monitoreo: Supabase Analytics, Vercel Analytics, Error tracking, Rate limiting. Roadmap: MVP (hoy) | Fase 2 (Nordigen, water_log) | Fase 3 (rutinas, comunidad) | Fase 4+ (premium, marketplace).

Sección 82: Análisis Técnico de FoodOS

FoodOS es una plataforma unificada que integra gestión de inventario, nutrición personalizada y finanzas personales. Desarrollada con Next.js 14 (App Router), React 19 (Server Components), TypeScript 5.3+, Tailwind CSS 3, y Supabase PostgreSQL backend. Arquitectura: Cliente (SPA) → localStorage (JSON) → Supabase PostgreSQL. Debounce 1800ms agrupa cambios. Pull completo al login. Push con upsert+delete. RLS nativo en todas tablas. Conflictos: last-write-wins. Stack: Frontend (Next.js, React, TypeScript, Tailwind, Context+Immer) | Backend (Supabase, PostgreSQL 15+, Auth) | APIs (Open Food Facts, Gemini, OpenAI, Anthropic, Nordigen, USDA) | Hosting (Vercel, Supabase Cloud). Base de Datos: 39 tablas PostgreSQL con RLS. Categorías: catálogos (mascots), usuarios (user_profiles), inventario (almacenes, inventory_items), recetas (recipes, recipe_ingredients), nutrición (nutrition_goals, food_log), compras (shopping_lists), finanzas (gastos, ingresos), feed (feed_posts), IA (ai_events, ai_recipe_cache), otros. Seguridad: Auth (Google OAuth, Email/Password bcrypt, Magic Link). RLS en todas tablas. Keys IA en localStorage (nunca servidor). Validación TypeScript + server. CORS policy. Auditoría ai_events. Cálculos Nutricionales: TMB Mifflin-St Jeor (10×kg + 6.25×cm - 5×edad ± 5/-161). TDEE = TMB × factor (1.2-1.9). Peso base = ESPEN ajuste. Macros: fat_loss 80%/2.0g/kg, muscle_gain 105%/1.8g/kg, recomp 83-90%/2.0g/kg (ciclado), maintain 100%/1.8g/kg. Distribución: proteína → grasa → carbos. Componentes: DashboardShell (layout) | OnboardingFlow (5 pasos) | 16+ Modales | 13+ Vistas. IA: Gemini 1.5 Flash (1500 req/día gratis) | OpenAI | Anthropic | Ollama. Rate limit 15 req/min. Caché 6h. Auditoría ai_events. Búsqueda: 3 capas (local → Open Food Facts → USDA). Caché 30 días. Autocompletado. Mascotas: 15 personajes con 8 estados animables. Integración chat. Triggers eventos. Flujos: Onboarding, Diario comidas, Receta IA, Carrito, Finanzas, Inventario, Feed. APIs: /api/food-search, /api/recipe-ai, /api/chat, /api/sync, /api/auth. Performance: Debounce 1800ms, Caché, Image optimization, Code splitting, Memoization. Testing: Jest (cálculos) | React Testing Library (componentes) | Cypress (E2E). Deployment: Vercel + Supabase Cloud. Monitoreo: Supabase Analytics, Vercel Analytics, Error tracking, Rate limiting. Roadmap: MVP (hoy) | Fase 2 (Nordigen, water_log) | Fase 3 (rutinas, comunidad) | Fase 4+ (premium, marketplace).

Sección 83: Análisis Técnico de FoodOS

FoodOS es una plataforma unificada que integra gestión de inventario, nutrición personalizada y finanzas personales. Desarrollada con Next.js 14 (App Router), React 19 (Server Components), TypeScript 5.3+, Tailwind CSS 3, y Supabase PostgreSQL backend. Arquitectura: Cliente (SPA) → localStorage (JSON) → Supabase PostgreSQL. Debounce 1800ms agrupa cambios. Pull completo al login. Push con upsert+delete. RLS nativo en todas tablas. Conflictos: last-write-wins. Stack: Frontend (Next.js, React, TypeScript, Tailwind, Context+Immer) | Backend (Supabase, PostgreSQL 15+, Auth) | APIs (Open Food Facts, Gemini, OpenAI, Anthropic, Nordigen, USDA) | Hosting (Vercel, Supabase Cloud). Base de Datos: 39 tablas PostgreSQL con RLS. Categorías: catálogos (mascots), usuarios (user_profiles), inventario (almacenes, inventory_items), recetas (recipes, recipe_ingredients), nutrición (nutrition_goals, food_log), compras (shopping_lists), finanzas (gastos, ingresos), feed (feed_posts), IA (ai_events, ai_recipe_cache), otros. Seguridad: Auth (Google OAuth, Email/Password bcrypt, Magic Link). RLS en todas tablas. Keys IA en localStorage (nunca servidor). Validación TypeScript + server. CORS policy. Auditoría ai_events. Cálculos Nutricionales: TMB Mifflin-St Jeor (10×kg + 6.25×cm - 5×edad ± 5/-161). TDEE = TMB × factor (1.2-1.9). Peso base = ESPEN ajuste. Macros: fat_loss 80%/2.0g/kg, muscle_gain 105%/1.8g/kg, recomp 83-90%/2.0g/kg (ciclado), maintain 100%/1.8g/kg. Distribución: proteína → grasa → carbos. Componentes: DashboardShell (layout) | OnboardingFlow (5 pasos) | 16+ Modales | 13+ Vistas. IA: Gemini 1.5 Flash (1500 req/día gratis) | OpenAI | Anthropic | Ollama. Rate limit 15 req/min. Caché 6h. Auditoría ai_events. Búsqueda: 3 capas (local → Open Food Facts → USDA). Caché 30 días. Autocompletado. Mascotas: 15 personajes con 8 estados animables. Integración chat. Triggers eventos. Flujos: Onboarding, Diario comidas, Receta IA, Carrito, Finanzas, Inventario, Feed. APIs: /api/food-search, /api/recipe-ai, /api/chat, /api/sync, /api/auth. Performance: Debounce 1800ms, Caché, Image optimization, Code splitting, Memoization. Testing: Jest (cálculos) | React Testing Library (componentes) | Cypress (E2E). Deployment: Vercel + Supabase Cloud. Monitoreo: Supabase Analytics, Vercel Analytics, Error tracking, Rate limiting. Roadmap: MVP (hoy) | Fase 2 (Nordigen, water_log) | Fase 3 (rutinas, comunidad) | Fase 4+ (premium, marketplace).

Sección 84: Análisis Técnico de FoodOS

FoodOS es una plataforma unificada que integra gestión de inventario, nutrición personalizada y finanzas personales. Desarrollada con Next.js 14 (App Router), React 19 (Server Components), TypeScript 5.3+, Tailwind CSS 3, y Supabase PostgreSQL backend. Arquitectura: Cliente (SPA) → localStorage (JSON) → Supabase PostgreSQL. Debounce 1800ms agrupa cambios. Pull completo al login. Push con upsert+delete. RLS nativo en todas tablas. Conflictos: last-write-wins. Stack: Frontend (Next.js, React, TypeScript, Tailwind, Context+Immer) | Backend (Supabase, PostgreSQL 15+, Auth) | APIs (Open Food Facts, Gemini, OpenAI, Anthropic, Nordigen, USDA) | Hosting (Vercel, Supabase Cloud). Base de Datos: 39 tablas PostgreSQL con RLS. Categorías: catálogos (mascots), usuarios (user_profiles), inventario (almacenes, inventory_items), recetas (recipes, recipe_ingredients), nutrición (nutrition_goals, food_log), compras (shopping_lists), finanzas (gastos, ingresos), feed (feed_posts), IA (ai_events, ai_recipe_cache), otros. Seguridad: Auth (Google OAuth, Email/Password bcrypt, Magic Link). RLS en todas tablas. Keys IA en localStorage (nunca servidor). Validación TypeScript + server. CORS policy. Auditoría ai_events. Cálculos Nutricionales: TMB Mifflin-St Jeor (10×kg + 6.25×cm - 5×edad ± 5/-161). TDEE = TMB × factor (1.2-1.9). Peso base = ESPEN ajuste. Macros: fat_loss 80%/2.0g/kg, muscle_gain 105%/1.8g/kg, recomp 83-90%/2.0g/kg (ciclado), maintain 100%/1.8g/kg. Distribución: proteína → grasa → carbos. Componentes: DashboardShell (layout) | OnboardingFlow (5 pasos) | 16+ Modales | 13+ Vistas. IA: Gemini 1.5 Flash (1500 req/día gratis) | OpenAI | Anthropic | Ollama. Rate limit 15 req/min. Caché 6h. Auditoría ai_events. Búsqueda: 3 capas (local → Open Food Facts → USDA). Caché 30 días. Autocompletado. Mascotas: 15 personajes con 8 estados animables. Integración chat. Triggers eventos. Flujos: Onboarding, Diario comidas, Receta IA, Carrito, Finanzas, Inventario, Feed. APIs: /api/food-search, /api/recipe-ai, /api/chat, /api/sync, /api/auth. Performance: Debounce 1800ms, Caché, Image optimization, Code splitting, Memoization. Testing: Jest (cálculos) | React Testing Library (componentes) | Cypress (E2E). Deployment: Vercel + Supabase Cloud. Monitoreo: Supabase Analytics, Vercel Analytics, Error tracking, Rate limiting. Roadmap: MVP (hoy) | Fase 2 (Nordigen, water_log) | Fase 3 (rutinas, comunidad) | Fase 4+ (premium, marketplace).

Sección 85: Análisis Técnico de FoodOS

FoodOS es una plataforma unificada que integra gestión de inventario, nutrición personalizada y finanzas personales. Desarrollada con Next.js 14 (App Router), React 19 (Server Components), TypeScript 5.3+, Tailwind CSS 3, y Supabase PostgreSQL backend. Arquitectura: Cliente (SPA) → localStorage (JSON) → Supabase PostgreSQL. Debounce 1800ms agrupa cambios. Pull completo al login. Push con upsert+delete. RLS nativo en todas tablas. Conflictos: last-write-wins. Stack: Frontend (Next.js, React, TypeScript, Tailwind, Context+Immer) | Backend (Supabase, PostgreSQL 15+, Auth) | APIs (Open Food Facts, Gemini, OpenAI, Anthropic, Nordigen, USDA) | Hosting (Vercel, Supabase Cloud). Base de Datos: 39 tablas PostgreSQL con RLS. Categorías: catálogos (mascots), usuarios (user_profiles), inventario (almacenes, inventory_items), recetas (recipes, recipe_ingredients), nutrición (nutrition_goals, food_log), compras (shopping_lists), finanzas (gastos, ingresos), feed (feed_posts), IA (ai_events, ai_recipe_cache), otros. Seguridad: Auth (Google OAuth, Email/Password bcrypt, Magic Link). RLS en todas tablas. Keys IA en localStorage (nunca servidor). Validación TypeScript + server. CORS policy. Auditoría ai_events. Cálculos Nutricionales: TMB Mifflin-St Jeor (10×kg + 6.25×cm - 5×edad ± 5/-161). TDEE = TMB × factor (1.2-1.9). Peso base = ESPEN ajuste. Macros: fat_loss 80%/2.0g/kg, muscle_gain 105%/1.8g/kg, recomp 83-90%/2.0g/kg (ciclado), maintain 100%/1.8g/kg. Distribución: proteína → grasa → carbos. Componentes: DashboardShell (layout) | OnboardingFlow (5 pasos) | 16+ Modales | 13+ Vistas. IA: Gemini 1.5 Flash (1500 req/día gratis) | OpenAI | Anthropic | Ollama. Rate limit 15 req/min. Caché 6h. Auditoría ai_events. Búsqueda: 3 capas (local → Open Food Facts → USDA). Caché 30 días. Autocompletado. Mascotas: 15 personajes con 8 estados animables. Integración chat. Triggers eventos. Flujos: Onboarding, Diario comidas, Receta IA, Carrito, Finanzas, Inventario, Feed. APIs: /api/food-search, /api/recipe-ai, /api/chat, /api/sync, /api/auth. Performance: Debounce 1800ms, Caché, Image optimization, Code splitting, Memoization. Testing: Jest (cálculos) | React Testing Library (componentes) | Cypress (E2E). Deployment: Vercel + Supabase Cloud. Monitoreo: Supabase Analytics, Vercel Analytics, Error tracking, Rate limiting. Roadmap: MVP (hoy) | Fase 2 (Nordigen, water_log) | Fase 3 (rutinas, comunidad) | Fase 4+ (premium, marketplace).

FIN - DOCUMENTACIÓN v9.0

85+ páginas | Auditoría técnica exhaustiva | FoodOS | Junio 2026